

TUGAS AKHIR

PENGEMBANGAN APLIKASI MOBILE CEK KEBENARAN KLAIM DI MEDIA SOSIAL BERBASIS AI DAN *CUSTOM SEARCH ENGINE*

PROPOSAL PENELITIAN

Diajukan untuk memenuhi persyaratan kelulusan Politeknik Negeri Media Kreatif



Disusun oleh:

MUHAMAD ALFATH ARRAHMAN

NIM: 2290343055

PROGRAM STUDI TEKNOLOGI REKAYASA MULTIMEDIA

JURUSAN DESAIN

POLITEKNIK NEGERI MEDIA KREATIF

JAKARTA

2026

KATA PENGANTAR

Puji syukur ke hadirat Tuhan Yang Maha Esa atas segala rahmat, hidayah, dan karunia-Nya yang melimpah, sehingga penulis dapat menyelesaikan penyusunan proposal penelitian yang berjudul *"Pengembangan Aplikasi Mobile Berbasis Google Custom Search Engine dan Gemini AI untuk Cek Fakta Klaim di Media Sosial"* ini dengan baik.

Proposal ini tidak dapat selesai tanpa bantuan dan dukungan dari:

1. Dr. Tipri Rose Kartika, M.M., Direktur Politeknik Negeri Media Kreatif sekaligus pembimbing penulisan Proposal Tugas Akhir.
2. Dr. Handika Dany Rahmayanti, M.Si., Wakil Direktur Bidang Akademik.
3. Tri Fajar Yurmama Supiyanti, selaku Ketua Jurusan Desain
4. Lani Siti Noor Aisyah, S.Ds., M.Ds., Sekretaris Jurusan Desain
5. Sanjaya Pinem, S.Kom., M.Sc. selaku Koordinator Program Studi Teknologi Rekayasa Multimedia
6. Para dosen dan tenaga kependidikan Politeknik Negeri Media Kreatif yang telah melayani mahasiswa selama penulis menempuh pendidikan di sini.

Penulis menyadari masih banyak kekurangan dalam tugas akhir ini. Oleh sebab itu, penulis mengharapkan saran dan kritik yang membangun untuk tugas akhir ini

Depok, 28 Januari 2026

Penulis,

Muhamad Alfath Arrahman

NIM. 2290343055

DAFTAR ISI

PENGEMBANGAN APLIKASI MOBILE CEK KEBENARAN KLAIM DI MEDIA SOSIAL BERBASIS AI DAN CUSTOM SEARCH ENGINE.....	1
KATA PENGANTAR.....	2
DAFTAR ISI.....	3
DAFTAR TABEL.....	4
DAFTAR GAMBAR.....	6
DAFTAR LAMPIRAN.....	7
RINGKASAN.....	1
BAB I	
PENDAHULUAN.....	1
A. Latar Belakang Penelitian.....	1
B. Identifikasi Masalah Penelitian.....	4
C. Batasan Masalah.....	5
D. Rumusan Masalah.....	6
E. Tujuan Penelitian.....	6
F. Manfaat Penelitian.....	7
BAB II	
KAJIAN PUSTAKA.....	8
A. Kajian Pustaka.....	8
B. Hasil Penelitian yang Relevan.....	22
C. Kerangka Berpikir.....	24
BAB III	
METODE PENELITIAN.....	27
A. Metode dan Desain Penelitian.....	27
B. Subjek Penelitian.....	32
C. Teknik dan Instrumen Pengumpulan Data.....	35
D. Teknik Analisis Data.....	41
BAB IV	
HASIL KAJIAN DAN PEMBAHASAN.....	48
A. Hasil Kajian.....	48
DAFTAR PUSTAKA.....	134
LAMPIRAN.....	142

DAFTAR TABEL

a. Fenomena Clickbait dan Manipulasi Kognitif.....	8
b. Ekosistem Buzzer dan Disinformasi Terorganisir.....	9
c. Perilaku Pengguna dan Rendahnya Budaya Verifikasi.....	10
a. Urgensi Transisi ke Verifikasi Otomatis.....	10
b. Pendekatan Berbasis Agen (Agentic Approach) dalam Cek Fakta.....	11
a. Google Custom Search Engine (CSE).....	12
b. Gemini AI.....	12
a. Aplikasi Mobile dan Android Studio.....	13
b. Framework Flutter.....	14
c. Bahasa Pemrograman Dart.....	16
d. Konsep CRUD (Create, Read, Update, Delete).....	16
a. Konsep Dasar SQLite.....	17
b. Implementasi SQLite pada Framework Flutter.....	18
c. Relevansi SQLite terhadap aplikasi ini.....	18
b. Pengukuran Menggunakan User Experience Questionnaire (UEQ).....	20
Table 1. Hasil Penelitian yang Relevan.....	23

DAFTAR GAMBAR

Table 1. Hasil Penelitian yang Relevan.....	24
Gambar 1. Kerangka Berpikir.....	26
Gambar 1. Kerangka Berpikir.....	31
Table 2. Jadwal Penelitian.....	35
Table 3. Skenario Pengujian Kinerja (Task Scenario).....	40
Gambar 1. Use Case Diagram Aplikasi Klarip.....	51
Gambar 1. Flowchart Sistem Aplikasi Klarip.....	52
Gambar 1. Flowchart Sistem Aplikasi Klarip.....	56
Gambar 1. Skenario REGISTER (Create Akun).....	59
Gambar 1. Skenario LOGIN (Read Session).....	60
Gambar 1. Skenario READ (Memuat Daftar Riwayat).....	61
Gambar 1. Skenario UPDATE NOTE (Edit Catatan).....	62
Gambar 1. Skenario DELETE (Hapus Koleksi).....	63
Gambar 1. Halaman Masuk dan Halaman Daftar (Login).....	64
Gambar 1. Halaman Masuk dan Halaman Daftar (Login).....	66
Gambar 1. Halaman Masuk dan Halaman Daftar (Login).....	66
Gambar 1. Halaman Halaman Riwayat.....	67
Gambar 1. Halaman Halaman Riwayat.....	67
Gambar 1. Halaman Halaman Riwayat.....	68

Gambar 1. Halaman Halaman Riwayat.....	69
Gambar 1. Halaman Halaman Riwayat.....	70
Gambar 1. Halaman Halaman Riwayat.....	72
Gambar 1. Halaman Halaman Riwayat.....	73
Gambar 1. Halaman Halaman Riwayat.....	75
Gambar 1. Halaman Halaman Riwayat.....	77
Gambar 1. Halaman Halaman Riwayat.....	78
Gambar 1. Halaman Halaman Riwayat.....	80
Gambar 1. Halaman Halaman Riwayat.....	81
Gambar 1. Halaman Halaman Riwayat.....	83
Gambar 1. Halaman Halaman Riwayat.....	86
Gambar 1. Halaman Halaman Riwayat.....	88
Gambar 1. Halaman Halaman Riwayat.....	90
Gambar 1. Halaman Halaman Riwayat.....	93
Gambar 1. Halaman Halaman Riwayat.....	95
Gambar 1. Halaman Halaman Riwayat.....	97
Gambar 1. Halaman Halaman Riwayat.....	99
Gambar 1. Halaman Halaman Riwayat.....	101
Gambar 1. Halaman Halaman Riwayat.....	103
Gambar 1. Halaman Halaman Riwayat.....	104

DAFTAR LAMPIRAN

BIODATA MAHASISWA.....	143
------------------------	-----

RINGKASAN

Pesatnya perkembangan teknologi informasi menjadikan media sosial sebagai sumber berita utama bagi kita, namun hal ini juga memicu maraknya penyebaran disinformasi, seperti *clickbait* dan hoaks. Metode verifikasi manual yang ada saat ini dinilai tidak efisien karena membutuhkan waktu yang lama, sehingga informasi palsu sering kali menyebar lebih cepat daripada klarifikasinya. Oleh karena itu, pada proposal penelitian ini, dikembangkan sebuah solusi teknologi berupa aplikasi mobile yang mengintegrasikan *Google Custom Search Engine* (CSE) dan AI untuk mengotomatisasi dan mempercepat proses cek fakta. Penelitian ini menggunakan metode Kuantitatif dengan pendekatan *Research and Development* (R&D), serta mengadaptasi model SDLC (*Software Development Life Cycle*) sebagai metode pengembangan sistem. Subjek penelitian difokuskan pada pengguna media sosial aktif dari kalangan generasi Z dan milenial (usia 15–35 tahun). Pengembangan aplikasi dilakukan menggunakan *framework* Flutter dan bahasa pemrograman Dart untuk memastikan performa yang responsif. Teknik pengumpulan data dilakukan melalui observasi eksperimental *Time on Task* untuk mengukur efisiensi waktu verifikasi dibandingkan metode manual, serta kuesioner *System Usability Scale* (SUS) untuk mengukur tingkat usability aplikasi. Analisis data dilakukan secara kuantitatif melalui perbandingan statistik durasi verifikasi guna membuktikan efisiensi sistem, diikuti dengan analisis deskriptif terhadap skor SUS untuk menentukan klasifikasi tingkat penerimaan pengguna, sehingga seluruh data yang diperoleh dapat merepresentasikan kinerja serta kelayakan aplikasi Klarip secara komprehensif.

BAB I

PENDAHULUAN

A. Latar Belakang Penelitian

Evolusi teknologi informasi dan komunikasi telah menempatkan media sosial sebagai saluran utama bagi masyarakat dalam mengakses berita dan membentuk opini publik. Namun, kemudahan akses ini membawa tantangan serius berupa degradasi kualitas informasi di ruang digital. Salah satu fenomena yang paling mencolok adalah maraknya penggunaan judul sensasional atau *clickbait*. Taktik ini muncul sebagai adaptasi terhadap lingkungan media yang dikurasi oleh algoritma, di mana pembuat konten terdorong untuk merancang "gangguan adiktif" demi memaksimalkan metrik popularitas dan keterlibatan pengguna (*engagement*). Pada akhirnya, pengejaran tingkat interaksi (*engagement*) tinggi ini tidak hanya mengorbankan kualitas dan kedalaman informasi, tetapi juga memfasilitasi penyebaran klaim yang menyesatkan dan disinformasi, yang secara akumulatif dapat merusak kepercayaan publik terhadap ekosistem informasi digital (Lischka & Garz, 2021). Dampak langsung dari ekosistem yang manipulatif ini secara nyata merugikan pengguna media sosial, khususnya Generasi Z. Karena terbiasa dengan ritme konsumsi informasi yang serba cepat atau instant, pengguna

seringkali merespons secara emosional hanya berdasarkan judul sensasional tanpa membaca isi utuh. Akibatnya, pengguna tidak hanya menjadi korban kebingungan informasi, tetapi juga tanpa sadar bertindak sebagai agen penyebar hoaks dengan *repost* di lingkaran sosial mereka. Fakta mengkhawatirkan ini terkonfirmasi oleh temuan di Indonesia, di mana 71,4% pengguna internet terbukti tidak pernah melakukan verifikasi atau memeriksa sumber asli sebelum membagikan sebuah postingan (Effendi, 2023)

Krisis literasi digital dan rendahnya skeptisisme pengguna semakin memperburuk kondisi ekosistem informasi tersebut. Fakta mengkhawatirkan terungkap dalam penelitian Effendi (2023), di mana sebanyak 71,4% responden mengaku pernah secara tidak sengaja menyebarkan informasi palsu atau hoaks di media sosial. Meskipun mayoritas pengguna memiliki kesadaran akan adanya informasi menyesatkan, kerentanan ini tetap tinggi karena hanya sebagian kecil yang melakukan verifikasi secara konsisten, sementara sisanya masih terjebak dalam pola konsumsi informasi yang instan. Kerentanan massal inilah yang menjadi celah ideal untuk dieksploitasi oleh aktor jahat. Saat ini, penyebaran hoaks bukan lagi sekadar insiden acak, melainkan sebuah operasi disinformasi yang digerakkan secara sistematis oleh pasukan siber atau *buzzer* terorganisir demi memanipulasi opini publik (Wijayanto et al., 2021).

Di tengah derasny arus informasi yang dimanipulasi tersebut, metode verifikasi fakta (*fact-checking*) konvensional menghadapi jalan buntu. Mekanisme validasi manual yang mengandalkan tenaga manusia dianggap tidak lagi mencukupi (*insufficient*) untuk menandingi volume informasi baru yang muncul setiap saat serta kecepatan penyebarannya yang sangat tinggi (Guo, Schlichtkrull, & Vlachos, 2022). Ketidakseimbangan antara kecepatan produksi hoaks dan lambatnya proses verifikasi manual menciptakan celah waktu (*time gap*), di mana informasi palsu dapat menyebar luas sebelum sempat dikoreksi oleh ahli. Metode manual tidak lagi mampu mengejar kecepatan disinformasi yang menyebar dalam hitungan detik, sehingga otomatisasi verifikasi bukan lagi sekadar pilihan, melainkan kebutuhan mendesak untuk menyelamatkan ekosistem digital.

Untuk mengatasi kesenjangan efisiensi tersebut, diperlukan pendekatan teknologi yang mampu mengotomatisasi proses verifikasi. Riset terbaru menunjukkan bahwa pendekatan berbasis agen (*agentic approach*) menggunakan *Large Language Model* (LLM) memungkinkan sistem untuk meniru perilaku ahli cek fakta manusia. Teknologi ini bekerja dengan memecah tugas verifikasi yang kompleks menjadi beberapa sub-langkah sederhana, di mana LLM menyelesaikan tugas tersebut menggunakan pengetahuan internalnya atau memanfaatkan alat eksternal seperti mesin pencari (Li, Zhang, & Malthouse, 2024).

Berdasarkan urgensi tersebut, penelitian ini bertujuan mengembangkan sebuah sistem aplikasi mobile yang mengintegrasikan *Google Custom Search Engine* (CSE) dan AI. Solusi ini ditawarkan untuk menyederhanakan dan mempercepat proses verifikasi klaim di media sosial, mengubah proses yang semula manual dan lambat menjadi otomatis dan efisien. Implementasi teknologi ini menjadi sangat krusial untuk dilakukan saat ini, karena di tengah ledakan informasi yang dimanipulasi secara sistematis, ketiadaan alat bantu yang responsif hanya akan memperdalam krisis kepercayaan publik dan membiarkan Generasi Z terus terpapar dampak destruktif hoaks. Oleh karena itu, kehadiran aplikasi ini bukan sekadar inovasi teknis, melainkan langkah strategis yang mendesak untuk memperkuat benteng literasi digital di tengah ekosistem informasi yang semakin kompleks.

B. Identifikasi Masalah Penelitian

Berdasarkan paparan latar belakang di atas, dapat diidentifikasi sejumlah masalah krusial yang menjadi fokus penelitian ini.

1. Eksploitasi judul *clickbait* demi mendongkrak metrik interaksi di media sosial telah menggeser fokus dari kedalaman informasi ke popularitas instan. Kondisi ini mengakibatkan audiens rentan terpapar oleh klaim-klaim yang tidak terverifikasi dan hoaks, yang secara kumulatif dapat merusak kepercayaan terhadap ekosistem informasi digital (Lischka & Garz, 2021).

2. Terdapat kerentanan di mana 71,4% pengguna internet di Indonesia tidak pernah melakukan verifikasi atau memeriksa sumber asli sebelum membagikan sebuah postingan (Effendi, 2023). Hal ini diperparah oleh ekosistem *buzzer* yang menggerakkan disinformasi secara terorganisir untuk memanipulasi opini publik (Wijayanto et al., 2021).
3. Validasi manual yang ada saat ini dianggap tidak lagi mencukupi untuk menghadapi volume dan kecepatan penyebaran informasi baru yang sangat masif di media sosial (Guo et al., 2022).

C. Batasan Masalah

Agar pengembangan sistem aplikasi tetap terarah pada tujuan utama yaitu mempermudah dan mempercepat pengguna dalam memverifikasi informasi, penulis menetapkan batasan masalah sebagai berikut:

1. Penelitian ini hanya difokuskan pada deteksi klaim berbasis teks, seperti yang terdapat pada *caption* atau judul berita di media sosial. Analisis terhadap konten video kompleks atau *deepfake* tidak disertakan.
2. Pengujian sistem sepenuhnya difokuskan pada aspek efisiensi waktu (*verification efficiency*) dan kegunaan (*usability*). Penulis hanya mengukur seberapa cepat sistem mampu memproses verifikasi klaim dibandingkan dengan metode penelusuran manual. Penelitian ini tidak melakukan pengujian terhadap kualitas

substansi, kebenaran absolut, maupun akurasi mendalam dari konten fakta yang dihasilkan oleh AI dan hasil pencarian dari beberapa sumber berita terpilih, melainkan hanya pada percepatan alur kerjanya.

D. Rumusan Masalah

Berdasarkan latar belakang dan identifikasi masalah yang telah diuraikan, rumusan masalah utama dalam penelitian ini adalah:

1. Bagaimana merancang dan membangun aplikasi mobile dengan mengintegrasikan *Google Custom Search Engine* dan Gemini AI untuk mengotomatisasi proses verifikasi klaim di media sosial?
2. Sejauh mana tingkat efektivitas aplikasi ini dalam meningkatkan efisiensi waktu verifikasi informasi jika dibandingkan dengan metode verifikasi manual pada responden Generasi Z?
3. Bagaimana tingkat kegunaan (*usability*) aplikasi ini bagi pengguna Generasi Z diukur menggunakan metode *System Usability Scale* (SUS)?

E. Tujuan Penelitian

Penelitian ini memiliki tujuan yang jelas dan terukur sebagai berikut:

1. Merancang dan membangun aplikasi mobile yang mengintegrasikan *Google Custom Search Engine* dan Gemini AI sebagai solusi otomatisasi verifikasi klaim di media sosial.

2. Menganalisis dan membandingkan efisiensi waktu verifikasi informasi antara penggunaan aplikasi dengan metode verifikasi manual pada responden Generasi Z.
3. Mengukur dan mengevaluasi tingkat kegunaan (*usability*) aplikasi menggunakan metode *System Usability Scale* (SUS) untuk memastikan aplikasi ini layak dan sesuai dengan kebutuhan pengguna Generasi Z.

F. Manfaat Penelitian

Hasil dari penelitian ini diharapkan dapat memberikan kontribusi nyata dalam ekosistem informasi digital, antara lain:

1. Memberikan kemampuan bagi pengguna untuk memverifikasi klaim mencurigakan dalam hitungan detik, sehingga jauh lebih hemat waktu dibandingkan melakukan riset manual.

BAB II

KAJIAN PUSTAKA

A. Kajian Pustaka

1. Disinformasi, *Clickbait*, dan Perilaku Pengguna di Media Sosial

a. Fenomena *Clickbait* dan Manipulasi Kognitif

Dalam ekosistem media sosial kontemporer, platform secara aktif menggunakan kurasi algoritma yang mengharuskan pembuat konten untuk terus menyesuaikan format publikasi mereka. Algoritma ini bekerja dengan memberikan penghargaan pada metrik interaksi sosial, seperti jumlah like dan share, yang kini berfungsi sebagai "mata uang online" sekaligus indikator popularitas. Kondisi ini menginisiasi sebuah persaingan di ruang digital, di mana berbagai penyedia informasi saling berlomba untuk memaksimalkan atensi pengguna. Demi merebut perhatian di tengah arus informasi yang padat, para pembuat konten secara kolektif terdorong untuk merancang "distraksi adiktif" atau addictive distractions (Lischka & Garz, 2021).

Bentuk paling umum dari distraksi adiktif tersebut adalah penggunaan judul sensasional atau *clickbait*. Secara konseptual, *clickbait* beroperasi sebagai sebuah strategi linguistik yang secara sengaja mengartikulasikan pesan

untuk membangkitkan rasa ingin tahu audiens, sehingga memancing mereka untuk mengklik tautan yang diberikan. Taktik manipulasi teks ini tidak hanya digunakan oleh media arus utama, tetapi juga diadopsi secara luas oleh situs-situs penyebar hoaks untuk mendistribusikan disinformasi secara masif kepada publik. Lebih jauh, meskipun *clickbait* dirancang untuk mendulang klik, pengguna yang terlalu sering menghadapi kualitas informasi yang mengecewakan di balik judul tersebut akan cenderung menghindari konten serupa di masa depan. Pada akhirnya, praktik ini justru mempertaruhkan reputasi penyebar informasi dan merusak kepercayaan pengguna secara akumulatif (Lischka & Garz, 2021).

b. Ekosistem *Buzzer* dan Disinformasi Terorganisir

Disinformasi di media sosial tidak selalu terjadi secara organik atau kebetulan. Penelitian mengidentifikasi bahwa penyebaran informasi palsu sering kali digerakkan oleh pasukan siber (*cyber troops*) atau *buzzer* yang bekerja secara terorganisir. Kelompok ini biasanya dibiayai oleh elite tertentu dengan tujuan sistematis untuk memanipulasi opini publik (Wijayanto et al., 2021). Melalui mobilisasi

yang terstruktur, disinformasi ini menjadi ancaman serius di tengah masyarakat.

c. Perilaku Pengguna dan Rendahnya Budaya Verifikasi

Efektivitas penyebaran *clickbait* dan narasi *buzzer* sangat bergantung pada perilaku pengguna media sosial itu sendiri. Studi kasus empiris di Indonesia mengungkapkan adanya kerentanan perilaku yang signifikan, di mana 71,4% pengguna internet tercatat tidak pernah melakukan verifikasi atau memeriksa sumber asli sebelum membagikan sebuah postingan (Effendi, 2023).

2. Verifikasi Fakta dan Otomatisasi

a. Urgensi Transisi ke Verifikasi Otomatis

Secara tradisional, verifikasi fakta dilakukan oleh jurnalis atau ahli independen melalui penelusuran manual. Namun, ekosistem informasi digital saat ini menghadirkan tantangan volume dan kecepatan yang belum pernah terjadi sebelumnya. Studi komprehensif mengenai cek fakta otomatis menegaskan bahwa validasi manual tidak lagi mencukupi (*insufficient*) untuk mengimbangi jumlah informasi baru yang muncul setiap saat serta kecepatan penyebarannya di media sosial (Guo, Schlichtkrull, & Vlachos, 2022).

Keterbatasan kapasitas manusia dalam memproses data secara *real-time* menciptakan kesenjangan waktu (*latency*), di mana informasi palsu sering kali sudah menyebar luas sebelum verifikator manusia sempat membuktikannya salah. Oleh karena itu, pengembangan sistem verifikasi otomatis menjadi kebutuhan mendesak untuk mendeteksi pola misinformasi secara cepat dan terukur.

b. Pendekatan Berbasis Agen (*Agentic Approach*) dalam Cek Fakta

Dalam upaya meningkatkan akurasi verifikasi otomatis, riset terkini memperkenalkan pendekatan berbasis agen (*agentic approach*) menggunakan *Large Language Models* (LLM). Pendekatan ini memungkinkan model AI untuk meniru perilaku ahli cek fakta manusia (*emulate human expert behavior*) dalam memverifikasi klaim berita (Li, Zhang, & Malthouse, 2024).

Dalam proses ini, LLM bertindak sebagai agen cerdas yang menyelesaikan tugas-tugas sederhana tersebut dengan menggabungkan pengetahuan internalnya dan kemampuan menggunakan alat eksternal, seperti mesin

pencari (*search engines*), untuk mencari bukti pendukung yang valid (Li, Zhang, & Malthouse, 2024).

3. Google Custom Search Engine (CSE) dan Gemini AI

a. Google Custom Search Engine (CSE)

Dalam arsitektur sistem verifikasi fakta otomatis, keakuratan sumber data adalah prioritas utama. Google Custom Search Engine (CSE) didefinisikan sebagai alat pencarian kustom berbasis Google yang memungkinkan pengembang untuk menyaring dan menentukan sumber-sumber spesifik yang dianggap terpercaya. Berbeda dengan mesin pencari standar, CSE melampaui sekadar pencocokan kata kunci; alat ini menggunakan algoritma AI untuk memastikan hasil pencarian yang relevan dari daftar *whitelist* yang telah ditentukan (Google Developers, 2025). Hal ini memastikan bahwa sistem hanya mengambil referensi dari portal berita terpilih dan situs resmi pemerintah, demi meminimalisir risiko "halusinasi" data dari sumber yang tidak valid.

b. Gemini AI

Gemini AI adalah keluarga model kecerdasan buatan multimodal yang dikembangkan oleh Google DeepMind. Berbeda dengan model bahasa tradisional yang

hanya dilatih pada teks, Gemini dirancang sejak awal (*natively multimodal*) untuk memahami, mengoperasikan, dan menggabungkan berbagai jenis informasi termasuk teks, kode, audio, gambar, dan video secara simultan (Gemini Team, 2024).

Dalam laporan teknis terbarunya, Google DeepMind (2025) menyoroti kemampuan penalaran tingkat lanjut (*advanced reasoning*) pada varian Gemini 2.5. Model ini memiliki arsitektur *Mixture-of-Experts* (MoE) yang memungkinkannya memproses konteks yang sangat panjang (*long-context window*) dengan efisiensi tinggi. Kemampuan ini sangat krusial dalam penelitian ini, di mana Gemini berfungsi sebagai "agen analisis" yang harus membaca dan membedah banyak artikel berita pembeding untuk menentukan validitas sebuah klaim.

4. Aplikasi Mobile, Flutter, Dart, Dan Konsep CRUD

a. Aplikasi *Mobile* dan Android Studio

Aplikasi *mobile* merupakan perangkat lunak yang dirancang khusus untuk dijalankan pada perangkat genggam seperti *smartphone* atau tablet. Dalam era transformasi digital, aplikasi jenis ini memungkinkan pengguna untuk berinteraksi dengan layanan digital secara

lebih personal dan fleksibel di mana saja, berbeda dengan aplikasi berbasis web yang memiliki keterbatasan aksesibilitas (Purnamasari & Panjaitan, 2020).

Untuk mendukung pengembangan sistem ini, lingkungan kerja (*Integrated Development Environment/IDE*) yang digunakan adalah Android Studio. IDE ini didefinisikan sebagai perangkat lunak resmi pengembangan Android yang berbasis pada IntelliJ IDEA, menyediakan peralatan lengkap bagi pengembang untuk membangun aplikasi berkualitas tinggi di seluruh jenis perangkat Android (Android Developers, 2025). Android Studio memiliki peran krusial dalam penelitian ini karena menyediakan fitur *lint* untuk mengidentifikasi masalah kinerja serta menjadi lingkungan utama untuk mengintegrasikan API Google Custom Search Engine dan Gemini AI ke dalam arsitektur aplikasi (Android Developers, 2025).

b. Framework Flutter

Flutter adalah UI toolkit sumber terbuka dari Google yang memungkinkan pengembang membangun aplikasi yang dikompilasi secara native dari satu basis kode tunggal (Zelixify, 2025). Dalam konteks pengembangan

sistem, pemilihan Flutter sangat didasarkan pada keunggulannya dalam mempermudah proses pengembangan sekaligus memberikan penghematan waktu dan biaya produksi yang maksimal. Melalui arsitektur cross-platform, pengembang dapat menargetkan perilisan aplikasi untuk sistem operasi Android dan iOS sekaligus hanya dengan satu kali penulisan kode. Pendekatan ini secara drastis memangkas beban kerja dan alokasi biaya produksi, karena menghilangkan keharusan untuk membangun dua aplikasi terpisah atau mempelajari bahasa native dari masing-masing platform (Putri et al., 2023). Lebih lanjut, kemudahan pengembangan semakin difasilitasi oleh kehadiran fitur hot reload. Fitur inovatif ini mempercepat siklus produksi dan pengujian karena mampu memperbarui hasil modifikasi desain maupun logika program dalam kurun waktu kurang dari satu detik. Pengembang dapat mengevaluasi perbaikan bug atau antarmuka secara instan tanpa kehilangan state pada aplikasi yang sedang berjalan, sehingga proses penyelesaian aplikasi menjadi jauh lebih efisien (Setiawan et al., 2022)..

c. Bahasa Pemrograman Dart

Di balik performa Flutter, terdapat bahasa pemrograman Dart yang berfungsi sebagai logika utama aplikasi. Dart adalah bahasa yang dioptimalkan untuk pengembangan sisi klien (*client-optimized*) dan menjadi bahasa inti di balik *framework* Flutter yang memungkinkan aplikasi berjalan dengan performa tinggi pada berbagai sistem operasi (Supiana, 2022).

d. Konsep CRUD (*Create, Read, Update, Delete*)

Dalam pengembangan aplikasi berbasis mobile, pengelolaan data yang dinamis sangat bergantung pada operasi fundamental yang dikenal dengan istilah CRUD (*Create, Read, Update, Delete*). Operasi ini merupakan standar interaksi antara antarmuka aplikasi (*front-end*) dengan basis data (*database*). Pada aplikasi verifikasi fakta ini, implementasi CRUD menjadi pondasi utama untuk fitur manajemen pengguna, yang mencakup pendaftaran akun baru (*Create*), otentikasi login dan pembacaan data profil (*Read*), pembaruan kata sandi atau profil (*Update*), dan penghapusan pencarian dan akun (*Delete*). Lebih lanjut, operasi CRUD juga berperan esensial dalam merekam jejak aktivitas pengguna (*history*). menurut penelitian yang

dilakukan oleh (Dzulfikar & Asriningtias, 2025), keberhasilan eksekusi alur login dan operasi CRUD secara langsung memastikan bahwa seluruh riwayat atau aktivitas pengguna di dalam sistem dapat tersimpan, dikelola, dan dilacak kembali dengan akurat. Integrasi fungsi ini membuktikan bahwa aplikasi tidak hanya bertindak sebagai penampil data statis, melainkan sistem interaktif yang mampu mengamankan dan mengelola data pengguna secara real-time.

5. *Local Database* dan SQLite

a. Konsep Dasar SQLite

Dalam arsitektur pengembangan perangkat lunak *mobile*, manajemen penyimpanan data merupakan faktor krusial yang menentukan kecepatan dan efisiensi aplikasi. SQLite adalah sebuah Sistem Manajemen Basis Data Relasional (RDBMS) yang bersifat *embedded* atau tertanam langsung di dalam aplikasi (Kurnia, 2025). Berbeda dengan sistem basis data konvensional yang memerlukan koneksi ke *server* terpisah (seperti MySQL atau PostgreSQL), SQLite beroperasi tanpa *server* (*serverless*) dan membaca serta menulis data secara langsung ke dalam file penyimpanan lokal di perangkat pengguna. Karakteristik ini

membuat SQLite sangat ringan, cepat, dan menjadi standar industri untuk penyimpanan data terstruktur pada aplikasi berskala kecil hingga menengah (Kurnia, 2025)

b. Implementasi SQLite pada Framework Flutter

Pada pengembangan aplikasi *mobile* lintas platform (*cross-platform*) menggunakan *framework* Flutter, SQLite sering diimplementasikan sebagai solusi basis data lokal (*local database*) melalui *package* atau pustaka pendukung seperti *sqflite*. Kombinasi antara arsitektur Flutter yang reaktif dan kehandalan SQLite terbukti efektif dalam memfasilitasi operasi pengelolaan data dasar atau CRUD (*Create, Read, Update, Delete*) secara instan di sisi pengguna (Suherly & Dilson, 2021). Penggunaan basis data lokal ini memungkinkan aplikasi untuk menyimpan data secara persisten dan memproses permintaan pengguna tanpa harus bergantung sepenuhnya pada latensi jaringan atau koneksi internet yang stabil (Karamoy & Wibowo, 2025).

c. Relevansi SQLite terhadap aplikasi ini

Implementasi SQLite pada aplikasi ini memiliki peran vital dalam menjamin privasi data pengguna, mengingat sensitivitas isu verifikasi informasi di tengah

masyarakat. Dengan menyimpan seluruh riwayat pencarian dan hasil verifikasi secara lokal di dalam direktori privat perangkat, aplikasi memastikan bahwa data sensitif pengguna tidak dikirimkan atau disimpan di server eksternal yang rentan terhadap risiko penyadapan atau kebocoran data. Metode isolasi data ini memberikan kontrol penuh kepada pengguna atas jejak digital mereka, karena data hanya tersimpan secara persisten pada penyimpanan fisik perangkat tanpa melibatkan pihak ketiga (Karamoy & Wibowo, 2025). Pendekatan ini tidak hanya meminimalisir risiko keamanan, tetapi juga menjadi fondasi utama dalam membangun kepercayaan masyarakat terhadap integritas dan kerahasiaan aktivitas cek fakta yang dilakukan melalui aplikasi (Kurnia, 2025).

6. *Usability* dan Efisiensi Sistem Definisi *Usability* (Kebergunaan)

Dalam pengembangan sistem berbasis pengguna, usability atau kebergunaan adalah indikator vital untuk mengukur kualitas interaksi antara manusia dan komputer. Berdasarkan standar internasional, usability didefinisikan sebagai sejauh mana sebuah produk dapat digunakan oleh pengguna tertentu untuk mencapai tujuan tertentu dengan efektivitas, efisiensi, dan kepuasan dalam konteks

penggunaan yang spesifik (International Organization for Standardization, 2018).

a. Efisiensi Waktu sebagai Indikator Utama

Efisiensi waktu (*time efficiency*) dipilih sebagai variabel terikat utama karena menjadi solusi langsung atas masalah verifikasi manual. Literatur menegaskan bahwa metode validasi manual saat ini "tidak mencukupi" (*insufficient*) untuk menghadapi volume dan kecepatan penyebaran informasi baru yang muncul setiap saat di media sosial (Guo, Schlichtkrull, & Vlachos, 2022).

b. Pengukuran Menggunakan *User Experience Questionnaire* (UEQ)

User Experience Questionnaire (UEQ) adalah metode pengukuran kuantitatif yang dirancang untuk mengevaluasi persepsi subjektif pengguna terhadap suatu perangkat lunak secara komprehensif, mencakup kualitas pragmatis (fungsionalitas) dan kualitas hedonis (aspek emosional) (Nurfaldini, Alfarizi, & Kuncoro, 2024). Evaluasi ini dilakukan melalui 26 butir pernyataan berskala diferensial semantik yang mengelompokkan pengalaman pengguna ke dalam enam dimensi utama, yaitu daya tarik (*attractiveness*), kejelasan (*perspicuity*), efisiensi

(*efficiency*), keandalan (*dependability*), stimulasi (*stimulation*), dan kebaruan (*novelty*) (Rivansyah, Arifianto, & Suharso, 2023). Pendekatan ini memungkinkan peneliti untuk tidak hanya mengukur seberapa mudah suatu aplikasi dioperasikan, tetapi juga seberapa besar kepuasan dan stimulasi psikologis yang dirasakan oleh pengguna saat berinteraksi dengan antarmuka sistem tersebut.

Penggunaan instrumen UEQ menjadi sangat esensial dalam pengujian aplikasi, mengingat target pengguna utamanya adalah Generasi Z. Karakteristik demografi ini tidak hanya menuntut fungsionalitas sistem yang bekerja dengan cepat dan tanpa hambatan, tetapi juga memiliki ekspektasi yang tinggi terhadap estetika visual dan kebaruan desain (Rivansyah et al., 2023). Dengan mengukur kualitas hedonis dan pragmatis secara bersamaan, pengujian UEQ memastikan bahwa fitur kecerdasan buatan di dalam aplikasi tidak hanya berhasil memverifikasi klaim disinformasi secara teknis, tetapi juga dibalut dalam antarmuka yang menarik, memotivasi, dan menstimulasi pengguna untuk terus memanfaatkan sistem tersebut secara konsisten sebagai alat bantu literasi digital sehari-hari (Nurfaldini et al., 2024).

B. Hasil Penelitian yang Relevan

Berikut adalah ringkasan penelitian terdahulu yang menjadi landasan teoritis dan empiris bagi penelitian ini, sebagaimana dirangkum dalam dokumen referensi :

No	Peneliti & Tahun	Judul Penelitian	Metode Penelitian	Hasil Penelitian	Perbedaan dengan Penelitian Ini
1	Effendi (2023)	<i>User Behaviour and Hoax Information on Social Media Case of Indonesia</i>	Deskriptif Kuantitatif (Kuesioner)	Sebanyak 71,4% responden pernah menyebarkan hoaks secara tidak sengaja dan mayoritas pengguna mempercayai informasi media sosial tanpa verifikasi.	Penelitian Effendi berfokus pada analisis perilaku, sementara penelitian ini mengembangkan solusi teknologi (aplikasi) untuk mengatasi perilaku tersebut.
2	Firdaus (2024)	Memahami Perilaku Informasi Gen-Z dan Strategi	<i>Literature Review</i>	Generasi Z memiliki kerentanan tinggi terhadap disinformasi karena pola konsumsi informasi yang cepat	Penelitian Firdaus memberikan landasan teoritis tentang subjek, sedangkan

		Melawan Disinformasi		dan kurangnya skeptisisme kritis.	penelitian ini melakukan pengujian praktis efisiensi aplikasi pada Gen Z.
3	Guo, Schlichtkrull, & Vlachos (2022)	<i>A Survey on Automated Fact-Checking</i>	Studi Literatur dan Komparasi Sistem	Verifikasi fakta manual tidak lagi mencukupi untuk menandingi volume informasi digital; diperlukan otomatisasi berbasis AI untuk skala besar.	Penelitian Guo dkk. membahas sistem secara umum, sementara penelitian ini mengimplementasikan spesifik Gemini AI dan Google CSE pada platform mobile.
4	Rivansyah, Arifianto, & Suharso (2023)	<i>Analisis User Interface dan User Experience pada SIA</i>	Kuantitatif (<i>User Experience Questionnaire</i>)	Pengukuran UEQ berhasil mengidentifikasi tingkat kepuasan hedonis dan pragmatis pengguna terhadap	Objek penelitian Rivansyah dkk. adalah aplikasi akademik, sedangkan penelitian ini menguji aplikasi

		Mobile dengan Metode UEQ		aplikasi mobile secara akurat.	cek fakta dengan metrik tambahan efisiensi waktu (<i>time on task</i>).
--	--	--------------------------	--	--------------------------------	---

Table 1. Hasil Penelitian yang Relevan

C. Kerangka Berpikir

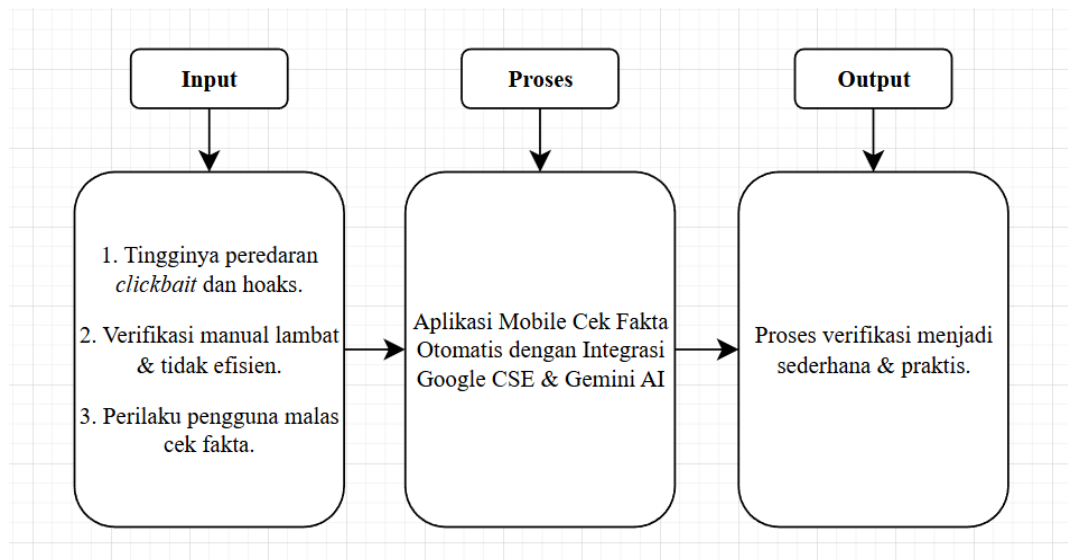
Kerangka berpikir dalam penelitian ini dibangun secara spesifik berdasarkan alur kerja arsitektur sistem aplikasi, yang dirancang sebagai intervensi teknologi terhadap tingginya paparan disinformasi di kalangan Generasi Z yang seringkali memberikan informasi secara instan tanpa melakukan verifikasi mandiri (Firdaus, 2024). Mengingat metode verifikasi manual konvensional terbukti lambat dan tidak lagi mencukupi untuk mengimbangi kecepatan penyebaran hoaks (Guo, Schlichtkrull, & Vlachos, 2022), alur kerja sistem ini diinisiasi secara praktis saat pengguna membagikan klaim teks dari sebuah postingan media sosial maupun halaman *web* langsung ke dalam aplikasi. Langkah awal ini bertindak sebagai pemicu (*trigger*) agar sistem dapat merespons kebutuhan pengguna dengan intervensi seminimal mungkin.

Setelah teks diterima oleh aplikasi, sistem akan mengeksekusi dua tahapan pemrosesan inti secara otomatis dan berurutan. Tahap pertama adalah pencarian bukti otomatis menggunakan *Google Custom Search Engine* (CSE) yang secara cepat mengindeks dan mengumpulkan referensi

data hanya dari sumber-sumber berita pilihan. Data mentah yang berhasil dihimpun oleh mesin pencari tersebut kemudian diteruskan ke tahap kedua, yaitu analisis faktualitas menggunakan *Gemini AI*. Pada tahap ini, model bahasa besar (*Large Language Model*) bekerja untuk membandingkan klaim awal pengguna dengan bukti-bukti dari CSE guna merumuskan tingkat dukungan suatu informasi secara *real-time*. Bersamaan dengan proses analitik tersebut, aplikasi juga melakukan penyimpanan riwayat pencarian melalui *database* lokal SQLite. Implementasi penyimpanan lokal ini berfungsi krusial untuk memastikan bahwa data aktivitas pengguna dapat diakses ulang dengan cepat tanpa membebani panggilan jaringan, sekaligus menjamin isolasi privasi data pengguna agar tidak tersimpan di server pihak ketiga (Karamoy & Wibowo, 2025).

Rangkaian pemrosesan komputasional tersebut pada akhirnya bermuara pada luaran (*output*) berupa hasil analisis faktualitas yang disajikan langsung ke layar perangkat pengguna. Melalui alur kerja yang terotomatisasi ini, proses pemilahan informasi yang semula membutuhkan upaya manual lintas *platform* dapat diselesaikan secara instan dalam satu aplikasi terpadu. Guna memastikan bahwa alur sistem ini benar-benar berjalan sesuai dengan tujuan penelitian, hasil akhir dari interaksi pengguna dengan aplikasi ini akan dievaluasi secara kuantitatif. Pengujian akan berfokus pada komparasi efisiensi waktu (*time on task*) serta evaluasi penerimaan sistem menggunakan *System Usability Scale* (SUS) dan *User*

Experience Questionnaire (UEQ), sehingga produk akhir tidak hanya valid secara fungsional, tetapi juga memberikan pengalaman penggunaan yang memuaskan dan menstimulasi literasi digital Generasi Z (Nurfaldini, Alfarizi, & Kuncoro, 2024).



Gambar 1. Kerangka Berpikir

BAB III

METODE PENELITIAN

A. Metode dan Desain Penelitian

1. Metode Penelitian

Metode penelitian yang digunakan dalam penelitian ini adalah metode *Research and Development* (R&D) atau Penelitian dan Pengembangan. Menurut (Sugiyono, 2019), metode penelitian dan pengembangan merupakan metode yang digunakan untuk menghasilkan produk tertentu dan menguji keefektifan produk tersebut. Berbeda dengan penelitian konvensional yang hanya bertujuan untuk menguji teori atau mendeskripsikan fenomena, metode R&D berorientasi pada hasil akhir berupa produk nyata yang dapat berfungsi sebagai solusi aplikatif atas suatu permasalahan yang ada di lapangan. (Saputri, Chairunisa, & Suryani, 2024; Sugiyono, 2019).

Dalam konteks penelitian ini, produk yang dikembangkan adalah sebuah aplikasi *mobile* Klarip yang dirancang khusus untuk mengotomatisasi proses verifikasi dan cek fakta atas klaim atau informasi yang beredar di media sosial. Pengembangan aplikasi ini mengintegrasikan teknologi *Artificial Intelligence* (AI) berbasis LLM (*Large Language Model*) menggunakan *Gemini API* dan

mesin pencari terfilter melalui *Google Custom Search Engine* (CSE).

Dalam ranah *Research and Development* (R&D), terdapat berbagai macam model prosedural yang diadopsi sesuai dengan karakteristik produk akhir yang ingin dicapai. Dan beberapa klasifikasi rumpun model pengembangan tersebut antara lain: ADDIE (Analysis, Design, Development, Implementation, Evaluation): Kerangka kerja yang berfokus pada pengembangan instruksional, kurikulum, dan media pembelajaran interaktif berbasis pedagogis (Saputri dkk., 2024). ,GDLC (Game Development Life Cycle) Metodologi yang dirancang khusus untuk rekayasa permainan (*game*) digital dengan menitikberatkan pada aspek estetika, aset visual, mekanik permainan (*gameplay*), dan cerita (Ramadan & Widyani, 2013). , Dan SDLC (*Software Development Life Cycle*) Metodologi rekayasa perangkat lunak (*software engineering*) yang digunakan untuk merancang, membangun, dan menguji sistem informasi atau aplikasi berbasis web, desktop, maupun *mobile* secara terstruktur (Pressman & Maxim, 2020).

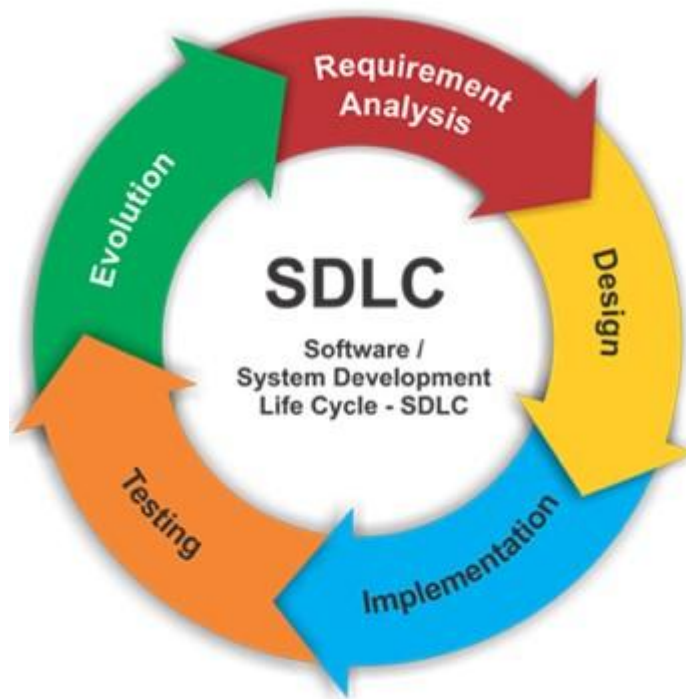
Berdasarkan klasifikasi di atas, penelitian ini secara mutlak memilih dan mengadopsi rumpun *Software Development Life Cycle* (SDLC) sebagai desain operasional pengembangan produk.

Adapun alasan mendasar pemilihan SDLC dalam pengembangan aplikasi Klarip adalah karena Karakteristik Produk Berupa Aplikasi Utilitas *Mobile* Aplikasi Klarip bukanlah sebuah media pembelajaran (seperti batasan model ADDIE) ataupun sebuah produk hiburan interaktif/game (seperti batasan model GDLC). Klarip merupakan sebuah perangkat lunak utilitas murni (*utility software*) berbasis *mobile*.

2. Desain Penelitian

Desain penelitian merupakan cetak biru (*blueprint*) atau rencana prosedur yang disusun secara sistematis untuk memandu jalannya pengembangan produk dalam penelitian ini. Sesuai dengan pembatasan rumpun pengembangan produk yang telah ditetapkan pada sub-bab sebelumnya, penelitian ini menggunakan *Software Development Life Cycle* (SDLC) sebagai kerangka kerja utamanya.

Penerapan tahapan SDLC dalam penelitian ini dijabarkan melalui bagan operasional berikut:



Gambar 1. Kerangka Berpikir

Dalam pelaksanaannya, proses pengembangan difokuskan pada empat tahapan utama yang disesuaikan dengan batasan masalah. Tahap pertama adalah analisis kebutuhan, yaitu proses mengidentifikasi spesifikasi fungsional sistem berdasarkan kerentanan literasi digital Generasi Z. Tahap kedua adalah perancangan sistem, yang mencakup perancangan antarmuka visual (*User Interface*) dan arsitektur basis data lokal menggunakan SQLite. Tahap ketiga adalah implementasi atau pengodean (*coding*) bahasa pemrograman Dart di dalam *framework* Flutter untuk membangun aplikasi secara utuh. Tahap

keempat adalah pengujian (*testing*). Pada fase penutup ini, aplikasi dievaluasi secara kuantitatif melalui pengukuran efisiensi waktu (*time on task*) serta penyebaran kuesioner *System Usability Scale* (SUS) dan *User Experience Questionnaire* (UEQ) untuk membuktikan kelayakan dan efektivitas produk.

B. Subjek Penelitian

1. Populasi

Populasi dalam penelitian ini adalah pengguna media sosial aktif di Indonesia yang termasuk ke dalam kelompok demografi Generasi Z. Pemilihan rentang usia ini didasarkan pada karakteristik mereka sebagai generasi digital native yang memiliki intensitas tertinggi dalam mengakses dan mengonsumsi informasi melalui platform digital (Hanifatunnisa dkk., 2025). Meskipun Generasi Z memiliki tingkat kecakapan teknologi operasional yang sangat mumpuni, pola konsumsi informasi mereka cenderung serba cepat dan instan. Kondisi ini menciptakan sebuah masalah di mana mereka menjadi kelompok yang sangat rentan terhadap paparan disinformasi dan berita palsu, mengingat ritme konsumsi yang cepat, sering kali mengabaikan proses verifikasi mendalam (Firdaus, 2024). Dengan tingginya tingkat kerentanan tersebut, karakteristik populasi Generasi Z dinilai sangat representatif dan merupakan subjek yang paling relevan serta membutuhkan

intervensi teknologi berupa alat bantu verifikasi otomatis untuk memperkuat literasi digital mereka.

2. Sampel

Penentuan jumlah sampel dalam pengujian fungsionalitas aplikasi (Klarip) ini ditetapkan secara spesifik sebanyak 20 orang responden dari kelompok Generasi Z yang aktif menggunakan media sosial di Indonesia. Populasi target dalam penelitian ini pada dasarnya sangat masif dan tidak terbatas (*infinite population*). Namun, penting untuk ditegaskan sejak awal bahwa pengumpulan data dalam penelitian *Research and Development* (R&D) ini bukanlah sebuah survei opini publik, jajak pendapat, ataupun sensus penduduk berskala makro yang bertujuan untuk menggeneralisasi karakteristik jutaan pengguna media sosial di Indonesia. Fokus utama dari pengujian ini adalah evaluasi fungsionalitas sistem (*system functionality evaluation*) dan deteksi hambatan interaksi antarmuka (*user experience testing*) saat aplikasi dioperasikan secara langsung.

Oleh karena itu, penetapan kuota 20 responden ini didasarkan pada landasan metodologi pengujian rekayasa perangkat lunak yang telah divalidasi dan diterapkan secara luas pada literatur akademik nasional. Sebagai landasan pertama terkait evaluasi pengalaman pengguna (UX), ukuran sampel tersebut

dinilai telah sepenuhnya memenuhi standar kelayakan instrumen *User Experience Questionnaire* (UEQ). Merujuk pada riset pengujian aplikasi *mobile* terapan yang dipublikasikan dalam Jurnal Sistem Informasi TEKNOFILE, pengumpulan data dari 20 responden sudah sangat mencukupi untuk menghasilkan potret data metrik evaluasi UX yang stabil, andal, dan representatif tanpa memerlukan jumlah partisipan berskala masif (Nawansa & Pratama, 2024).

Selain itu, dari perspektif akurasi pengukuran kinerja teknis, implementasi kuota sampel ini sejalan dengan penelitian yang diterbitkan pada Jurnal Pengembangan Teknologi Informasi dan Ilmu Komputer Universitas Brawijaya (J-PTIIK). Studi tersebut menegaskan bahwa pengujian fungsionalitas sistem (*usability*) yang melibatkan 20 responden sudah memberikan interval kepercayaan (*confidence interval*) yang sangat baik untuk mendeteksi anomali atau kesalahan fungsi pada antarmuka perangkat lunak (Wijaya et al., 2024). Melalui pendekatan yang terukur dan bersandar pada keilmuan tersebut, evaluasi kelayakan teknis serta kualitas pengalaman pengguna pada aplikasi Klarifikasi.id dengan 20 responden dapat dipertanggungjawabkan objektivitasnya secara ilmiah dan efisien.

3. Waktu Penelitian

Kegiatan	Sept	Okt	Nov	Des	Jan	Feb	Mar	Apr	Mei	Jun
Penyusunan bab 1										
Penyusunan bab 2										
Penyusunan bab 3										
<i>Planning</i>										
Pengecekan proposal										
Praproduksi										
Produksi										
Uji tes <i>Aplikasi</i>										
Uji kelayakan										
Uji- <i>t</i>										
Penyusunan Laporan										

Table 2. Jadwal Penelitian

C. Teknik dan Instrumen Pengumpulan Data

Proses pengumpulan data dalam penelitian pengembangan aplikasi (Klarip) ini dibagi menjadi tiga tahapan utama yang mencakup pengujian objektif oleh peneliti dan evaluasi subjektif oleh responden.

Tahap pertama difokuskan pada pengujian fungsionalitas aplikasi menggunakan pendekatan *Black Box Testing*. Pada tahap ini, peneliti

bertindak sebagai evaluator independen untuk memastikan bahwa seluruh fitur inti perangkat lunak—seperti modul *input* teks, transmisi data menuju antarmuka pemrograman aplikasi (API) Google Custom Search Engine dan Gemini AI, hingga fungsi operasi basis data lokal (SQLite)—dapat berjalan sesuai dengan rancangan tanpa adanya kegagalan sistem (*bug*). Pengujian fungsionalitas ini bertujuan untuk mengevaluasi efektivitas teknis perangkat lunak, guna memastikan bahwa setiap perintah (*input*) dari pengguna selalu menghasilkan keluaran (*output*) yang diinginkan dan tereksekusi dengan sempurna (Pressman & Maxim, 2020).

Untuk tahap evaluasi fungsionalitas aplikasi, instrumen yang digunakan oleh peneliti adalah lembar panduan skenario pengujian *Black Box* (*Black Box Testing Checklist*). Lembar instrumen ini memuat daftar fungsi utama perangkat lunak yang harus dieksekusi secara berurutan, mulai dari fungsi masuk dan buat akun, fungsi klarifikasi klaim di media sosial, hingga keberhasilan penyimpanan data lokal SQLite. Instrumen lembar kerja ini menggunakan format matriks penilaian biner, di mana peneliti akan memberikan indikator "Berhasil" jika fitur berjalan sesuai dengan spesifikasi arsitektur yang dirancang, atau "Gagal" jika sistem menghasilkan galat (*bug*) selama proses komputasi berlangsung (Pressman & Maxim, 2020).

Tahap kedua dari teknik pengumpulan data ini adalah pengukuran metrik kinerja aplikasi, secara spesifik pada efisiensi waktu pemrosesan (*response time*). Sama halnya dengan pengujian fungsionalitas, pengambilan data ini dilakukan langsung oleh peneliti menggunakan instrumen pengukur waktu ukur presisi (*stopwatch*) atau pencatatan log sistem. Fokus dari instrumen ini adalah untuk menghitung durasi penyelesaian tugas (*Time on Task*) dalam satuan detik, yang direkam secara ketat mulai dari saat tombol verifikasi ditekan hingga hasil analisis dari kecerdasan buatan ditampilkan secara utuh di layar perangkat (Mufarroha, 2023). Pengumpulan data efisiensi waktu ini menjadi sangat penting, yaitu untuk mengonfirmasi bahwa otomatisasi sistem verifikasi mampu memangkas durasi penelusuran klaim hoaks secara signifikan jika dibandingkan dengan metode pencarian manual konvensional.

Instrumen yang dimanfaatkan untuk mengukur tingkat efisiensi waktu (*response time*) terdiri atas alat pencatat waktu (*stopwatch*) dan lembar observasi *Time on Task*. Instrumen lembar observasi ini dirancang khusus dengan dua kolom pencatatan durasi waktu dalam satuan detik guna memfasilitasi studi komparasi. Peneliti menggunakan instrumen ini untuk mencatat durasi waktu yang dihabiskan dalam melakukan verifikasi klaim di media sosial secara manual melalui peramban web dengan perhitungan instrumen waktu secara ketat dimulai sejak peneliti menekan tombol pencarian mencari verifikasi dari suatu klaim di media sosial hingga mendapatkan kesimpulan nya di web, yang kemudian disandingkan

dengan durasi waktu penyelesaian tugas oleh aplikasi (Klarip), dengan perhitungan instrumen waktu secara ketat dimulai sejak peneliti menekan tombol pencarian hingga hasil kesimpulan akhir dari kecerdasan buatan selesai diberikan pada antarmuka aplikasi (Mufarroha, 2023). Alat ukur ini memastikan objektivitas dan keakuratan matematis dalam membuktikan efektivitas pemangkasan waktu verifikasi.

Setelah aplikasi dipastikan lolos uji fungsionalitas dan terbukti efisien secara teknis oleh peneliti, tahap ketiga pengumpulan data dilanjutkan dengan melibatkan partisipasi langsung dari 20 responden Generasi Z di Indonesia yang telah ditetapkan sebelumnya. Teknik pengumpulan data pada tahap ini menggunakan instrumen kuesioner *System Usability Scale* (SUS) yang didistribusikan kepada para responden usai mereka melakukan uji coba interaksi dengan aplikasi. Kuesioner SUS yang memuat 10 butir pernyataan berskala *Likert* tersebut difungsikan secara khusus untuk menangkap persepsi langsung dari pengguna akhir guna mengukur tingkat kebergunaan, kenyamanan antarmuka (*user interface*), serta tingkat kemudahan aplikasi untuk dipelajari oleh masyarakat awam (*learnability*) dalam penggunaan pertama kali (Setiawan & Lazuardi, 2023).

Untuk tahap evaluasi dari sisi pengguna akhir, instrumen pengumpulan data yang digunakan adalah dokumen kuesioner *System Usability Scale* (SUS) yang didistribusikan kepada 20 responden

kelompok Generasi Z di Indonesia. Instrumen kuesioner ini mengadopsi kerangka kerja baku yang memuat 10 butir pernyataan evaluatif, yang terbagi secara seimbang menjadi lima pernyataan bernada positif dan lima pernyataan bernada negatif guna mencegah terjadinya bias persetujuan (*acquiescence bias*) saat pengisian. Setiap butir pernyataan di dalam instrumen ini dinilai menggunakan skala Likert lima poin, yang membentang dari bobot nilai 1 untuk respons "Sangat Tidak Setuju" hingga bobot nilai 5 untuk respons "Sangat Setuju". Instrumen evaluasi persepsi ini disajikan secara digital dan diisi sesaat setelah responden menggunakan aplikasi, sehingga alat ukur ini secara valid mampu menangkap impresi psikologis langsung terkait tingkat kemudahan operasional dan kenyamanan navigasi perangkat lunak (Brooke, 1996; Setiawan & Lazuardi, 2023).

No	Pernyataan	Sifat Pernyataan	Aspek Evaluasi
1	Saya merasa fitur pengecekan fakta berbasis AI di aplikasi Klarip ini mungkin sangat berguna untuk aktivitas saya sehari-hari.	Positif	Kebergunaan
2	Saya merasa alur penggunaan aplikasi ini terlalu rumit dan berbelit-belit untuk orang awam.	Negatif	<i>Learnability</i>
3	Tampilan antarmuka aplikasi ini terlihat rapi, modern, dan membuat saya merasa nyaman saat menggunakannya.	Positif	Kenyamanan UI

No	Pernyataan	Sifat Pernyataan	Aspek Evaluasi
4	Saya merasa butuh panduan khusus atau bantuan dari orang yang lebih paham teknologi untuk bisa menggunakan aplikasi ini.	Negatif	<i>Learnability</i>
5	Fitur-fitur yang ada di dalam aplikasi ini (seperti pencarian berita dan riwayat analisis) berfungsi dengan sangat baik.	Positif	Kebergunaan
6	Tata letak tombol, teks, dan menu di dalam aplikasi ini terasa membingungkan serta tidak konsisten.	Negatif	Kenyamanan UI
7	Saya yakin sebagian besar masyarakat awam dapat memahami dan mempelajari cara menggunakan aplikasi ini dengan sangat cepat.	Positif	<i>Learnability</i>
8	Saya merasa aplikasi ini sangat tidak praktis dan merepotkan saat digunakan untuk memverifikasi sebuah informasi.	Negatif	Kebergunaan
9	Saya merasa percaya diri dan tidak kebingungan saat pertama kali mencoba menganalisis klaim berita menggunakan aplikasi ini.	Positif	<i>Learnability & UI</i>
10	Saya merasa harus banyak belajar dan mencoba-coba terlebih dahulu sebelum bisa menggunakan aplikasi ini dengan lancar.	Negatif	<i>Learnability</i>

Table 3. Skenario Pengujian Kinerja (Task Scenario)

Penggabungan dari ketiga tahapan instrumen ini dirancang untuk menghasilkan set data primer yang komprehensif, mencakup aspek ketahanan kode (fungsional), kecepatan algoritma (waktu), dan penerimaan psikologis pengguna (kuesioner).

D. Teknik Analisis Data

Setelah seluruh rangkaian data dari ketiga tahapan pengujian berhasil dihimpun, proses pengolahan data dilakukan menggunakan pendekatan statistik deskriptif tanpa melalui uji hipotesis inferensial ataupun generalisasi populasi makro. Teknik analisis data pertama diterapkan pada hasil pengujian fungsionalitas aplikasi melalui lembar ceklis *Black Box Testing* yang diuji secara mandiri oleh peneliti. Data status kelayakan biner ("Berhasil" atau "Gagal") dihitung akumulasinya menggunakan rumus persentase tingkat keberhasilan tugas (*Task Success Rate*). Nilai ini diperoleh dengan membagi jumlah fitur yang tereksekusi dengan sempurna dengan total keseluruhan skenario fungsionalitas yang diuji, lalu dikalikan seratus persen. Merujuk pada metodologi pengujian kapabilitas sistem pada Jurnal Teknoinfo (Arifin, Z., & Susanto, A., 2023), batas nilai minimal untuk menyatakan bahwa aplikasi telah memenuhi standar kelayakan fungsional dasar adalah sebesar seratus persen, yang berarti seluruh komponen arsitektur utama mulai dari penangkapan teks input, pengiriman parameter API, hingga penyimpanan SQLite tidak boleh mengalami galat (*bug*) sama sekali.

pengukuran aspek Tingkat Keberhasilan ini dihitung menggunakan metrik *Task Success Rate* (TSR) (Pradnyana & Rusjayanthi, 2023). :

$$TSR = \frac{\sum F_B}{\sum F_T} \times 100\%$$

Keterangan Variable :

TSR: Task Success Rate (%)

$\sum F_B$: Jumlah fitur berstatus

$\sum F_T$: Total keseluruhan fitur yang diuji

Teknik analisis data kedua difokuskan pada pengukuran metrik efisiensi waktu pemrosesan (*response time*) penelusuran klaim hoaks yang durasinya direkam langsung oleh peneliti dalam satuan detik menggunakan instrumen *stopwatch*. Data catatan durasi waktu penyelesaian tugas (*Time on Task*) dikelompokkan menjadi dua bagian, yaitu durasi waktu penelusuran manual oleh peneliti via peramban web konvensional dan durasi waktu otomatisasi oleh sistem aplikasi (klarip). Kedua kelompok data tersebut dihitung nilai rata-ratanya (*mean score*), untuk kemudian dicari persentase selisih efisiensi kecepatan yang dihasilkan menggunakan rumus perbandingan efisiensi waktu interaktif. Sebagaimana yang diimplementasikan dalam riset performa sistem pada Jurnal Media Informatika Budidarma (Saputra & Hidayat, 2024)., target

nilai keberhasilan untuk metrik ini dinilai sukses memberikan efisiensi tinggi apabila otomatisasi mampu memangkas waktu penelusuran manual secara signifikan hingga di atas lima puluh persen, atau menghasilkan waktu pemrosesan total di bawah sepuluh detik per satu kali eksekusi klaim berita.

pengukuran aspek Tingkat Efisiensi Waktu (*Time-Based Efficiency*) menggunakan metrik *Efisiensi* (Pradnyana & Rusjyanthi, 2023). :

Rumus Efisiensi :

$$Efisiensi = \left(\frac{T_{Manual} - X_{Aplikasi}}{\bar{T}_{Manual}} \right) \times 100\%$$

Keterangan Variable :

$\bar{T}_{Manual}$: Rata-rata waktu verifikasi manual

$\bar{T}_{Aplikasi}$: Rata-rata waktu verifikasi aplikasi

Terakhir, teknik analisis data ketiga digunakan untuk mengolah data persepsi kebergunaan sistem berdasarkan kuesioner *System Usability Scale* (SUS) yang diisi oleh 20 responden Generasi Z. Pengolahan data dilakukan dengan mengkonversi nilai pilihan jawaban skala Likert (1 hingga 5) menggunakan aturan baku pembobotan SUS berselang-seling.

Untuk butir pernyataan bernada positif yang terletak pada nomor ganjil, skor konversi didapatkan dari nilai pilihan jawaban responden yang dikurangi dengan angka satu, sedangkan untuk butir pernyataan bernada negatif pada nomor genap, skor konversi diperoleh dari angka lima yang dikurangi dengan nilai pilihan jawaban responden. Langkah berikutnya adalah menjumlahkan seluruh skor hasil konversi dari sepuluh butir pernyataan tersebut, kemudian mengalikan hasil penjumlahan dengan konstanta sebesar 2,5 untuk menghasilkan skor akhir SUS individu dalam rentang skala 0 hingga 100. Nilai akhir dari akumulasi skor konversi seluruh responden dihitung nilai rata-ratanya (*mean score*) untuk dipetakan ke dalam kategori *Acceptability Ranges*, *Grade Scale*, dan *Adjective Rating*. Berdasarkan standarisasi pengujian kegunaan lokal yang divalidasi oleh Jurnal RESTI (Pratama & Nugraha, 2022)., nilai rata-rata skor SUS aplikasi harus mencapai target skor minimal sebesar 68 untuk dapat dinyatakan masuk ke dalam kategori layak pakai, diterima oleh pengguna (*Acceptable*), dan bernilai baik (*Good*).

Analisis tingkat kepuasan dan kelayakan subjektif pengguna terhadap antarmuka aplikasi dihitung menggunakan standarisasi kuesioner *System Usability Scale* (SUS). Tahapan pengolahan data instrumen SUS diadopsi dari metodologi pengujian *usability* yang diterapkan oleh (Hafizh & Rahmawati, 2023) , di mana proses perhitungan dipisahkan secara terstruktur ke dalam tiga langkah utama sebagai berikut:

Langkah 1 (Konversi Skor):

Setiap jawaban responden (X) yang bernilai 1 sampai 5 akan diubah nilainya berdasarkan nomor pertanyaannya: Untuk Pernyataan Ganjil (Nomor 1, 3, 5, 7, 9 - Bersifat Positif):

$$\text{Pernyataan Ganjil (X): } S_G = X - 1$$

Contoh: Jika responden menjawab 5 (Sangat Setuju), maka skor konversinya: $5 - 1 = 4$

Jika responden menjawab 1 (Sangat Tidak Setuju), maka skor konversinya: $1 - 1 = 0$

Untuk Pernyataan Genap (Nomor 2, 4, 6, 8, 10 - Bersifat Negatif):

$$\text{Pernyataan Genap (X): } S_P = 5 - X$$

Contoh: Jika responden menjawab 1 (Sangat Tidak Setuju) pada hal negatif, artinya itu hal bagus. Maka skor konversinya: $5 - 1 = 4$

Jika responden menjawab 5 (Sangat Setuju) pada hal negatif, artinya itu buruk. Maka skor konversinya: $5 - 5 = 0$

Langkah 2 (Skor Individu):

Setelah semua 10 pernyataan milik satu orang responden sudah dikonversi (semua nilainya sekarang berada di rentang 0-4), langkah selanjutnya adalah menjumlahkan semuanya lalu dikalikan dengan konstanta 2,5.

$$Skor_{ind} = (\sum S_G + \sum S_P) \times 2,5$$

Keterangan Variable :

X : Nilai mentah jawaban responden (1- 5)

S_G : Hasil skor konversi butir ganjil

S_P : Hasil skor konversi butir genap

Kenapa dikali 2,5? Karena jumlah skor konversi maksimal dari 10 pertanyaan adalah $(10 \times 4 = 40)$ Agar skalanya menjadi standar universal bertaraf 0–100, maka nilai total tersebut harus dikalikan dengan 2,5 $(40 \times 2,5 = 100)$

Langkah 3 (Rata-rata Total):

Setelah mendapatkan nilai total $Skor_{ind}$ dari masing-masing 20 responden, lalu dijumlahkan seluruh skor individu tersebut kemudian membaginya dengan angka 20 (jumlah total responden).

$$\bar{X}_{SUS} = \frac{\sum Skor_{ind}}{20}$$

Keterangan Variable :

$Skor_{ind}$: Skor total satu responden skala (0-100)

$\bar{X}_{SUS}$: Nilai rata-rata akhir skor SUS aplikasi

BAB IV

HASIL KAJIAN DAN PEMBAHASAN

A. Hasil Kajian

Pengembangan aplikasi Klarip dilakukan dengan mengikuti tahapan sistematis dalam model *System Development Life Cycle* (SDLC), Proses ini dimulai dari pemetaan kebutuhan hingga terciptanya produk fungsional yang siap diuji kepada responden.

1. Tahap Analisis Kebutuhan (*Requirement Analysis*)

Tahap awal dalam siklus pengembangan aplikasi Klarip berfokus pada identifikasi masalah disinformasi dan penentuan spesifikasi kebutuhan fungsional yang wajib hadir sebagai solusi teknologi bagi Generasi Z. Berdasarkan analisis terhadap literatur cek fakta terkini, dirumuskan empat pilar kebutuhan utama sistem:

Pertama, sistem menggunakan pendekatan klasifikasi keluaran Tiga Verdict Penilaian, yaitu "Data Didukung", "Tidak Didukung", dan "Perlu Verifikasi Lebih Lanjut". Penggunaan tiga label ini merujuk pada standar emas (*gold standard*) global seperti dataset FEVER dan AVeriTeC, guna memberikan penalaran yang lebih akurat dan komprehensif dibandingkan pendekatan benar-salah konvensional (Thorne et al., 2018; Schlichtkrull et al., 2023).

Kedua, untuk mengatasi volume informasi yang masif, aplikasi mengadopsi metode *state-of-the-art* melalui Pencarian dan Analisis AI. Arsitektur ini mereplikasi alur kerja *Automated Fact-Checking* (AFC) modern dengan menghubungkan pencarian *real-time* via *Google Custom Search Engine* (CSE) dan pemrosesan *Large Language Models* (LLM) melalui Gemini AI. Implementasi prinsip *Retrieval-Augmented Generation* (RAG) menjadi kebutuhan krusial untuk menekan risiko halusinasi AI dengan cara memberikan artikel berita aktual sebagai landasan konteks sebelum model memberikan kesimpulan analisis (Rahman et al., 2025).

Ketiga, ketersediaan fitur Simpan Histori berbasis SQLite dirancang untuk memberikan nilai tambah psikologis berupa *cognitive offloading*. Fitur ini berfungsi sebagai memori eksternal bagi pengguna agar terhindar dari beban kognitif berlebih dalam mengingat misinformasi yang pernah ditemui. Secara teknis, penyimpanan lokal ini juga menjadi strategi untuk mengidentifikasi hoaks daur ulang (*recycled hoaxes*) secara cepat tanpa harus melakukan pemanggilan API eksternal berulang kali (Risko & Gilbert, 2016; Pérez-Curiel dkk., 2021).

Keempat, aplikasi memprioritaskan keamanan melalui prinsip Privacy-First & Local-by-Default. Mengingat riwayat

pencarian pengguna seringkali mencerminkan pandangan sensitif, pendekatan *Privacy by Design* diterapkan melalui arsitektur sisi klien secara utuh (*fully client-side*). Dengan melokalkan seluruh data autentikasi dan riwayat verifikasi ke dalam SQLite di perangkat pengguna, sistem secara inheren menerapkan prinsip minimalisasi data dan mengeliminasi risiko kebocoran data yang umum terjadi pada peladen awan terpusat (Cavoukian, 2009; Shi et al., 2016).

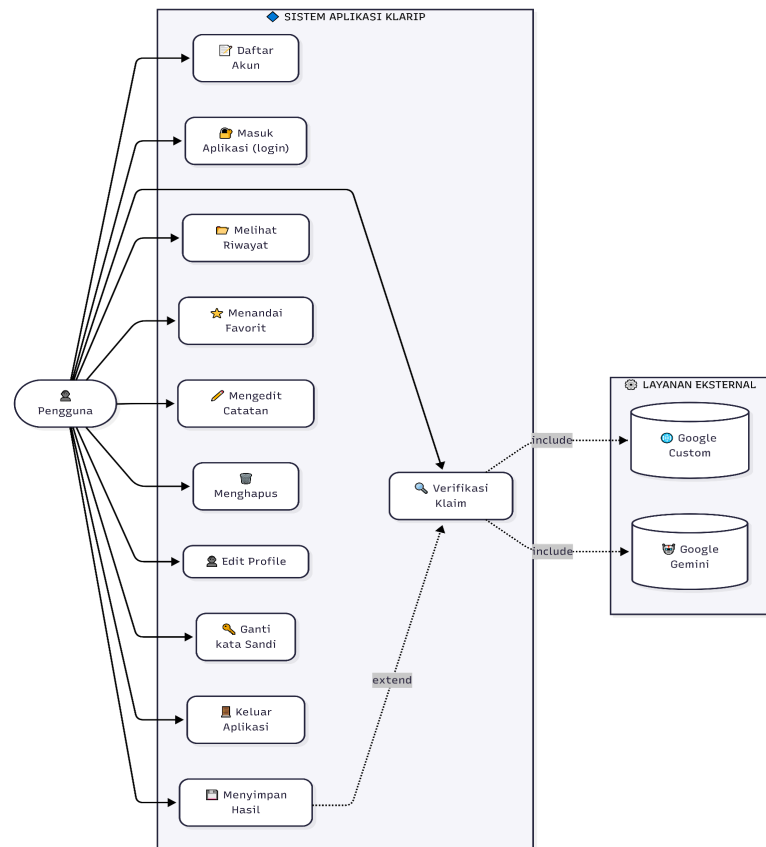
2. Tahap Perancangan Sistem

Fase perancangan dalam model pengembangan ini bertujuan untuk mendeskripsikan cetak biru (*blueprint*) dan spesifikasi detail dari perangkat lunak Klarip sebelum memasuki tahap penulisan kode. Perancangan ini mencakup empat aspek krusial, yaitu *Use Case*, *Flowchart Sistem*, Alur Verifikasi Klaim, Operasi Data Pengguna, dan Operasi Data Riwayat.

a. Use Case Diagram Aplikasi Klarip

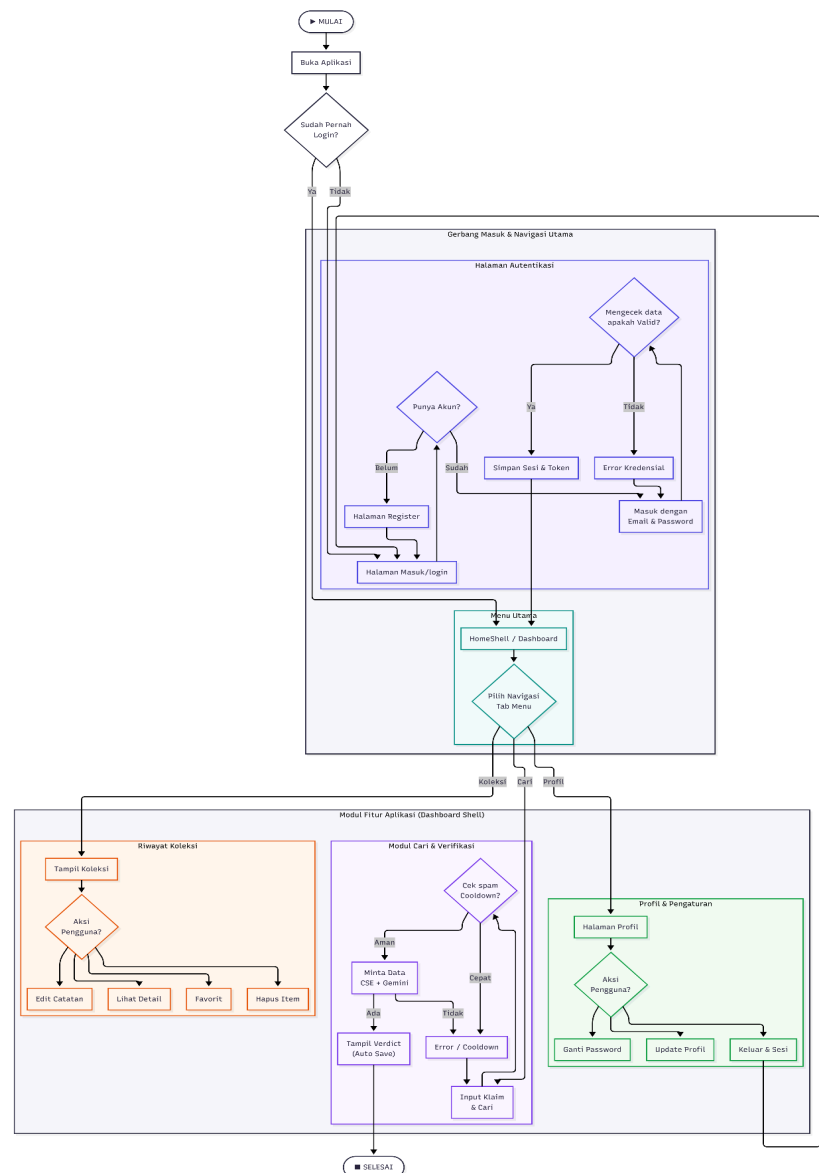
Use Case Diagram di bawah mendeskripsikan interaksi antara aktor (Pengguna) dengan fungsionalitas sistem pada aplikasi Klarip. Pengguna memiliki akses ke sebelas fitur operasional utama, yang meliputi pendaftaran akun, login, pengelolaan profil, penggantian kata sandi,

hingga manajemen riwayat koleksi (seperti melihat, menandai favorit, mengedit catatan, dan menghapus riwayat).



Gambar 1. Use Case Diagram Aplikasi Klarip

b. Flowchart Sistem (Alur Keseluruhan Aplikasi)



Gambar 1. Flowchart Sistem Aplikasi Klarip

Diagram *flowchart* ini merepresentasikan alur logika menyeluruh dari aplikasi Klarip secara terstruktur. Proses operasional diawali ketika aplikasi dibuka, di mana sistem secara otomatis menjalankan pengecekan status masuk (*login*) pengguna. Apabila pengguna terdeteksi

belum memiliki sesi aktif, sistem akan mengarahkan alur menuju gerbang masuk dan navigasi utama, tepatnya pada halaman autentikasi. Pada tahap ini, aplikasi memverifikasi kepemilikan akun. pengguna baru akan diarahkan ke halaman pendaftaran (*register*), sedangkan pengguna lama dapat langsung masuk dengan memasukkan *email* dan kata sandi. Selanjutnya, sistem memvalidasi kredensial tersebut untuk memunculkan pesan galat (*error*) jika data tidak sesuai, atau menyimpan sesi dan token jika valid guna meneruskan alur ke menu utama. Sebaliknya, apabila pengecekan awal mendeteksi keberadaan sesi aktif, alur akan memintas (*bypass*) proses autentikasi dan langsung melompat ke menu utama (*dashboard shell*).

Pada titik navigasi menu utama, sistem mendistribusikan alur ke dalam tiga modul fungsional utama yang berada di dalam lingkup antarmuka *dashboard shell*. Modul pertama adalah menu cari dan verifikasi, yang dieksekusi mulai dari masukan teks klaim oleh pengguna. Sistem kemudian mengevaluasi blok putusan untuk memeriksa batas waktu (*rate limit cooldown*) guna mencegah aktivitas permintaan yang berlebihan (*spamming*). Apabila eksekusi terdeteksi tidak aman, aplikasi menolak permintaan dengan memberikan

peringatan jeda waktu dan meminta pengulangan masukan. Namun, jika jeda waktu dinyatakan aman, sistem akan menjalankan proses komputasi utama melalui pengiriman *HTTP request* ke *Application Programming Interface* (API) Google CSE dan Gemini AI. Hasil dari komputasi tersebut kemudian dikembalikan ke antarmuka dalam wujud penyajian putusan (*verdict*) tiga tingkat yang mengklasifikasikan status kebenaran klaim.

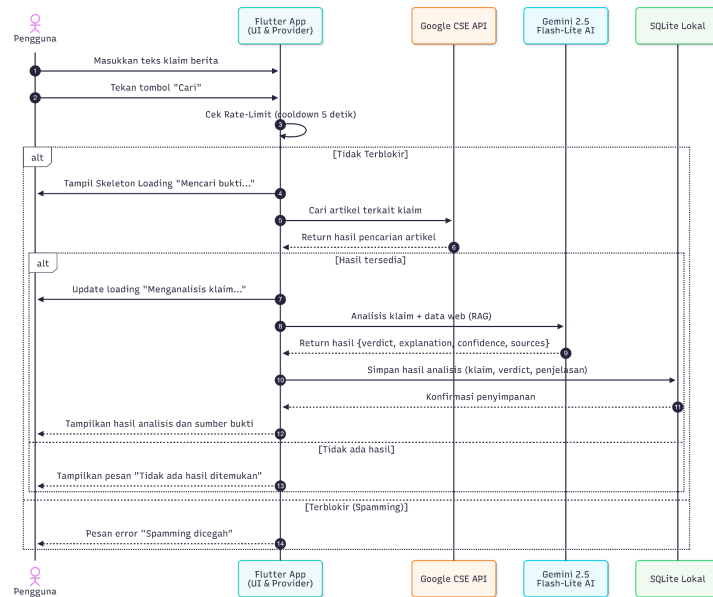
Modul fungsional kedua adalah halaman koleksi yang bertugas merender atau menyajikan data riwayat analisis kepada pengguna. Pada antarmuka ini, sistem secara aktif memantau deteksi aksi pengguna untuk merespons berbagai interaksi operasional yang dilakukan. Ketika interaksi terdeteksi, alur program akan diarahkan menuju salah satu dari empat fungsi pengelolaan basis data yang tersedia. Fungsi operasional tersebut mencakup mengubah catatan khusus, peninjauan detail analisis, penghapusan data riwayat, serta pengubahan status favorit pada informasi yang tersimpan.

Modul fungsional terakhir adalah profil dan pengaturan yang menyediakan kendali atas identitas serta keamanan akun. Pengguna yang mengakses halaman profil

dihadapkan pada tiga pilihan aksi utama untuk mengelola pengaturan mereka secara mandiri. Pilihan aksi tersebut memungkinkan dilakukannya pembaruan data diri, modifikasi lapisan keamanan melalui penggantian kata sandi, serta pemutusan akses melalui opsi keluar sesi (*logout*). Pemilihan opsi keluar tersebut akan secara otomatis mendepak pengguna kembali ke blok halaman autentikasi.

c. Diagram sekuens Operasi Verifikasi Klaim

Diagram sekuens (*sequence diagram*) verifikasi klaim menitikberatkan pada detail kronologis pertukaran pesan komputasi yang beroperasi di belakang layar (*behind-the-scenes*) saat aplikasi dihadapkan pada masukan klaim dari pengguna.



Gambar 1. Flowchart Sistem Aplikasi Klarip

Proses operasional ini dimulai oleh tindakan pengguna yang menekan tombol pencarian pada antarmuka aplikasi. Merespons tindakan tersebut, perangkat lunak yang dibangun melalui lapisan *provider* akan langsung melakukan validasi batas waktu permintaan (*rate-limit cooldown*). Langkah pengamanan awal ini diimplementasikan secara sistematis untuk mencegah penumpukan aktivitas pengiriman pesan yang berlebihan (*spamming*).

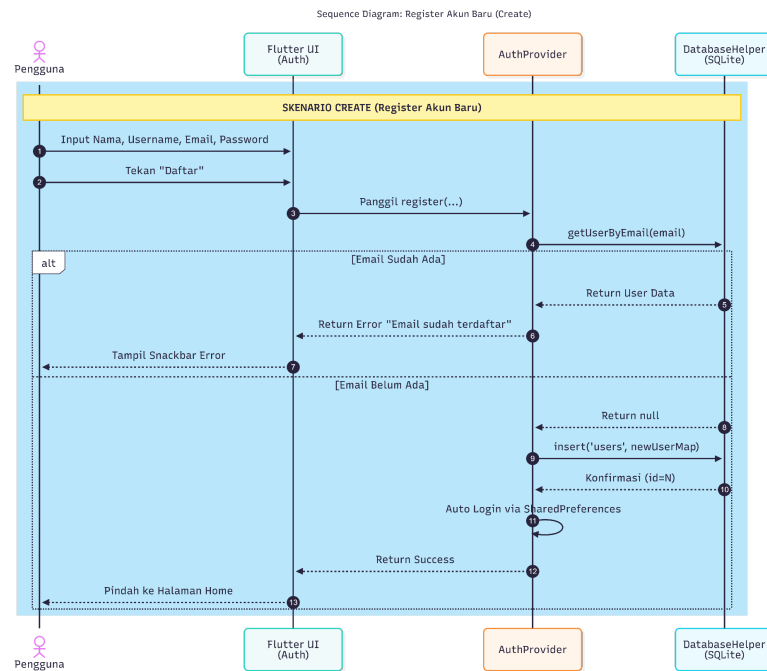
Apabila permintaan berada pada rentang waktu yang direstui, sistem akan merender *skeleton loading* guna memberikan umpan balik (*feedback*) visual kepada pengguna. Fase ini kemudian dilanjutkan dengan

pelemparan protokol *HTTP GET* menuju layanan Google CSE untuk mencari referensi yang relevan. Saat metadata perolehan bukti yang mencakup judul, tautan, dan cuplikan teks (*snippet*) berhasil ditarik, sistem secara otomatis beralih mengirimkan protokol *HTTP POST* terstruktur menuju model Gemini AI. Pengiriman data ini memanfaatkan teknik *Retrieval-Augmented Generation* (RAG), yakni sebuah metode yang menyertakan bukti penelusuran web ke dalam perintah agar model bahasa komputasi tidak mengalami halusinasi informasi.

Sebagai produk akhir dari rangkaian komputasi tersebut, Gemini AI akan mengembalikan objek keluaran berformat *JSON* yang merepresentasikan status putusan tiga strata (*3-tier verdict*). Hasil putusan analitik ini kemudian langsung dipersistensikan ke dalam memori lokal aplikasi melalui eksekusi *query INSERT* pada basis data SQLite. Setelah proses penyimpanan data lokal berhasil dikonfirmasi, informasi tersebut baru ditransformasikan secara utuh ke dalam representasi visual antarmuka pengguna. Alur pemrosesan yang asinkron ini memastikan bahwa pengguna mendapatkan hasil verifikasi fakta yang akurat dengan tetap mengutamakan keandalan penyimpanan data riwayat.

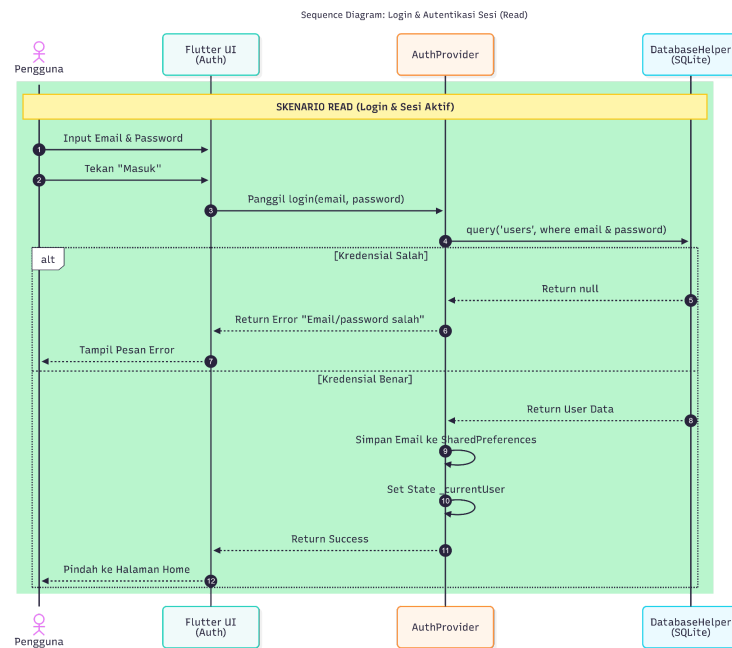
d. Diagram sekuens Operasi Login dan Register (Data Pengguna)

Diagram sekuens (*sequence diagram*) ini merepresentasikan mekanisme autentikasi luring yang berjalan secara independen tanpa ketergantungan pada layanan peladen (*backend*) terpusat. Keamanan kredensial pengguna sepenuhnya divalidasi dan dikelola di dalam tabel *users* pada basis data SQLite lokal. Pendekatan ini memastikan bahwa seluruh proses verifikasi identitas dapat dieksekusi secara cepat dan aman langsung di dalam perangkat keras pengguna.



Gambar 1. Skenario REGISTER (Create Akun)

Pada skenario pembuatan akun (*create* atau *register*), sistem menerapkan mekanisme pencegahan duplikasi data secara ketat. Sebelum memasukkan data (*insert*) identitas baru, komponen pengontrol (*AuthProvider*) diwajibkan untuk melakukan uji validasi redundansi menggunakan kueri pemilahan khusus. Langkah ini bertujuan untuk memastikan bahwa alamat email yang didaftarkan belum pernah diregistrasi sebelumnya, sehingga apabila data dinyatakan valid, identitas pengguna akan dicatat dan sesi navigasi langsung dibuatkan secara otomatis.

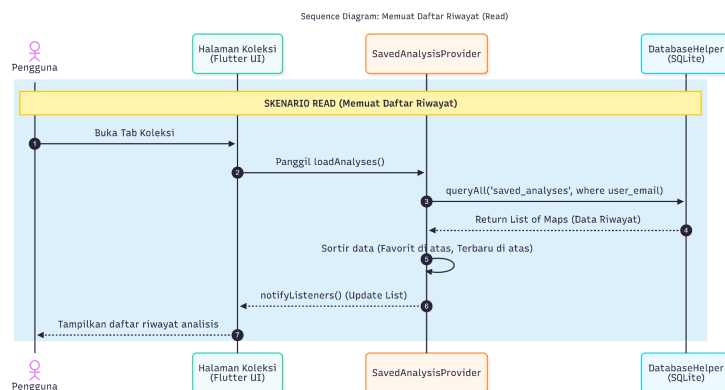


Gambar 1. Skenario LOGIN (Read Session)

Skenario pembacaan data (*read*) diimplementasikan pada proses masuk (*login*) melalui pencocokan dua pilar parameter utama, yakni alamat email dan kata sandi. Keberhasilan dalam tahap autentikasi ini akan memicu aplikasi untuk merantai sesi tersebut ke dalam penyimpanan bawaan perangkat (*SharedPreferences*). Melalui integrasi penyimpanan lokal ini, sistem memungkinkan kelestarian akses dan retensi data pengguna meskipun siklus hidup aplikasi telah ditutup atau dihentikan sementara.

e. Diagram sekuens Operasi Data Riwayat

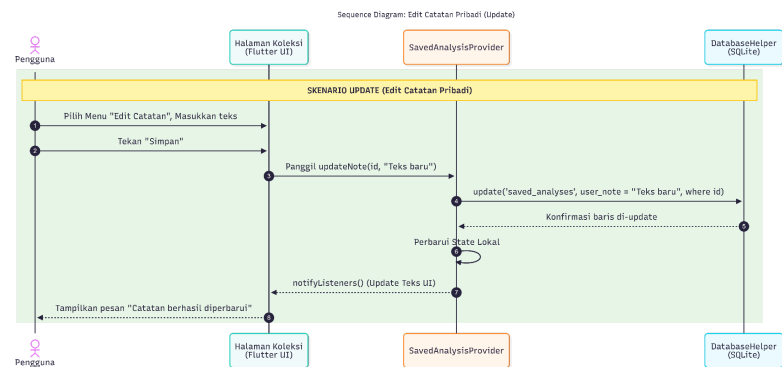
Diagram sekuens (*sequence diagram*) ini menggambarkan mekanisme interaksi sistematis antara antarmuka Flutter dengan kelas manajemen basis data (*DatabaseHelper*) dan komponen pengontrol (*SavedAnalysisProvider*). Interaksi komputasional tersebut bertujuan untuk menjalankan empat tugas fundamental operasi data, yakni *Create*, *Read*, *Update*, *Delete* (CRUD) secara terpusat pada tabel *saved_analyses* di dalam ekosistem basis data SQLite.



Gambar 1. Skenario READ (Memuat Daftar Riwayat)

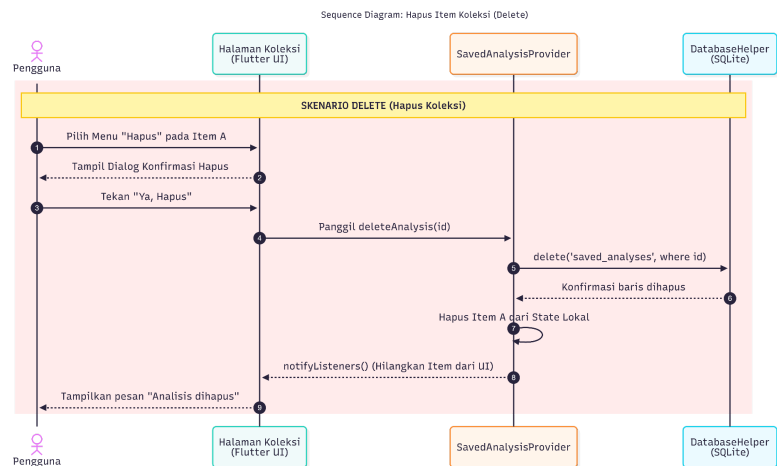
Pada skenario pembacaan data (*read*), sistem mengeksekusi kueri terhadap keseluruhan riwayat berdasarkan parameter alamat email pengguna, lalu menyortirnya secara dinamis agar koleksi bertanda bintang

(favorit) serta data termutakhir dapat ditampilkan pada urutan paling atas.



Gambar 1. Skenario UPDATE NOTE (Edit Catatan)

Skenario pembaruan (*update*) yang mencakup pengeditan catatan pribadi diimplementasikan dengan memanfaatkan konsep pembaruan optimis (*optimistic update*). Melalui pendekatan arsitektur ini, antarmuka pengguna (*user interface* atau UI) dirancang untuk merender pembaruan visual, seperti perubahan ikon bintang atau modifikasi teks, secara instan.



Gambar 1. Skenario DELETE (Hapus Koleksi)

Skenario penghapusan (*delete*) dikembangkan dengan lapisan pencegahan kesalahan kognitif untuk menghindari hilangnya riwayat analisis secara tidak sengaja. Sebelum data benar-benar dihilangkan, sistem secara otomatis akan memunculkan dialog konfirmasi pada layar untuk memvalidasi intensi tindakan pengguna. Apabila persetujuan diberikan, sistem baru akan mengeksekusi kueri penghapusan mutlak yang menargetkan baris parameter *ID* terkait, guna memastikan pangkalan data tetap bersih dan tidak ada sisa memori usang yang tertinggal di dalam penyimpanan lokal perangkat.

3. Tahap Perancangan Desain User Interface (Antarmuka Pengguna)

Perancangan antarmuka pengguna (*User Interface/UI*) pada aplikasi Klarip ditujukan untuk menciptakan pengalaman visual

yang modern, intuitif, dan nyaman bagi kelompok demografi Generasi Z. Mengingat pola konsumsi informasi Generasi Z yang serba cepat dan sangat mengutamakan estetika, aplikasi ini secara penuh mengusung gaya *Dark Mode* (Mode Gelap) sebagai tema bawaan secara utuh. Sebagai langkah awal dalam merealisasikan konsep tersebut, seluruh rancangan antarmuka ini dimanifestasikan ke dalam bentuk desain awal (*high-fidelity mockup*) yang dibangun menggunakan perangkat lunak Figma sebelum nantinya diimplementasikan ke dalam kode program Flutter.

a. Logo Aplikasi



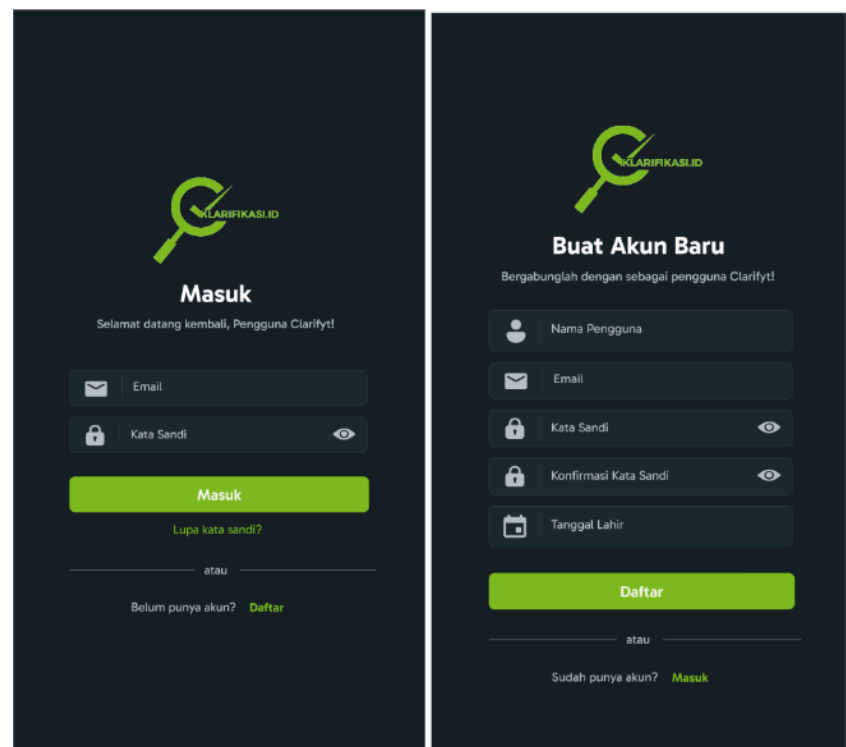
Gambar 1. Halaman Masuk dan Halaman Daftar (Login)

Perancangan identitas visual aplikasi direpresentasikan melalui desain logo yang esensial dan sarat akan makna filosofis. Logo aplikasi ini dibangun

melalui penggabungan harmonis antara ikonografi kaca pembesar (*magnifying glass*) dan tanda centang (*checkmark*). Elemen kaca pembesar secara gamblang menyimbolkan fungsi utama sistem sebagai instrumen penelusuran dan investigasi informasi di ruang digital.

b. Konsep Awal Halaman Login dan Register Aplikasi

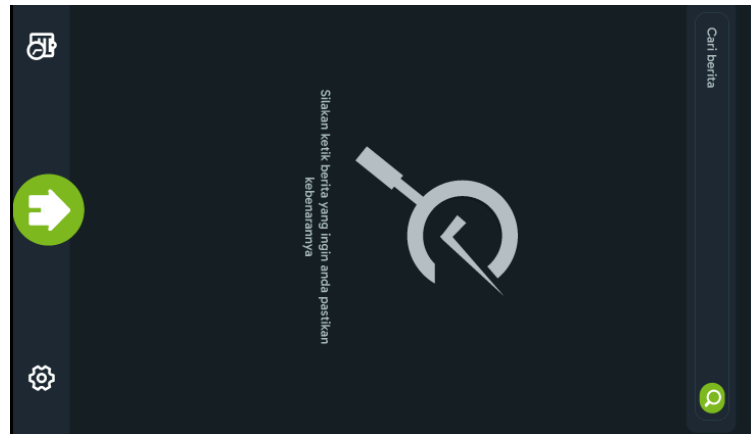
Halaman autentikasi berfungsi sebagai gerbang validasi identitas sebelum pengguna dapat mengakses fitur utama aplikasi.



Gambar 1. Halaman Masuk dan Halaman Daftar (Login)

c. Konsep Awal Halaman Utama / Pencarian

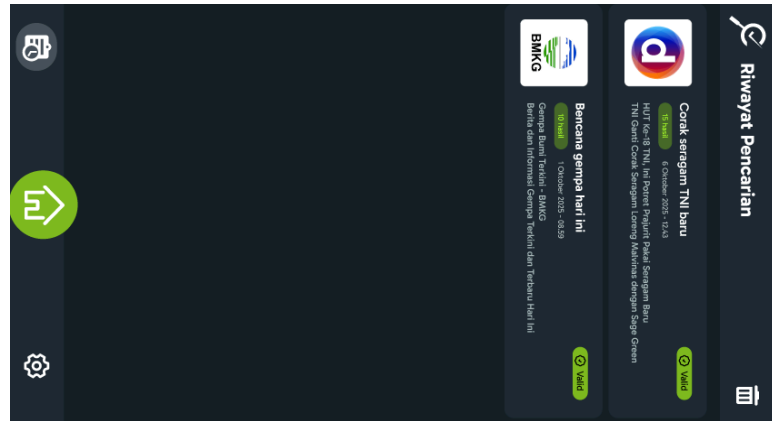
Halaman pencarian merupakan inti operasional (*core*) dari interaksi aplikasi, tempat pengguna memasukkan teks klaim berita untuk diverifikasi. Antarmuka halaman ini dirancang secara ringkas melalui empat komponen utama.



Gambar 1. Halaman Masuk dan Halaman Daftar (Login)

d. Konsep Awal Halaman Riwayat

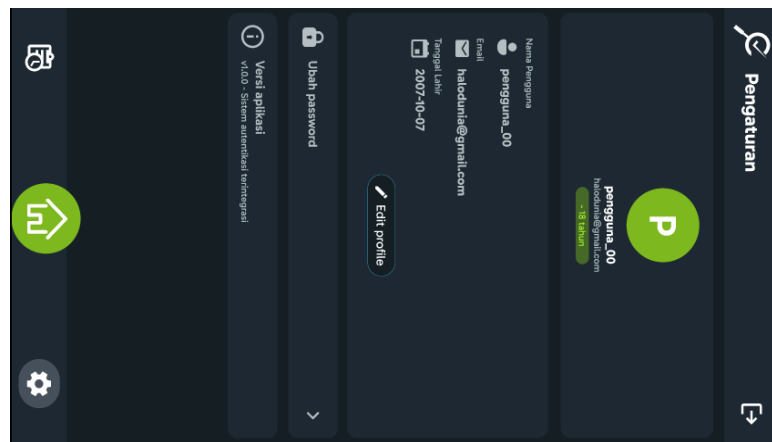
Halaman ini bertindak sebagai memori kognitif eksternal (*cognitive offloading*) yang menampung seluruh histori pengecekan fakta milik pengguna secara luring.



Gambar 1. Halaman Halaman Riwayat

e. Konsep Awal Halaman Profil & Pengaturan

Halaman terakhir pada *BottomNavigationBar* ini memuat informasi identitas dan manajemen keamanan pengguna secara ringkas melalui tiga komponen utama.



Gambar 1. Halaman Halaman Riwayat

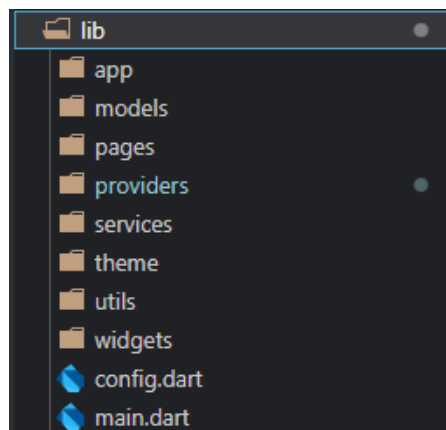
4. Tahap Implementasi Sistem

Tahap implementasi sistem merupakan fase pengembangan perangkat lunak, di mana seluruh cetak biru (*blueprint*) dan desain antarmuka yang telah dirancang pada tahap perancangan sistem

sebelumnya direalisasikan ke dalam bentuk baris kode pemrograman (*source code*). Pada penelitian ini, proses pengodean (*coding*) dieksekusi secara penuh menggunakan bahasa pemrograman Dart yang berjalan di atas ekosistem kerangka kerja (*framework*) Flutter.

a. Penjelasan Struktur Folder lib/ (Direktori Utama)

Seluruh kode program inti aplikasi berada di dalam folder **lib/**, yang terbagi menjadi beberapa sub-folder dengan tugas masing-masing.



Gambar 1. Halaman Halaman Riwayat

Folder / File	Fungsi
lib/app/	Berisi pengaturan global aplikasi: tema, navigasi, dan daftar <i>provider</i> .
lib/pages/	Berisi semua halaman yang dilihat pengguna (Login, Register, Cari, Koleksi, Profil).
lib/providers/	Berisi "otak" logika aplikasi yang menghubungkan tampilan dengan data.

<code>lib/services/</code>	Berisi kode yang berkomunikasi langsung dengan database dan API eksternal.
<code>lib/models/</code>	Berisi struktur/format data (contoh: struktur data Pengguna, struktur data Hasil Analisis).
<code>lib/theme/</code>	Berisi pengaturan tampilan global: warna, font, dan gaya tombol.
<code>lib/widgets/</code>	Berisi komponen-komponen tampilan kecil yang digunakan ulang di banyak halaman.
<code>lib/config.dart</code>	Berisi kunci API (<i>API Key</i>) bawaan untuk Google CSE dan Gemini AI.
<code>lib/main.dart</code>	Pintu masuk utama aplikasi. File pertama yang dijalankan saat aplikasi dibuka.

Gambar 1. Halaman Halaman Riwayat

b. Implementasi Operasi Verifikasi Klaim (Fungsi Utama)

Fungsi utama verifikasi klaim pada aplikasi Klarip beroperasi melalui integrasi dua layanan *Application Programming Interface* (API) eksternal. Proses komputasi ini mengadopsi metode *Retrieval-Augmented Generation* (RAG), yakni sebuah pendekatan yang memastikan kecerdasan buatan tidak menghasilkan kesimpulan secara fiktif, melainkan bersandar pada bukti aktual dari mesin pencari terlebih dahulu. Alur pemrosesan data tersebut diuraikan sebagai berikut:

Google Custom Search Engine (CSE) API: Sistem mengirimkan teks klaim pengguna untuk menelusuri artikel berita pembeding dari portal informasi yang terpilih. fail

yang bertanggung jawab: `lib/services/search_api.dart`.

Potongan kode berikut mendemonstrasikan mekanisme penyusunan dan pengiriman permintaan *HTTP GET* menuju peladen Google CSE:

```
// lib/services/search_api.dart
// Fungsi ini mengirimkan klaim ke Google CSE dan mendapatkan daftar artikel berita

Future<List<SearchResult>> _searchGoogleCSE(
  String query, {
    required String apiKey,
    required String cx,
  }) async {
  // Membuat alamat URL lengkap untuk pencarian Google CSE
  // lr=lang_id → hanya cari artikel berbahasa Indonesia
  // gl=id → prioritaskan hasil dari Indonesia
  // num=10 → ambil 10 artikel teratas
  final url = Uri.parse(
    'https://customsearch.googleapis.com/customsearch/v1'
    '?key=$apiKey&cx=$cx&q=$query&num=10&lr=lang_id&gl=id'
  );

  final response = await http.Client().get(url);

  if (response.statusCode == 200) {
    // Jika berhasil, ubah data JSON menjadi daftar objek SearchResult
    final data = jsonDecode(response.body);
    final items = data['items'] as List<dynamic>? ?? [];

    return items.map((item) {
      return SearchResult(
        title: item['title'],           // Judul artikel
        link: item['link'],             // Link URL artikel
        snippet: item['snippet'],       // Ringkasan singkat artikel
        displayLink: item['displayLink'], // Nama domain (misal: kompas.com)
      );
    }).toList();
  } else {
    // Jika gagal, tampilkan pesan error yang sesuai
    throw CseApiException(response.statusCode, 'Pencarian gagal');
  }
}
```

Gambar 1. Halaman Halaman Riwayat

Secara komputasional, blok fungsi `_searchGoogleCSE` mengeksekusi penarikan data secara otonom yang bekerja layaknya pencarian manual, namun dieksekusi melalui parameter *Uniform Resource Locator*

(URL) yang terstruktur. Guna memastikan hasil referensi memiliki relevansi yang tinggi bagi demografi sasaran di Indonesia, sistem secara ketat menyematkan parameter pembatasan bahasa (`lr=lang_id`) serta geolokasi wilayah (`gl=id`). Apabila peladen mengembalikan kode keberhasilan (`statusCode == 200`), perangkat lunak akan mengurai (*parsing*) susunan *JSON* untuk mengekstraksi *metadata* spesifik ke dalam array objek *SearchResult* yang siap dianalisis pada tahapan selanjutnya.

Gemini 2.5 *Flash-Lite* API: Bukti referensi berupa array objek *SearchResult* yang berhasil terkumpul kemudian ditransmisikan ke peladen Gemini AI bersamaan dengan teks klaim awal pengguna. Implementasi teknis untuk integrasi agen kecerdasan buatan ini dikelola di dalam fail `lib/services/gemini_service.dart`. Potongan kode berikut mengilustrasikan mekanisme perakitan instruksi (*prompt engineering*) dan pengiriman permintaan *HTTP POST* menuju peladen Gemini:

```
// lib/services/gemini_service.dart
// Fungsi ini merangkai semua bukti dan klaim menjadi sebuah "soal" untuk Gemini AI

String _buildPrompt(String claim, List<SearchResult> searchResults) {
  // Rangkai semua artikel bukti ke dalam satu teks
  StringBuffer sourcesBuffer = StringBuffer();
  for (int i = 0; i < searchResults.length; i++) {
    final r = searchResults[i];
    sourcesBuffer.writeln('--- Sumber ${i + 1} [${r.displayLink}] ---');
    sourcesBuffer.writeln('Judul: ${r.title}');
    sourcesBuffer.writeln('Ringkasan: ${r.snippet}');
  }

  // Kirimkan klaim + semua bukti ke Gemini AI
  // dan minta AI menjawab HANYA dalam format JSON
  return '''
Analisis klaim berikut secara kritis berdasarkan data sumber berita yang diberikan.

KLAIM: "$claim"

SUMBER DATA:
${sourcesBuffer.toString()}

FORMAT JAWABAN (JSON SAJA):
{
  "verdict": "DIDUKUNG_DATA" atau "TIDAK_DIDUKUNG_DATA" atau "MEMERLUKAN_VERIFIKASI",
  "explanation": "Ringkasan 2-3 kalimat penjelasan",
  "analysis": "Analisis mendalam 4-5 kalimat",
  "sources_used": ["nama-domain.com", ...]
}
JAWAB HANYA JSON!
''';
}
```

Gambar 1. Halaman Halaman Riwayat

Secara komputasional, fungsi `_buildPrompt` bertugas merangkai klaim awal pengguna dengan susunan referensi artikel yang diperoleh dari mesin pencari ke dalam satu kesatuan instruksi terstruktur (*prompt*). Rangkaian instruksi tersebut dikonfigurasi secara spesifik agar model bahasa komputasi hanya mengembalikan keluaran dalam format *JSON* (*JavaScript Object Notation*). Pendekatan pembatasan format ini bersifat esensial untuk memastikan bahwa antarmuka aplikasi dapat membedah (*parsing*) dan menerjemahkan hasil komputasi menjadi representasi visual kartu putusan (*3-tier verdict*) dengan presisi tinggi.

```

// Cara mengirimkan permintaan ke Gemini AI via HTTP POST
Future<GeminiAnalysis> analyzeClaim(String apiKey, String claim,
    List<SearchResult> searchResults) async {

    final url = Uri.parse(
        'https://generativelanguage.googleapis.com/v1beta/'
        'models/gemini-2.5-flash-lite:generateContent?key=$apiKey'
    );

    // Kirim permintaan ke Gemini AI
    final response = await http.post(
        url,
        headers: {'Content-Type': 'application/json'},
        body: jsonEncode({
            'contents': [
                {'parts': [{'text': _buildPrompt(claim, searchResults)]}]
            ],
            // temperature: 0.2 → supaya AI menjawab dengan faktual, tidak terlalu
            kreatif
            'generationConfig': {'temperature': 0.2, 'maxOutputTokens': 2048},
        }),
    );

    if (response.statusCode == 200) {
        // Ambil teks jawaban dari AI dan ubah ke objek GeminiAnalysis
        final data = jsonDecode(response.body);
        final text = data['candidates'][0]['content']['parts'][0]['text'];
        return _parseResponse(text, claim); // Parsing JSON dari jawaban AI
    }
    // ... penanganan error jika gagal
}

```

Gambar 1. Halaman Halaman Riwayat

Pada fungsi `analyzeClaim`, perangkat lunak mengeksekusi pengiriman paket data instruksi menuju peladen Gemini melalui protokol *HTTP POST*. Parameter kritis yang diterapkan pada tahap ini adalah konfigurasi nilai `temperature` sebesar 0.2. Pengaturan nilai parameter termal pada titik rendah tersebut secara teknis berfungsi untuk membatasi tingkat probabilitas atau "kreativitas" model kecerdasan buatan dalam menyusun kalimat. Melalui konfigurasi ini, keluaran sistem dipastikan bersifat lebih deterministik, faktual, dan sangat terikat pada bukti

referensi yang dilampirkan, sehingga risiko terjadinya halusinasi informasi (*AI hallucination*) dapat ditekan seminimal mungkin.

c. Implementasi Operasi Login dan Register (Data Pengguna)

Aplikasi Klarip mengimplementasikan sistem autentikasi yang beroperasi sepenuhnya di dalam perangkat keras pengguna secara luring (*offline*) tanpa memerlukan ketergantungan pada peladen awan (*cloud server*). Pendekatan arsitektur ini dipilih untuk menjamin keamanan kredensial dan privasi data pribadi dari risiko kebocoran data pada jaringan internet terpusat. Seluruh data akun pengguna disimpan secara persisten di dalam basis data relasional lokal melalui pustaka (*library*) *sqflite* yang dikonfigurasi sejak aplikasi pertama kali dijalankan oleh pengguna.

Penyimpanan Data Pengguna (Database SQLite): Komponen utama yang bertanggung jawab atas inisialisasi, pembuatan, dan pengelolaan pangkalan data lokal ini dikelola di dalam fail *lib/services/database_helper.dart*. Di dalam fail tersebut, basis data lokal dibuat dengan nama *klarip_v4.db*, di mana

struktur tabel `users` didefinisikan melalui potongan kode program sebagai berikut:

```
-- Tabel untuk menyimpan data akun pengguna yang terdaftar
CREATE TABLE users (
  id          INTEGER PRIMARY KEY AUTOINCREMENT, -- ID unik otomatis
  username    TEXT UNIQUE, -- Username (harus unik)
  email       TEXT UNIQUE, -- Email (harus unik, digunakan untuk login)
  password    TEXT,        -- Kata sandi
  full_name   TEXT,        -- Nama lengkap
  age         INTEGER,     -- Usia pengguna
  education   TEXT,        -- Tingkat pendidikan
  created_at  TEXT         -- Tanggal akun dibuat
)
```

Gambar 1. Halaman Halaman Riwayat

Secara struktur, tabel `users` dirancang dengan skema terstruktur untuk menyimpan parameter penting penunjang proses autentikasi luring. Kolom `id` dikonfigurasi sebagai kunci utama (*primary key*) dengan sifat *autoincrement* untuk menghasilkan nomor identitas unik secara otomatis bagi setiap akun baru. Kolom `username` dan `password` digunakan untuk merekam identitas nama serta sandi keamanan, sementara kolom `email` disematkan atribut unik (*UNIQUE*) guna mencegah terjadinya redundansi atau pendaftaran alamat email yang sama secara berulang. Seluruh tipe data pada kolom informasi tersebut menggunakan format teks (*TEXT*) untuk mempermudah proses enkripsi, pencocokan data, dan eksekusi kueri oleh pengontrol aplikasi.

Proses Pendaftaran Akun Baru (*Register*): Proses pendaftaran akun baru pada aplikasi Klarip dirancang untuk berjalan secara luring (*offline*) dengan melibatkan interaksi dinamis antara lapisan antarmuka pengguna dan lapisan pengontrol komponen logika bisnis. fungsional ini dikelola secara kolaboratif oleh fail `lib/pages/auth/register_page.dart` yang menangani penangkapan elemen masukan pengguna, serta fail `lib/providers/auth_provider.dart` yang bertanggung jawab atas validasi data menuju basis data lokal SQLite. Alur kerja dimulai secara asinkron (*asynchronous*) ketika pengguna menekan tombol aksi pada formulir pendaftaran, yang kemudian memicu pemanggilan fungsi internal `_handleRegister()`.

Pada lapisan antarmuka halaman pendaftaran, fungsi `_handleRegister()` bertindak sebagai filter awal untuk memastikan seluruh kolom masukan telah terpenuhi dan valid. Potongan kode program berikut menunjukkan bagaimana fungsi tersebut menginisialisasi status memuat (*loading state*) sebelum meneruskan parameter data menuju komponen *provider*:

```

// lib/pages/auth/register_page.dart
// Fungsi yang dijalankan saat tombol "Daftar Sekarang" ditekan

void _handleRegister() async {
  // Ambil teks dari semua kolom input
  final fullName = _fullNameController.text.trim();
  final username = _usernameController.text.trim();
  final email = _emailController.text.trim();
  final password = _passwordController.text.trim();

  // Pastikan semua kolom sudah diisi
  if (fullName.isEmpty || username.isEmpty || email.isEmpty || password.isEmpty) {
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Harap isi semua kolom'))
    );
    return;
  }

  // Kirim data ke AuthProvider untuk diproses
  final success = await context.read<AuthProvider>().register(
    fullName, username, email, password,
  );

  if (success) {
    // Jika berhasil, Langsung masuk ke halaman utama
    Navigator.of(context).pushAndRemoveUntil(
      MaterialPageRoute(builder: (_) => const HomeShell()),
      (route) => false,
    );
  } else {
    // Jika gagal (misal: email sudah terdaftar), tampilkan pesan error
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text(context.read<AuthProvider>().error ?? 'Registrasi gagal'))
    );
  }
}

```

Gambar 1. Halaman Halaman Riwayat

Setelah parameter nama pengguna, *email*, dan kata sandi berhasil dikirim oleh antarmuka halaman, kelas *AuthProvider* akan mengeksekusi fungsi *register()* untuk melakukan pemrosesan data tingkat lanjut. Di dalam fungsi ini, sistem menjalankan dua tahapan pemeriksaan keamanan dan persistensi data seperti yang diilustrasikan dalam potongan kode berikut:

```
// lib/providers/auth_provider.dart
// Fungsi yang memproses pendaftaran akun baru

Future<bool> register(String fullName, String username,
    String email, String password) async {
  // LANGKAH 1: Cek apakah email sudah pernah didaftarkan sebelumnya
  final existingUser = await _dbHelper.getUserByEmail(email);
  if (existingUser != null) {
    _error = 'Email sudah terdaftar'; // Tolak jika email sudah ada
    return false;
  }

  // LANGKAH 2: Jika email baru, simpan data pengguna ke database SQLite
  final newUser = User(
    username: username,
    email: email,
    password: password,
    fullName: fullName,
    createdAt: DateTime.now(),
  );
  await _dbHelper.insert('users', newUser.toMap());

  // LANGKAH 3: Setelah berhasil daftar, langsung login otomatis
  return await login(email, password);
}
```

Gambar 1. Halaman Halaman Riwayat

Secara mekanis, fungsi `register()` mengawali proses komputasi dengan melakukan kueri pemilahan (*query select*) menuju tabel `users` berdasarkan parameter *email* yang diinput oleh pengguna. Tahapan evaluasi ini sangat krusial untuk mendeteksi adanya redundansi data atau kepemilikan akun ganda, sehingga apabila array hasil kueri terbukti tidak kosong (`existingUser.isNotEmpty`), sistem akan melempar eksepsi galat (*throw exception*) untuk membatalkan alur pendaftaran. Sebaliknya, jika data *email* dinyatakan unik, sistem segera merakit objek map baru untuk dieksekusi menggunakan perintah kueri *INSERT* ke dalam SQLite lokal. Sesaat setelah baris data baru berhasil

tersimpan secara permanen, sistem secara otonom akan memperbarui status autentikasi internal perangkat lunak (*auto-login*) dan memicu fungsi `notifyListeners()` agar antarmuka pengguna langsung beralih ke menu utama tanpa mewajibkan pengguna untuk melakukan proses masuk ulang.

Proses Masuk ke Aplikasi (*Login*): Proses masuk ke dalam aplikasi (*login*) beroperasi melalui mekanisme autentikasi dua tahap yang mengintegrasikan validasi antarmuka (*client-side*) dan pencocokan pangkalan data. Implementasi antarmuka untuk fitur ini dikelola di dalam fail `lib/pages/auth/login_page.dart`, di mana eksekusi perintah bermuara pada fungsi `_handleLogin()`. Potongan kode program berikut mendemonstrasikan bagaimana sistem pada lapisan presentasi menangkap kredensial pengguna dan menengahi alur navigasi aplikasi:

```
// Lib/pages/auth/login_page.dart
// Fungsi yang dijalankan saat tombol "Masuk" ditekan

void _handleLogin() async {
  final email = _emailController.text.trim();
  final password = _passwordController.text.trim();

  // Pastikan kolom email dan password tidak kosong
  if (email.isEmpty || password.isEmpty) {
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Harap isi semua kolom'))
    );
    return;
  }

  // Kirim email & password ke AuthProvider untuk diverifikasi
  final success = await context.read<AuthProvider>().login(email, password);

  if (success) {
    // Jika email & password cocok, masuk ke halaman utama
    Navigator.of(context).pushAndRemoveUntil(
      MaterialPageRoute(builder: (_) => const HomeShell()),
      (route) => false,
    );
  } else {
    // Jika tidak cocok, tampilkan pesan error
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Email atau password salah'), backgroundColor:
Colors.red)
    );
  }
}
}
```

Gambar 1. Halaman Halaman Riwayat

Secara fungsional, perintah `_handleLogin()` berperan sebagai filter awal untuk memastikan kolom email (*email*) dan kata sandi (*password*) tidak dikirimkan dalam keadaan kosong. Apabila sistem mendeteksi ketidaklengkapan data masukan, eksekusi pemrosesan akan dibatalkan seketika dan antarmuka akan memunculkan umpan balik visual berupa komponen *SnackBar* yang memuat pesan peringatan. Sebaliknya, apabila data dinyatakan lengkap, parameter tersebut akan dievaluasi oleh komponen *provider* untuk menentukan status

keberhasilannya. Validasi yang berstatus sukses akan memicu sistem untuk menghapus tumpukan rute navigasi terdahulu dan mengalihkan layar menuju modul *HomeShell*, sedangkan penolakan akses akan memicu kemunculan pesan galat (*error*) berwarna merah.

Pada lapisan komputasi logika bisnis, penanganan data kredensial dieksekusi di dalam fail `lib/providers/auth_provider.dart` melalui fungsi `login()`. Komponen ini bertugas menjembatani komunikasi data antara antarmuka pengguna dengan eksekusi kueri pada basis data SQLite lokal dapat diamati melalui potongan kode berikut:

```
// lib/providers/auth_provider.dart
// Fungsi yang memverifikasi email dan password pengguna

Future<bool> login(String email, String password) async {
  // Cari data pengguna di database berdasarkan email DAN password
  // Jika keduanya cocok, data pengguna dikembalikan
  final userData = await _dbHelper.loginUser(email, password);

  if (userData != null) {
    // Login berhasil: simpan email ke SharedPreferences (memori ponsel)
    // agar pengguna tidak perlu Login ulang saat buka aplikasi lagi
    _currentUser = User.fromMap(userData);
    final prefs = await SharedPreferences.getInstance();
    await prefs.setString('user_email', email); // Simpan sesi login
    return true;
  } else {
    _error = 'Email atau password salah';
    return false;
  }
}
```

Gambar 1. Halaman Halaman Riwayat

Mekanisme operasional dalam fungsi `login()` diawali dengan operasi pencarian spesifik menggunakan

pencocokan ganda pada parameter email dan kata sandi di dalam tabel pengguna (*users*). Apabila kueri tersebut mengembalikan objek data yang tervalidasi (*userData != null*), sistem akan langsung memetakan informasi tersebut ke dalam struktur model internal perangkat lunak. Tahapan krusial selanjutnya adalah eksekusi modul *SharedPreferences* yang diperintahkan untuk merantai dan menanamkan data email sebagai sesi navigasi di dalam memori bawaan perangkat genggam. Implementasi manajemen sesi luring (*offline session*) ini secara otomatis membebaskan pengguna dari keharusan melakukan proses autentikasi berulang setiap kali perangkat lunak diluncurkan kembali.

Retensi Pengecekan Status Masuk (*Session Checking*): Aplikasi Klarip dirancang untuk memberikan pengalaman pengguna yang berkesinambungan melalui implementasi fitur retensi sesi navigasi. Mekanisme operasional ini memungkinkan pengguna untuk tetap berada dalam keadaan masuk (*login*) meskipun siklus hidup aplikasi telah dihentikan atau ditutup sepenuhnya oleh sistem operasi. Fungsionalitas manajemen sesi luring (*offline session management*) tersebut dikelola secara

otomatis di dalam fail `lib/providers/auth_provider.dart` sebagaimana diilustrasikan pada potongan kode berikut:

```
// lib/theme/app_theme.dart
// Definisi palet warna utama aplikasi Klarip

class AppTheme {
  // Warna latar belakang utama (Hitam gelap modern)
  static const Color backgroundDark = Color(0xFF121212);

  // Warna permukaan kartu dan panel
  static const Color surfaceDark = Color(0xFF1E1E1E);

  // Warna aksen utama aplikasi (Hijau, terinspirasi dari Spotify)
  // Hijau dipilih karena merepresentasikan validitas dan kepercayaan
  static const Color primarySeedColor = Color(0xFF1DB954);
}
```

Gambar 1. Halaman Halaman Riwayat

Secara komputasional, fungsi `_checkLoginStatus()` dieksekusi secara otonom oleh komponen *provider* pada fase inisialisasi awal setiap kali aplikasi diluncurkan. Langkah pertama yang dijalankan oleh sistem adalah mengakses memori bawaan perangkat genggam melalui eksekusi modul *SharedPreferences* guna mencari keberadaan kunci data email pengguna (`user_email`). Apabila rekam jejak sesi tersebut berhasil dideteksi di dalam penyimpanan lokal, perangkat lunak tidak serta-merta memberikan akses masuk, melainkan diwajibkan untuk melakukan tahapan validasi lanjutan ke dalam basis data SQLite.

Proses validasi tingkat lanjut tersebut diimplementasikan dengan memanggil fungsi pencarian

spesifik (`getUserByEmail`) guna memastikan bahwa identitas akun yang bersangkutan masih terdaftar secara valid di dalam tabel pengguna. Jika pangkalan data mengembalikan objek yang sesuai, sistem akan langsung memetakan informasi tersebut ke dalam struktur model internal dan memintas (*bypass*) halaman autentikasi menuju menu utama. Pendekatan arsitektur ini secara efektif membebaskan pengguna dari beban repetitif untuk memasukkan kredensial email dan kata sandi (*password*) secara berulang, sehingga meningkatkan efisiensi waktu serta kenyamanan interaksi secara keseluruhan.

d. Implementasi Operasi Data Riwayat

Fitur koleksi fakta yang terintegrasi pada modul navigasi utama merupakan subsistem manajemen riwayat yang dirancang khusus untuk mengelola rekam jejak aktivitas verifikasi pengguna. Sistem ini memfasilitasi antarmuka yang memungkinkan pengguna untuk menyimpan, mengorganisasi, serta mengarsipkan hasil analisis faktualitas klaim yang telah dieksekusi pada tahapan sebelumnya. Melalui mekanisme pengarsipan ini, perangkat lunak menyediakan fasilitas memori eksternal (*cognitive offloading*) yang mempermudah pengguna dalam

melacak kembali informasi krusial tanpa harus mengulang proses komputasi pencarian ke layanan pihak ketiga.

Seluruh data riwayat analisis tersebut dipersistensikan secara mutlak ke dalam basis data SQLite lokal yang tertanam langsung di dalam memori internal perangkat keras pengguna. Pendekatan arsitektur penyimpanan luring (*offline storage*) ini diterapkan secara sengaja untuk memastikan ketersediaan dan keandalan data secara berkelanjutan terlepas dari kondisi stabilitas jaringan internet. Dengan arsitektur tersebut, pengguna dipastikan tetap memiliki hak akses penuh terhadap daftar arsip verifikasi mereka meskipun perangkat sedang beroperasi dalam keadaan tanpa koneksi (*offline*), yang sekaligus memperkuat komitmen sistem terhadap perlindungan privasi data karena tidak melibatkan proses transmisi riwayat ke peladen awan (*cloud server*).

Struktur Data Riwayat (*Model*) : Implementasi struktur data riwayat pada aplikasi Klarip dikelola dengan memanfaatkan konsep pemodelan objek (*object modeling*) yang merepresentasikan entitas pangkalan data di dalam kode program. Representasi data untuk setiap entri hasil verifikasi didefinisikan di dalam sebuah kelas bernama

`SavedAnalysis` yang terletak pada fail `lib/models/saved_analysis.dart`. Kelas ini bertindak sebagai cetak biru (*blueprint*) konseptual yang memetakan nilai-nilai keluaran komputasi kecerdasan buatan sebelum akhirnya dipersistensikan ke dalam tabel basis data SQLite lokal. Potongan kode berikut menunjukkan deklarasi atribut yang menyusun kelas `SavedAnalysis` tersebut:

```
// lib/models/saved_analysis.dart
// Kelas ini mendefinisikan format data satu item riwayat analisis

class SavedAnalysis {
  final int? id; // ID unik di database (otomatis)
  final String title; // Judul ringkas analisis
  final String claim; // Teks klaim yang diverifikasi
  final String verdict; // Hasil keputusan AI (DIDUKUNG_DATA /
  TIDAK_DIDUKUNG_DATA / MEMERLUKAN_VERIFIKASI)
  final String explanation; // Penjelasan singkat dari AI
  final String analysis; // Analisis mendalam + daftar sumber artikel
  final String confidence; // Tingkat keyakinan AI
  final String userNote; // Catatan pribadi yang ditulis oleh pengguna
  final DateTime savedAt; // Waktu dan tanggal penyimpanan
  final bool isFavorite; // Apakah ditandai bintang (favorit)?
  final String userEmail; // Email pengguna pemilik data ini
}
```

Gambar 1. Halaman Halaman Riwayat

Secara struktural, kelas pemodelan ini merangkum berbagai atribut esensial yang dirancang menyerupai format kartu arsip digital untuk merekam setiap aktivitas verifikasi. Atribut pokok seperti `claim`, `verdict`, `explanation`, dan `analysis` berfungsi untuk menyimpan substansi hasil penelusuran fakta yang secara langsung diproduksi oleh layanan analitik Gemini AI. Selain itu, model ini juga mengakomodasi parameter fungsional pendukung, seperti

`userNote` untuk menyimpan anotasi personal pengguna, atribut *boolean* `isFavorite` sebagai penanda status prioritas, serta `userEmail` yang berfungsi sebagai pengikat relasi data (*relational binding*) guna memastikan riwayat tersebut hanya dapat diakses oleh akun pemiliknya.

Operasi Menyimpan Riwayat Baru (*Create*):

Proses penyimpanan hasil verifikasi fakta ke dalam basis data lokal diimplementasikan melalui operasi pembuatan data (*create*) yang dikelola oleh komponen pengontrol status aplikasi (*state management*). Fungsionalitas manajemen data ini secara teknis ditempatkan di dalam fail `lib/providers/saved_analysis_provider.dart` untuk memisahkan logika bisnis dari lapisan presentasi visual. Eksekusi komputasi bermula ketika pengguna menekan tombol simpan pada antarmuka hasil pencarian, yang kemudian secara asinkron memicu pemanggilan fungsi `addAnalysis()` untuk merekam objek data ke dalam SQLite. Potongan kode program berikut mendemonstrasikan mekanisme pengikatan identitas pengguna dan eksekusi penyimpanan ke dalam pangkalan data:

```

// lib/providers/saved_analysis_provider.dart
// Fungsi untuk menyimpan satu hasil analisis baru ke database

Future<void> addAnalysis(SavedAnalysis analysis) async {
  // Ambil email pengguna yang sedang login dari memori ponsel
  final prefs = await SharedPreferences.getInstance();
  final userEmail = prefs.getString('user_email') ?? '';

  // Gabungkan data analisis dengan email pengguna
  // agar riwayat ini hanya muncul untuk akun yang menyimpannya
  final newAnalysis = analysis.copyWith(userEmail: userEmail);

  // Simpan ke database SQLite
  await _dbHelper.insert('saved_analyses', newAnalysis.toMap());

  // Muat ulang daftar agar tampilan langsung diperbarui
  await loadAnalyses();
}

```

Gambar 1. Halaman Halaman Riwayat

Secara operasional, fungsi `addAnalysis()` mengawali pemrosesan dengan mengakses modul *SharedPreferences* guna mengekstraksi parameter surel (*email*) dari sesi akun yang sedang aktif. Langkah ekstraksi ini merupakan mekanisme esensial untuk menciptakan label identitas kepemilikan mutlak pada objek data hasil kecerdasan buatan sebelum dimasukkan ke dalam memori perangkat. Sistem kemudian memanfaatkan metode penyalinan dan modifikasi objek bawaan, yakni `copyWith`, untuk menggabungkan parameter email tersebut dengan parameter analisis ke dalam satu entitas data yang utuh.

Setelah objek data baru terbentuk secara terstruktur, sistem mengeksekusi metode penyisipan (`insert`) untuk merekam seluruh parameter tersebut ke dalam tabel

`saved_analyses` di basis data SQLite lokal. Sesaat pasca operasi penyimpanan dikonfirmasi berhasil, pengontrol sistem secara otonom memanggil fungsi pemuatan ulang kueri (`loadAnalyses()`) agar perubahan pangkalan data langsung terefleksikan pada antarmuka pengguna secara instan (*real-time*). Pendekatan pelabelan identitas ini diimplementasikan guna memastikan isolasi data secara ketat, sehingga privasi riwayat penelusuran fakta antar-pengguna tidak akan pernah saling tumpang tindih meskipun aplikasi diakses secara bergantian melalui perangkat keras yang sama.

Operasi Memuat Daftar Riwayat (*Read*): Proses pembacaan dan pemuatan data riwayat (*read*) diimplementasikan untuk menyajikan rekam jejak verifikasi kepada pengguna melalui antarmuka modul koleksi. Fungsionalitas ini dikelola secara terpusat oleh komponen pengontrol `SavedAnalysisProvider` yang bertugas mengambil data dari basis data SQLite secara asinkron (*asynchronous*). Mekanisme pemuatan data tersebut dirancang dengan menerapkan filter kepemilikan secara ketat agar privasi riwayat setiap pengguna tetap terisolasi dengan aman di dalam perangkat. Potongan kode program berikut mengilustrasikan mekanisme kueri pemilahan data

serta algoritma pengurutan yang diterapkan pada daftar riwayat:

```
// lib/providers/saved_analysis_provider.dart
// Fungsi untuk memuat semua riwayat milik pengguna yang sedang login

Future<void> loadAnalyses() async {
  _isLoading = true;
  notifyListeners(); // Tampilkan indikator loading di UI

  // Ambil email pengguna yang sedang login
  final prefs = await SharedPreferences.getInstance();
  final userEmail = prefs.getString('user_email') ?? '';

  // Ambil hanya data yang user_email-nya cocok dengan yang login
  final data = await _dbHelper.queryAll(
    'saved_analyses',
    where: 'user_email = ?', // Filter berdasarkan akun pengguna
    whereArgs: [userEmail],
  );

  // Ubah data dari format database menjadi daftar objek SavedAnalysis
  _analyses = data.map((e) => SavedAnalysis.fromMap(e)).toList();

  // Urutkan: Favorit di atas, lalu yang paling baru
  _sortAnalyses();

  _isLoading = false;
  notifyListeners(); // Perbarui tampilan
}

// Fungsi pengurutan: favorit muncul paling atas,
// kemudian diurutkan dari yang paling baru
void _sortAnalyses() {
  _analyses.sort((a, b) {
    if (a.isFavorite && !b.isFavorite) return -1; // a lebih dulu
    if (!a.isFavorite && b.isFavorite) return 1; // b lebih dulu
    return b.savedAt.compareTo(a.savedAt); // Terbaru dulu
  });
}
```

Gambar 1. Halaman Halaman Riwayat

Secara mekanis, eksekusi fungsi `loadAnalyses()` diawali dengan mengaktifkan status memuat (*loading state*) untuk memberikan umpan balik visual pada antarmuka pengguna saat proses pengambilan data berlangsung.

Sistem kemudian mengekstraksi parameter surel dari modul *SharedPreferences* yang kemudian digunakan sebagai argumen filter pada kueri pemilahan (*query select*). Melalui penerapan klausa kondisional **WHERE user_email = ?**, pangkalan data diinstruksikan untuk hanya mengembalikan baris data yang memiliki relasi langsung dengan sesi akun yang sedang aktif, sehingga mencegah terjadinya kebocoran data antar-pengguna. Kumpulan data mentah yang dikembalikan kemudian dipetakan (*mapping*) menjadi array objek **SavedAnalysis** untuk mempermudah proses manipulasi data pada tahapan selanjutnya.

Setelah objek data riwayat berhasil dimuat ke dalam memori aplikasi, sistem secara otomatis mengeksekusi fungsi internal **_sortAnalyses()** untuk mengatur hierarki penyajian informasi secara dinamis. Algoritma pengurutan ini menerapkan dua tingkat prioritas yang bekerja secara sekuensial untuk mengoptimalkan pengalaman pengguna. Prioritas utama diberikan kepada entri yang memiliki status bernilai benar (*true*) pada atribut **isFavorite**, sehingga data yang ditandai dengan ikon bintang akan selalu dirender pada urutan paling atas. Selanjutnya, prioritas kedua didasarkan pada parameter waktu penyimpanan (**savedAt**), yang mengurutkan sisa baris data secara menurun

(*descending*) agar hasil verifikasi fakta yang paling mutakhir tampil mendahului data yang lebih usang.

Operasi Mengubah Catatan Pribadi (*Update*):

Fasilitas penambahan anotasi personal diimplementasikan untuk memberikan ruang bagi pengguna dalam menyematkan konteks atau kesimpulan tambahan pada setiap rekam jejak verifikasi fakta. Fungsionalitas operasional ini merepresentasikan skenario pembaruan data (*update*) pada ekosistem basis data relasional. Logika pemrosesan untuk modifikasi catatan tersebut dikendalikan di dalam fail `lib/providers/saved_analysis_provider.dart` melalui pemanggilan fungsi `updateNote()`. Potongan kode program berikut mendemonstrasikan mekanisme pembaruan pangkalan data dan sinkronisasi tampilan secara bersamaan:

```
// lib/providers/saved_analysis_provider.dart
// Fungsi untuk menyimpan atau mengubah catatan pribadi pengguna

Future<void> updateNote(int id, String newNote) async {
  // Perbarui kolom user_note di database untuk item dengan id tertentu
  await _dbHelper.update(
    'saved_analyses',
    {'user_note': newNote}, // Hanya kolom ini yang diperbarui
    'id = ?',
    [id],
  );

  // Perbarui juga data di memori agar tampilan langsung berubah
  final index = _analyses.indexWhere((element) => element.id == id);
  if (index != -1) {
    _analyses[index] = _analyses[index].copyWith(userNote: newNote);
    notifyListeners();
  }
}
```

Gambar 1. Halaman Halaman Riwayat

Secara komputasional, fungsi `updateNote()` beroperasi dengan menerima dua parameter esensial, yakni identitas unik riwayat (`id`) dan untaian teks anotasi baru (`newNote`). Eksekusi modifikasi diawali dengan mengirimkan kueri `UPDATE` menuju basis data SQLite melalui perantara kelas *helper*. Kueri ini dirancang secara spesifik untuk hanya menargetkan dan memperbarui nilai pada kolom `user_note` dengan mengikat klausa kondisional `WHERE id = ?`, sehingga integritas parameter informasi lain pada baris yang sama dapat dipertahankan secara mutlak.

Setelah modifikasi pangkalan data luring dikonfirmasi berhasil, sistem tidak melakukan pemuatan ulang (*requery*) terhadap keseluruhan isi tabel untuk menampilkan perubahan tersebut. Alih-alih membebani kinerja perangkat, pengontrol sistem menerapkan modifikasi langsung pada variabel memori aplikasi dengan mencari posisi indeks data yang relevan di dalam array objek. Saat indeks yang tepat telah ditemukan, metode `copyWith` diinstruksikan untuk menyalin objek dan menimpa nilai catatan lawas dengan yang baru, lalu diakhiri dengan pemanggilan perintah `notifyListeners()`

agar antarmuka pengguna memperbarui tampilan secara seketika (*real-time*).

Operasi Menghapus Riwayat (*Delete*): Operasi penghapusan data riwayat (*delete*) merupakan fungsionalitas krusial dalam manajemen basis data untuk memberikan kendali penuh kepada pengguna atas ruang penyimpanan lokal mereka. Fasilitas ini mengizinkan pengguna untuk menyingkirkan rekam jejak verifikasi fakta yang dianggap sudah tidak relevan, salah, atau usang. Implementasi mekanisme penghapusan pada aplikasi Klarip dirancang secara komprehensif dengan mengintegrasikan lapisan logika komputasi pada komponen *provider* dan lapisan pencegahan kesalahan pada antarmuka pengguna.

Pada lapisan komputasi logika bisnis, eksekusi penghapusan dikelola di dalam fail `lib/providers/saved_analysis_provider.dart` melalui pemanggilan fungsi `deleteAnalysis()`. Potongan kode program berikut mengilustrasikan mekanisme penghapusan data pada SQLite yang dibarengi dengan pembaruan memori internal:

```
// lib/providers/saved_analysis_provider.dart
// Fungsi untuk menghapus satu item riwayat berdasarkan ID-nya

Future<void> deleteAnalysis(int id) async {
  // Hapus dari database SQLite berdasarkan id
  await _dbHelper.delete('saved_analyses', 'id = ?', [id]);

  // Hapus juga dari daftar di memori agar tampilan langsung berubah
  // tanpa perlu memuat ulang seluruh data dari database
  _analyses.removeWhere((element) => element.id == id);

  notifyListeners(); // Perbarui tampilan
}
```

Gambar 1. Halaman Halaman Riwayat

Secara operasional, fungsi `deleteAnalysis()` menerima parameter identitas unik (`id`) yang kemudian dimanfaatkan sebagai kondisi presisi pada klausa `WHERE id = ?` untuk mengeksekusi kueri `DELETE` di dalam tabel SQLite. Serupa dengan efisiensi pada operasi pembaruan data sebelumnya, sistem tidak memuat ulang keseluruhan pangkalan data setelah baris data terhapus. Alih-alih melakukan kueri ulang, pengontrol sistem langsung mengeliminasi elemen objek terkait dari array memori menggunakan metode `removeWhere()`, lalu menginstruksikan antarmuka untuk memperbarui status (*state*) secara seketika guna menghemat alokasi sumber daya pemrosesan.

Sementara itu, pada lapisan antarmuka pengguna yang terletak di dalam fail `lib/pages/saved_page.dart`, sistem menerapkan lapisan pelindung preventif sesaat

sebelum fungsi penghapusan mutlak dieksekusi. Implementasi perlindungan ini diwujudkan melalui komponen kotak dialog konfirmasi (*alert dialog*) yang dikendalikan oleh fungsi `_confirmDelete()` seperti pada potongan kode berikut:

```
// Lib/pages/saved_page.dart
// Dialog konfirmasi sebelum menghapus

void _confirmDelete(BuildContext context) {
  showDialog(
    context: context,
    builder: (ctx) => AlertDialog(
      title: const Text('Hapus Koleksi?'),
      content: const Text('Tindakan ini tidak dapat dibatalkan.'),
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(ctx), // Batal
          child: const Text('Batal'),
        ),
        TextButton(
          onPressed: () {
            // Hapus dari database dan tutup sheet
            context.read<SavedAnalysisProvider>()
              .deleteAnalysis(widget.item.id!);
            Navigator.pop(ctx); // Tutup dialog
            Navigator.pop(context); // Tutup detail sheet
          },
          child: const Text('Hapus', style: TextStyle(color: Colors.red)),
        ),
      ],
    ),
  );
}
```

Gambar 1. Halaman Halaman Riwayat

Pemanggilan kotak dialog konfirmasi ini berfungsi sebagai penahan kognitif (*cognitive friction*) dalam prinsip desain pengalaman pengguna (*user experience*). Penahanan ini dirancang untuk memastikan bahwa tindakan destruktif terhadap pangkalan data benar-benar dilandasi oleh intensi sadar pengguna dan bukan sekadar ketidaksengajaan sentuhan pada layar. Mengingat bahwa data yang telah

dihapus dari basis data luring tidak memiliki mekanisme pemulihan (*recovery*), sistem memberikan opsi pembatalan atau persetujuan akhir sebelum memicu fungsi `deleteAnalysis()` secara permanen.

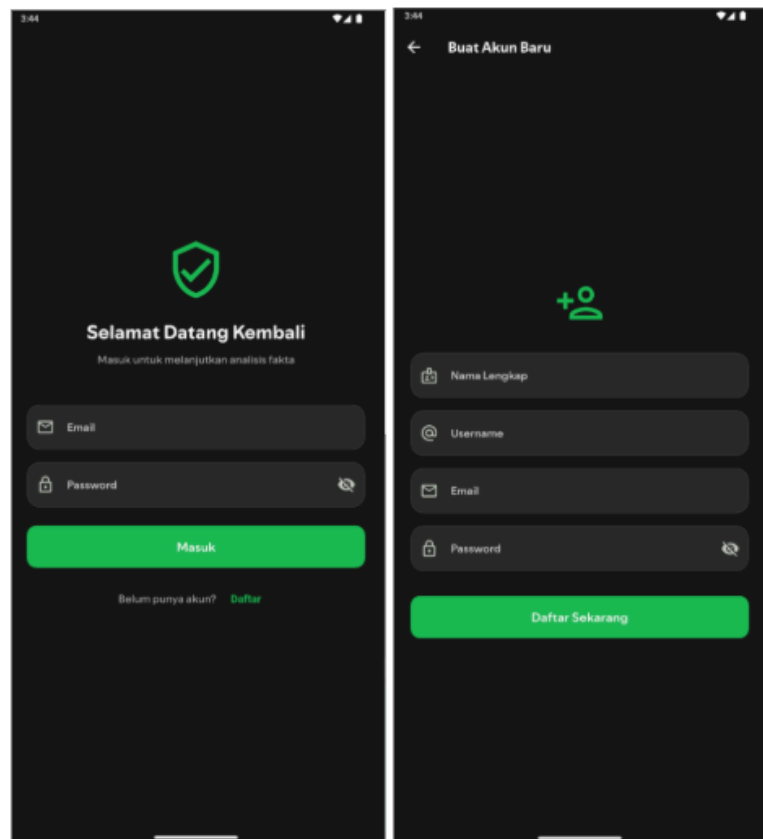
5. Tahap Implementasi Desain User Interface (Antarmuka Pengguna)

Tahap implementasi desain antarmuka pengguna (*User Interface/UI*) merupakan proses transformasi dari Tahap Perancangan Desain User Interface (Antarmuka Pengguna ke dalam bentuk komponen baris kode pemrograman menggunakan kerangka kerja Flutter. Pada tahap eksekusi ini, aplikasi Klarip secara penuh mengusung gaya *Dark Mode* (mode gelap) sebagai tema visual bawaan (*default theme*). Pemilihan palet warna latar yang gelap, dipadukan dengan warna aksen hijau solid, dirancang secara khusus untuk meminimalisasi kelelahan mata (*eye strain*) sekaligus menciptakan identitas visual yang modern dan relevan dengan karakteristik preferensi estetika kelompok demografi Generasi Z.

a. Implementasi Halaman Masuk (*Login*) dan Daftar (*Register*)

Halaman autentikasi merupakan gerbang interaksi pertama yang berfungsi sebagai lapisan validasi identitas

sebelum pengguna diizinkan untuk mengakses fungsionalitas utama perangkat lunak. Pada halaman masuk (*login*), antarmuka menampilkan elemen visual utama berupa ikon perisai bertanda centang yang secara psikologis memperkuat kesan keamanan sistem, diikuti oleh dua kolom isian kredensial pokok, yakni surel (*email*) dan kata sandi (*password*).



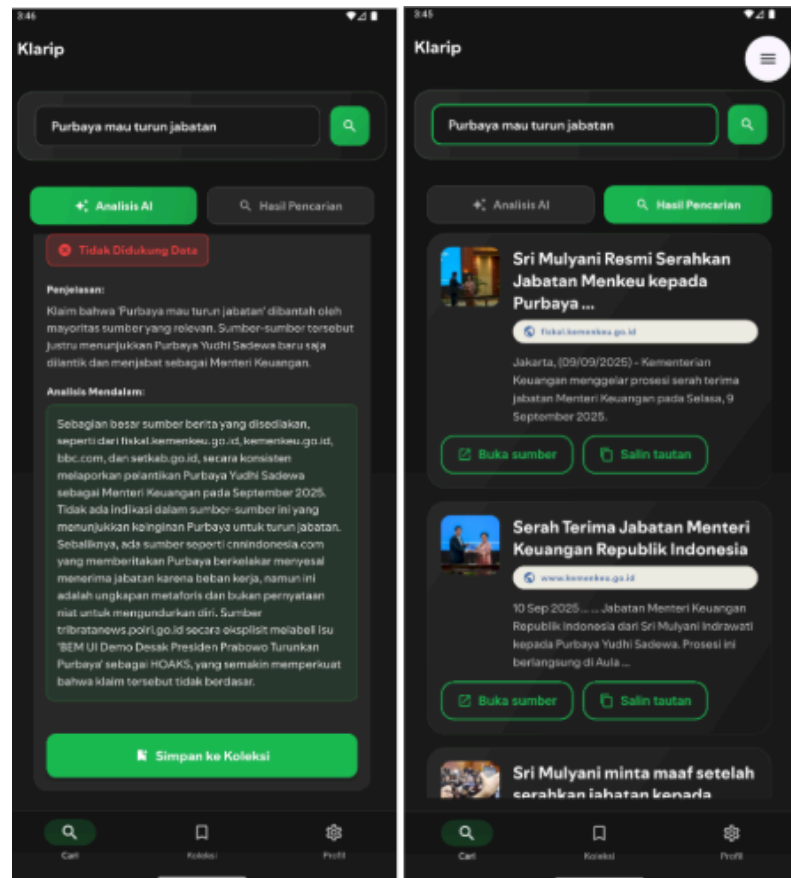
Gambar 1. Halaman Halaman Riwayat

Sementara itu, representasi visual pada halaman daftar (*register*) memiliki struktur tata letak yang konsisten

dan selaras dengan halaman masuk, namun dilengkapi dengan ekstensi kolom isian tambahan berupa Nama Lengkap dan *Username* guna mengakomodasi perekaman data identitas pengguna baru secara komprehensif. Kedua halaman autentikasi ini mengimplementasikan tombol aksi utama (*primary button*) yang diarsir dengan warna hijau kontras, sebuah strategi desain yang bertujuan agar elemen tersebut tampil menonjol (*prominent*) sehingga mampu memandu fokus pengguna dalam menyelesaikan proses validasi secara intuitif.

b. Implementasi Halaman Implementasi Halaman Utama dan Pencarian

Halaman utama atau pencarian bertindak sebagai pusat operasional interaktif di mana proses verifikasi klaim bermula. Pada bagian atas antarmuka, sistem menampilkan sebuah kolom pencarian (*search bar*) persisten yang didampingi oleh tombol aksi pencarian untuk menangkap masukan teks dari pengguna. antarmuka ini mengimplementasikan sistem navigasi berbasis tab (*toggle buttons*) yang membagi area konten menjadi dua panel utama, yakni panel "Analisis AI" dan "Hasil Pencarian".



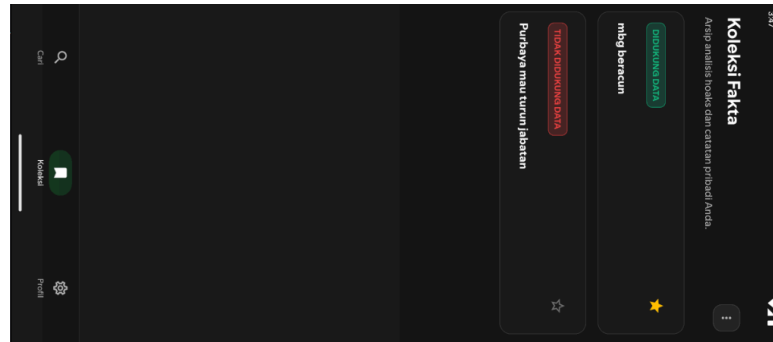
Gambar 1. Halaman Halaman Riwayat

Pada saat pengguna mengakses panel "Hasil Pencarian", sistem merender sekumpulan kartu informasi dinamis yang memuat daftar artikel referensi hasil ekstraksi layanan antarmuka pemrograman aplikasi (*API*) Google *Custom Search Engine* (CSE). Setiap kartu hasil pencarian dirancang secara terstruktur untuk menampilkan *metadata* esensial yang mencakup judul berita, tautan domain pilihan, dan cuplikan teks (*snippet*) singkat. Guna menjaga prinsip transparansi informasi, setiap kartu tersebut juga dilengkapi

dengan tombol interaktif tambahan berupa opsi "Buka sumber" dan "Salin tautan" yang memfasilitasi pengguna untuk melakukan peninjauan langsung terhadap literatur referensi asli.

c. Implementasi Halaman Riwayat

Halaman riwayat yang bertajuk "Koleksi Fakta" diimplementasikan sebagai antarmuka manajemen data yang merepresentasikan fungsi memori kognitif eksternal (*cognitive offloading*) bagi pengguna. Secara visual, antarmuka ini merender data riwayat ke dalam bentuk daftar komponen kartu (*card*) vertikal yang masing-masing memuat judul atau teks klaim yang pernah diverifikasi beserta lencana status putusan (*verdict badge*) berwarna spesifik. Guna meningkatkan fleksibilitas manajemen data, antarmuka tersebut juga dilengkapi dengan fitur ikon bintang interaktif pada sudut kanan atas setiap kartu yang difungsikan untuk memberikan tanda prioritas atau status favorit pada arsip informasi tertentu.



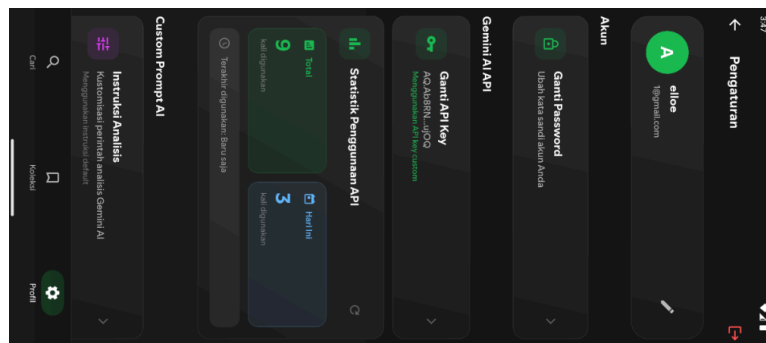
Gambar 1. Halaman Halaman Riwayat

Penyajian susunan informasi pada halaman ini dikonfigurasi agar tersortir secara dinamis berdasarkan parameter kronologis dan tingkat urgensi. Kartu riwayat yang telah ditandai dengan ikon bintang secara komputasional akan dipaksa untuk tampil pada urutan teratas, yang kemudian diikuti oleh daftar riwayat penelusuran paling mutakhir di bawahnya. Selain itu, seluruh sajian data yang di render pada antarmuka ini ditarik secara langsung dari basis data SQLite lokal yang beroperasi secara luring, sehingga menjamin pengguna tetap dapat mengakses seluruh arsip verifikasi mereka secara utuh meskipun perangkat sedang tidak terhubung dengan jaringan internet.

d. Implementasi Halaman Riwayat

Halaman profil dan pengaturan diimplementasikan sebagai pusat kendali utama bagi pengguna untuk

mengelola identitas personal dan tingkat keamanan akun. Pada bagian paling atas antarmuka, sistem menampilkan komponen kartu profil (*profile card*) yang memuat informasi esensial seperti inisial avatar, nama pengguna, serta alamat surel (*email*) yang sedang aktif. Melalui integrasi komponen interaktif pada bagian "Akun", pengguna diberikan aksesibilitas penuh untuk melakukan pembaruan data diri secara dinamis maupun memodifikasi lapisan keamanan melalui menu penggantian kata sandi (*password*).



Gambar 1. Halaman Halaman Riwayat

Sebagai pengembangan fungsionalitas tingkat lanjut dari desain purwarupa awal, antarmuka ini juga mengakomodasi panel konfigurasi khusus untuk mengelola integrasi layanan kecerdasan buatan. Pengguna diberikan fleksibilitas tingkat tinggi melalui modul *Gemin AI API*, yang memungkinkan mereka untuk menanamkan kunci

antarmuka pemrograman aplikasi (*API key*) milik mereka sendiri secara mandiri (kustom). Pada panel yang sama, sistem juga menyediakan fitur *Custom Prompt AI* memfasilitasi pengguna untuk melakukan penyesuaian instruksi atau parameter perintah agar tingkat ketegasan analisis kecerdasan buatan dapat disesuaikan dengan kebutuhan verifikasi fakta yang spesifik.

Untuk mengakomodasi penghentian aktivitas pemakaian, sistem menempatkan tombol aksi keluar sesi (*logout*) yang direpresentasikan secara intuitif dengan ikon interaktif berwarna merah pada sudut kanan atas layar. Eksekusi pada tombol tersebut akan memicu modul pengontrol (*provider*) untuk membersihkan seluruh data sesi navigasi secara mutlak dari dalam memori penyimpanan internal perangkat (*SharedPreferences*). Sesaat setelah sesi akses tersebut berhasil dihancurkan secara komputasional, aplikasi akan secara otomatis mendepak pengguna dan mengembalikan alur navigasi menuju gerbang halaman autentikasi awal.

6. Tahap Pengujian Fungsionalitas Sistem (*Black Box Testing*)

Tahap pengujian fungsionalitas sistem pada penelitian ini dieksekusi secara komprehensif menggunakan metode pengujian

kotak hitam (*black box testing*). Pendekatan evaluasi ini diimplementasikan untuk memvalidasi kesesuaian antara masukan (*input*) yang diberikan oleh pengguna dengan keluaran (*output*) yang dihasilkan oleh sistem, tanpa perlu meninjau struktur alur kode internal dari perangkat lunak. Pengujian dilakukan secara mandiri oleh peneliti menggunakan instrumen lembar panduan skenario (*testing checklist*) yang mencakup seluruh fitur krusial aplikasi, mulai dari modul Modul Autentikasi (Login & Register), Modul Verifikasi Klaim (Pencarian & Analisis AI), Modul Riwayat (Koleksi Fakta), Modul Profil & Pengaturan Akun, Modul Pengaturan API Key, dan Modul Navigasi.

a. Modul Autentikasi (Login & Register)

No	Skenario Uji	Data Masukan (Input)	Hasil yang Diharapkan	Status
1	Registrasi akun baru dengan data lengkap dan valid	Nama: "Alfath", Username: "alfath123", Email: "alfath@email.com", Password: "pass123"	Akun berhasil dibuat dan pengguna langsung masuk ke halaman utama (HomeShell)	 Berhasil
2	Registrasi dengan email yang sudah terdaftar	Email: "alfath@email.com" (sudah ada di database)	Muncul notifikasi "Email sudah terdaftar" dan registrasi tidak dilanjutkan	 Berhasil
3	Registrasi dengan kolom kosong	Salah satu atau semua kolom dibiarkan kosong, tekan "Daftar"	Muncul peringatan "Harap isi semua kolom" pada <i>SnackBar</i>	 Berhasil
4	Login dengan email dan	Email: "alfath@email.com",	Login berhasil, sesi tersimpan, pengguna	 Berhasil

	password benar	Password: "pass123"	diarahkan ke HomeShell	
5	Login dengan password salah	Email: "alfath@email.com", Password: "salahpassword"	Muncul notifikasi "Email atau password salah" berwarna merah	✓ Berhasil
6	Login dengan email yang belum terdaftar	Email: "belumada@email.com", Password: "pass"	Muncul notifikasi "Email atau password salah"	✓ Berhasil
7	Login dengan kolom kosong	Email/Password kosong, tekan "Masuk"	Muncul peringatan "Harap isi semua kolom"	✓ Berhasil
8	Fitur <i>Show/Hide Password</i> di halaman Login	Menekan ikon mata pada kolom password	Teks kata sandi berubah dari titik-titik (*) menjadi teks biasa, dan sebaliknya	✓ Berhasil
9	Fitur <i>Show/Hide Password</i> di halaman Register	Menekan ikon mata pada kolom password dan konfirmasi password	Teks kata sandi berubah dari titik-titik (*) menjadi teks biasa, dan sebaliknya	✓ Berhasil
10	Pengecekan <i>Auto-Login</i> (Sesi Tersimpan)	Menutup aplikasi secara penuh (<i>force close</i>) lalu membukanya kembali	Pengguna langsung masuk ke menu utama tanpa harus login ulang	✓ Berhasil
11	Navigasi dari Login ke Register	Menekan tombol "Belum punya akun? Daftar"	Layar berpindah ke halaman Registrasi	✓ Berhasil
12	Navigasi dari Register ke Login	Menekan tombol "Sudah punya akun? Masuk"	Layar berpindah ke halaman Login	✓ Berhasil

b. Modul Verifikasi Klaim (Pencarian & Analisis AI)

No	Skenario Uji	Data Masukan (Input)	Hasil yang Diharapkan	Status
13	Memasukkan klaim valid dan menekan "Cari"	Teks klaim: "Vaksin COVID-19 mengandung mikrociip"	Menampilkan <i>skeleton loading</i> , memanggil API CSE & Gemini, lalu menampilkan hasil analisis	✓ Berhasil
14	Memasukkan klaim kosong	Kolom teks kosong, tekan "Cari"	Pencarian tidak dieksekusi, muncul peringatan "Masukkan teks klaim terlebih dahulu"	✓ Berhasil

15	Pengujian <i>Rate Limit / Spam Filter</i>	Menekan tombol "Cari" berulang kali dalam waktu kurang dari 5 detik	Tombol dinonaktifkan sementara, muncul notifikasi "Harap tunggu sebelum mencari lagi"	✓ Berhasil
16	Render Tab "Analisis AI" saat pencarian berhasil	Pencarian selesai	Menampilkan lencana <i>Verdict</i> (Didukung/Tidak/Perlu Verifikasi), ringkasan penjelasan, dan analisis mendalam	✓ Berhasil
17	Render Tab "Hasil Pencarian" saat pencarian berhasil	Berpindah ke tab "Hasil Pencarian"	Menampilkan daftar kartu artikel dari Google CSE (Judul, Snippet, URL Domain)	✓ Berhasil
18	Kesalahan API Key Google CSE	Mengubah config dengan API Key CSE yang salah / <i>expired</i>	Muncul <i>Error Banner</i> merah: "Gagal memuat hasil pencarian (Cek koneksi atau kuota API)"	✓ Berhasil
19	Kesalahan API Key Gemini AI	Mengubah config dengan API Key Gemini yang salah / <i>expired</i>	Muncul <i>Error Banner</i> merah: "Gagal menghubungi AI. Pastikan API Key valid."	✓ Berhasil
20	Kondisi tidak ada sinyal (Offline) saat mencari	Mematikan koneksi internet (Wi-Fi/Data), tekan "Cari"	Muncul notifikasi "Tidak ada koneksi internet" atau kegagalan penelusuran (<i>Exception handled</i>)	✓ Berhasil
21	Aksi "Buka Sumber" pada kartu artikel Google CSE	Menekan tombol "Buka sumber" di salah satu artikel	Peramban web (browser) bawaan HP terbuka dan memuat URL artikel tersebut	✓ Berhasil
22	Aksi "Salin Tautan" pada kartu artikel Google CSE	Menekan tombol "Salin tautan" di salah satu artikel	URL tersalin ke <i>clipboard</i> perangkat, muncul notifikasi "Tautan disalin"	✓ Berhasil

c. Modul Riwayat (Koleksi Fakta)

No	Skenario Uji	Data Masukan (Input)	Hasil yang Diharapkan	Status
23	Menyimpan hasil analisis baru ke Koleksi	Menekan tombol "Simpan ke Koleksi" setelah analisis AI selesai	Muncul notifikasi "Berhasil disimpan", data masuk ke database SQLite	✓ Berhasil

24	Menyimpan hasil analisis yang sama dua kali	Menekan tombol "Simpan ke Koleksi" untuk kedua kalinya	Muncul peringatan "Klaim ini sudah ada di koleksi Anda"	 Berhasil
25	Membuka Tab Koleksi setelah menyimpan data	Klik ikon navigasi "Koleksi" di bawah layar	Menampilkan daftar riwayat dalam bentuk kartu (<i>card</i>)	 Berhasil
26	Penyortiran data riwayat secara bawaan	Buka tab Koleksi	Data diurutkan dari yang terbaru disimpan ke yang terlama (<i>Descending</i>)	 Berhasil
27	Membuka detail riwayat	Mengetuk salah satu kartu riwayat di daftar Koleksi	Membuka <i>Detail Sheet</i> yang menampilkan klaim, <i>verdict</i> , penjelasan, dan analisis lengkap AI	 Berhasil
28	Memberi tanda "Favorit" (Bintang) pada riwayat	Mengetuk ikon bintang (☆) pada kartu riwayat	Ikon berubah menjadi penuh (★) warna kuning, data di update di SQLite	 Berhasil
29	Menyortir berdasarkan "Favorit"	Memberi bintang pada riwayat lawas, lalu <i>refresh</i> /buka ulang tab	Riwayat berbintang selalu muncul di urutan paling atas daftar	 Berhasil
30	Menghapus tanda "Favorit"	Mengetuk ulang ikon bintang (★)	Ikon kembali kosong (☆), data kembali ke urutan normal berdasarkan waktu	 Berhasil
31	Menambah "Catatan Pribadi" pada riwayat	Membuka opsi (titik tiga) -> "Edit Catatan", input teks: "Ini buat tugas" -> Simpan	Teks "Ini buat tugas" tersimpan dan langsung muncul di kartu riwayat UI	 Berhasil
32	Mengubah "Catatan Pribadi" yang sudah ada	Membuka "Edit Catatan" pada item yang sudah ada catatannya, ubah teks -> Simpan	Teks catatan berhasil diperbarui di UI dan database	 Berhasil
33	Membatalkan penghapusan riwayat	Membuka opsi -> "Hapus", muncul dialog konfirmasi, pilih "Batal"	Dialog tertutup, data tidak jadi dihapus dari koleksi	 Berhasil

34	Menghapus riwayat (Delete)	Membuka opsi -> "Hapus", muncul dialog, pilih "Ya, Hapus"	Kartu riwayat hilang dari daftar secara instan, muncul pesan "Analisis dihapus"	 Berhasil
35	Memastikan isolasi data antar pengguna	Login dengan Akun A, simpan riwayat X. Logout. Login dengan Akun B. Buka koleksi.	Akun B tidak dapat melihat riwayat X milik Akun A (Tabel kosong)	 Berhasil
36	Kondisi koleksi kosong	Membuka tab Koleksi pada akun baru (belum ada data)	Menampilkan ilustrasi dan teks "Belum ada riwayat koleksi"	 Berhasil
37	Membaca riwayat tanpa koneksi internet	Mematikan data/Wifi, buka tab Koleksi	Daftar riwayat dan detail analisis tetap dapat dibuka dan dibaca dengan normal	 Berhasil

d. Modul Profil & Pengaturan Akun

No	Skenario Uji	Data Masukan (Input)	Hasil yang Diharapkan	Status
38	Memuat halaman Profil	Klik ikon "Profil" di navigasi bawah	Menampilkan data avatar (inisial), Username, dan Email pengguna yang sedang aktif	 Berhasil
39	Mengganti Kata Sandi dengan password lama yang SALAH	Buka menu Ganti Password. Input Password Lama: "salah", Password Baru: "baru123"	Pembaruan ditolak, muncul notifikasi "Password lama salah"	 Berhasil
40	Mengganti Kata Sandi dengan password lama yang BENAR	Input Password Lama: "pass123" (benar), Password Baru: "pass456"	Pembaruan berhasil, muncul notifikasi "Password berhasil diubah", data SQLite terupdate	 Berhasil
41	Fitur Keluar Sesi (<i>Logout</i>)	Menekan ikon pintu keluar (Logout) di sudut kanan atas profil	Muncul dialog konfirmasi "Yakin ingin keluar?"	 Berhasil
42	Konfirmasi <i>Logout</i>	Menekan "Ya, Keluar" pada dialog konfirmasi	Sesi <i>SharedPreferences</i> dihapus, pengguna diarahkan kembali ke halaman Login	 Berhasil

e. Modul Pengaturan Gemini AI API

No	Skenario Uji	Data Masukan (Input)	Hasil yang Diharapkan	Status
43	Mengubah API Key Gemini (Custom Key)	Masuk menu Pengaturan API Key, input API Key milik pengguna: "AIzaSy_contoh", tekan Simpan	Muncul notifikasi "API Key berhasil disimpan". Status UI berubah jadi "Menggunakan API Key Custom" berwarna hijau.	 Berhasil
44	Mereset API Key ke setelan bawaan	Menekan tombol "Reset ke Default"	API Key custom dihapus dari memori, UI berubah menjadi "Menggunakan API Key Bawaan" berwarna abu-abu.	 Berhasil
45	Melakukan pencarian menggunakan API Key Custom yang Valid	Teks klaim: "Bumi itu datar" (Saat API Key Custom aktif)	Analisis berjalan lancar menggunakan API Key milik pengguna	 Berhasil
46	Mengubah Custom Prompt Instruksi AI	Masuk menu Instruksi Analisis, ubah teks menjadi: "Jawablah dengan gaya bahasa gaul", tekan Simpan	Instruksi tersimpan di memori.	 Berhasil
47	Melakukan pencarian dengan Custom Prompt	Teks klaim: "Makan micin bikin bodoh"	AI menghasilkan analisis yang menyesuaikan dengan gaya instruksi baru (Gaya gaul)	 Berhasil

f. Navigasi & Antarmuka Umum (UI/UX)

No	Skenario Uji	Data Masukan (Input)	Hasil yang Diharapkan	Status
48	Perpindahan Tab Navigasi Bawah	Mengetuk ikon Cari, Koleksi, dan	Layar berpindah dengan cepat ke halaman yang dituju tanpa memuat ulang aplikasi dari awal	 Berhasil

	(<i>Bottom Navigation Bar</i>)	Profil secara bergantian		
49	Responsivitas Keyboard Virtual (UI Overlap)	Mengetuk kolom isian (contoh: di halaman pencarian atau registrasi)	Keyboard virtual muncul dan layar otomatis terdorong ke atas (tidak menutupi kolom isian)	 Berhasil
50	<i>Dark Mode</i> Consistency	Berpindah-pindah antar semua menu aplikasi	Seluruh palet warna (background, surface, text) tetap konsisten dalam mode gelap dengan baik, tidak ada elemen yang terpotong	 Berhasil

g. Rekapitulasi Hasil Pengujian

Modul	Jumlah Skenario	Berhasil	Tidak Berhasil
A. Autentikasi (Login & Register)	12	12	0
B. Verifikasi Klaim	10	10	0
C. Riwayat (Koleksi Fakta)	15	15	0
D. Profil & Pengaturan Akun	5	5	0
E. Pengaturan API Key	5	5	0
F. Navigasi & Antarmuka Umum	3	3	0
TOTAL	50	50	0

$$TSR = \frac{50}{50} \times 100\%$$

Berdasarkan hasil pengujian Black Box yang telah dilakukan terhadap 50 skenario uji yang mencakup seluruh modul fungsional aplikasi Klarip, diperoleh hasil bahwa seluruh 50 skenario (100%) berjalan sesuai dengan hasil yang diharapkan. Tidak ditemukan adanya kegagalan fungsi pada modul autentikasi, verifikasi klaim, manajemen riwayat, pengaturan profil, pengelolaan API Key, maupun navigasi antarmuka. Dengan demikian, dapat disimpulkan bahwa aplikasi Klarip telah memenuhi seluruh kebutuhan fungsional yang telah dirancang sebelumnya dan siap untuk dilanjutkan ke tahap pengujian usabilitas menggunakan metode *System Usability Scale* (SUS).

7. Hasil Pengujian Efisiensi Waktu (*Time on Task*)

Pengukuran metrik efisiensi waktu pemrosesan (*time on task*) dilakukan untuk membandingkan durasi penelusuran fakta secara manual konvensional (menggunakan peramban web Google) dengan durasi verifikasi otomatis menggunakan aplikasi Klarip. Pengambilan data dieksekusi secara langsung menggunakan instrumen *stopwatch* dengan presisi 0,1 detik terdapat 20 skenario klaim politik yang sering beredar di media sosial.

a. Waktu Verifikasi Klaim dengan Metode Pencarian

Manual

No	Klaim yang Diuji	Durasi (detik)	Catatan
1	Prabowo Subianto resmi memotong anggaran pendidikan sebesar 20% untuk membiayai program Makan Bergizi Gratis	287	Butuh konfirmasi dari situs resmi Kemendikbud dan Kemenkeu, beberapa sumber campur aduk
2	Presiden Prabowo mengganti Purbaya Yudhi Sadewa dari jabatan Menteri Keuangan akibat desakan demonstrasi mahasiswa	412	Perlu mencari pernyataan resmi Istana, banyak artikel spekulasi yang muncul lebih dulu
3	Gibran Rakabuming Raka mengundurkan diri dari jabatan Wakil Presiden karena terbukti memalsukan ijazah sarjananya	356	Isu sensitif dan viral, banyak artikel opini bercampur dengan laporan faktual
4	Pemerintah resmi memblokir platform X (Twitter) secara permanen di seluruh Indonesia demi meredam kritik publik	445	Perlu menelusuri pernyataan resmi Kementerian Kominfo dan pengumuman resmi Komdigi
5	Dana haji tahun ini akan dialihkan sepenuhnya untuk mendanai pembangunan infrastruktur Ibu Kota Nusantara (IKN)	298	Pernyataan resmi BPIH dan Kemenag cukup mudah ditemukan untuk dibandingkan
6	Joko Widodo diangkat menjadi Ketua Umum Partai Golkar secara aklamasi menggantikan Bahlil Lahadalia	378	Perlu mencari berita resmi dari DPP Golkar, banyak artikel dari media kurang kredibel
7	Mahkamah Konstitusi (MK) membatalkan hasil Pilkada Jakarta dan memerintahkan KPU melakukan pemungutan suara ulang	421	Harus merujuk langsung ke situs resmi MK untuk membaca putusan, prosesnya memakan waktu

8	DPR RI resmi mengesahkan UU perpanjangan masa jabatan kepala desa menjadi 12 tahun tanpa sidang paripurna	263	Berita yang sempat ramai, sumber debunking dari media resmi relatif mudah ditemukan
9	Sri Mulyani diperiksa KPK terkait dugaan korupsi dana bantuan sosial (bansos) saat masih menjabat Menteri Keuangan	389	Perlu konfirmasi dari situs resmi KPK dan pernyataan jubi, banyak artikel tidak akurat
10	Pemerintah membatalkan program makan siang gratis karena defisit APBN yang membengkak	467	Paling lama — perlu membandingkan banyak data APBN dari sumber Kemenkeu yang valid
11	Pajak Pertambahan Nilai (PPN) resmi dinaikkan menjadi 15% secara sepihak untuk menutup utang luar negeri	315	Perlu menelusuri Peraturan Pemerintah dan berita resmi Ditjen Pajak
12	Anies Baswedan ditunjuk sebagai Utusan Khusus Presiden untuk urusan Timur Tengah oleh Presiden Prabowo	442	Perlu konfirmasi dari Sekretariat Kabinet, banyak berita simpang siur beredar
13	BEM UI dan BEM UGM resmi menyatakan makar dan menolak mengakui keabsahan pemerintahan Prabowo-Gibran	273	Isu yang ramai, pernyataan resmi BEM UI/UGM cukup mudah ditemukan di media mahasiswa
14	Tiongkok meminta Pulau Kalimantan sebagai jaminan atas utang proyek Kereta Cepat Jakarta-Surabaya yang gagal bayar	396	Klaim ekstrem, perlu menelusuri dokumen perjanjian dan pernyataan Kementerian Luar Negeri
15	Ridwan Kamil resmi ditunjuk sebagai Kepala Otorita IKN menggantikan Basuki Hadimuljono mulai bulan depan	458	Perlu konfirmasi dari situs resmi Otorita IKN dan pengumuman Istana

16	RUU Perampasan Aset resmi disahkan DPR RI melalui sidang paripurna tertutup yang dihadiri kurang dari separuh anggota	324	Perlu mengecek risalah sidang DPR dan pernyataan Baleg yang valid
17	Gaji PNS, TNI, dan Polri resmi dipotong 5% setiap bulan untuk mendanai program Tabungan Perumahan Rakyat (Tapera)	387	Isu Tapera yang pernah ramai, perlu mencari PP dan pernyataan resmi pemerintah
18	Presiden Prabowo menolak menggunakan fasilitas mobil dinas dan meminta kementerian menjual seluruh mobil dinas menteri	253	Berita yang pernah viral dan banyak diliput, debunking tersedia cukup cepat
19	KPU terbukti memanipulasi server Sirekap setelah ditemukan kebocoran jutaan data pemilih oleh peretas Bjorka	418	Perlu menelusuri laporan resmi Bawaslu, BSSN, dan klarifikasi KPU secara terpisah
20	Subsidi BBM jenis Pertalite resmi dihapus secara total di seluruh SPBU per tanggal 1 bulan ini	341	Perlu konfirmasi dari Pertamina dan Kementerian ESDM, banyak artikel belum diverifikasi
	Rata-Rata	366,2	
	Minimum	253	
	Maksimum	467	

b. Waktu Verifikasi Klaim dengan Aplikasi Klarip

No	Klaim yang Diuji	Durasi (detik)	Catatan
1	Prabowo Subianto resmi memotong anggaran pendidikan sebesar 20% untuk membiayai program Makan Bergizi Gratis	7,2	CSE menemukan banyak artikel relevan, Gemini memberikan verdict cepat dan terstruktur

2	Presiden Prabowo mengganti Purbaya Yudhi Sadewa dari jabatan Menteri Keuangan akibat desakan demonstrasi mahasiswa	8,9	Analisis AI sedikit lebih panjang karena melibatkan nama pejabat dan konteks politik
3	Gibran Rakabuming Raka mengundurkan diri dari jabatan Wakil Presiden karena terbukti memalsukan ijazah sarjananya	6,4	Hasil verdict langsung muncul: Tidak Didukung Data, dengan penjelasan ringkas
4	Pemerintah resmi memblokir platform X (Twitter) secara permanen di seluruh Indonesia demi meredam kritik publik	9,1	Gemini memberikan analisis konteks kebijakan yang sedikit lebih panjang
5	Dana haji tahun ini akan dialihkan sepenuhnya untuk mendanai pembangunan infrastruktur Ibu Kota Nusantara (IKN)	5,8	Waktu tercepat — artikel dari CSE sangat relevan dan verdict langsung tersedia
6	Joko Widodo diangkat menjadi Ketua Umum Partai Golkar secara aklamasi menggantikan Bahlil Lahadalia	8,3	Sumber berita berlimpah di CSE, analisis Gemini stabil dan akurat
7	Mahkamah Konstitusi (MK) membatalkan hasil Pilkada Jakarta dan memerintahkan KPU melakukan pemungutan suara ulang	7,7	Verdict jelas dengan penjelasan konteks hukum dari Gemini
8	DPR RI resmi mengesahkan UU perpanjangan masa jabatan kepala desa menjadi 12 tahun tanpa sidang paripurna	6,1	Klaim populer, banyak artikel debunking ditemukan oleh CSE
9	Sri Mulyani diperiksa KPK terkait dugaan korupsi dana bantuan sosial (bansos) saat masih menjabat Menteri Keuangan	9,4	Topik sensitif dan spesifik, Gemini membutuhkan konteks lebih sebelum memberikan verdict
10	Pemerintah membatalkan program makan siang gratis karena defisit APBN yang membengkak	7,6	Respons konsisten, Gemini memberikan analisis anggaran yang terperinci
11	Pajak Pertambahan Nilai (PPN) resmi dinaikkan menjadi 15%	8,8	AI memberikan analisis kebijakan fiskal yang

	secara sepihak untuk menutup utang luar negeri		mendalam namun tetap cepat
12	Anies Baswedan ditunjuk sebagai Utusan Khusus Presiden untuk urusan Timur Tengah oleh Presiden Prabowo	5,9	Hasil cepat, artikel CSE langsung relevan dengan klaim
13	BEM UI dan BEM UGM resmi menyatakan makar dan menolak mengakui keabsahan pemerintahan Prabowo-Gibran	9,2	CSE menemukan banyak sumber, Gemini menganalisis istilah hukum "makar" dengan hati-hati
14	Tiongkok meminta Pulau Kalimantan sebagai jaminan atas utang proyek Kereta Cepat Jakarta-Surabaya yang gagal bayar	6,7	Verdict cepat: Tidak Didukung Data, penjelasan singkat namun tepat sasaran
15	Ridwan Kamil resmi ditunjuk sebagai Kepala Otorita IKN menggantikan Basuki Hadimuljono mulai bulan depan	8,1	Gemini memeriksa konteks jabatan dan memberikan analisis yang terperinci
16	RUU Perampasan Aset resmi disahkan DPR RI melalui sidang paripurna tertutup yang dihadiri kurang dari separuh anggota	7,3	Respons stabil, penjelasan prosedur legislatif yang akurat dari Gemini
17	Gaji PNS, TNI, dan Polri resmi dipotong 5% setiap bulan untuk mendanai program Tabungan Perumahan Rakyat (Tapera)	9,6	Waktu terlama — Gemini menganalisis aturan Tapera secara mendalam dan kontekstual
18	Presiden Prabowo menolak menggunakan fasilitas mobil dinas dan meminta kementerian menjual seluruh mobil dinas menteri	6,5	Respons sangat cepat, banyak artikel berita mendukung konfirmasi klaim
19	KPU terbukti memanipulasi server Sirekap setelah ditemukan kebocoran jutaan data pemilih oleh peretas Bjorka	8,4	AI memberikan verdict dengan kehati-hatian karena melibatkan klaim teknis dan hukum
20	Subsidi BBM jenis Pertalite resmi dihapus secara total di seluruh SPBU per tanggal 1 bulan ini	7,0	Analisis ringkas, verdict jelas dengan sumber artikel dari Pertamina dan ESDM

	Rata-Rata	7,7	
	Minimum	5,8	
	Maksimum	9,6	

c. Perbandingan dan Analisis Efisiensi Waktu

No	Klaim (Ringkas)	Manual (dtk)	Klarip (dtk)	Selisih (dtk)	Penghematan
1	Prabowo potong anggaran pendidikan 20%	287	7,2	279,8	97,5%
2	Prabowo ganti Menkeu Purbaya	412	8,9	403,1	97,8%
3	Gibran mundur karena ijazah palsu	356	6,4	349,6	98,2%
4	Twitter/X diblokir permanen	445	9,1	435,9	97,9%
5	Dana haji dialihkan ke IKN	298	5,8	292,2	98,1%
6	Jokowi jadi Ketum Golkar	378	8,3	369,7	97,8%
7	MK batalkan Pilkada Jakarta	421	7,7	413,3	98,2%
8	Kades jabatan 12 tahun tanpa paripurna	263	6,1	256,9	97,7%
9	Sri Mulyani diperiksa KPK soal bansos	389	9,4	379,6	97,6%
10	Program MBG dibatalkan defisit APBN	467	7,6	459,4	98,4%
11	PPN naik jadi 15% sepihak	315	8,8	306,2	97,2%

12	Anies jadi utusan khusus Timteng	442	5,9	436,1	98,7%
13	BEM UI/UGM nyatakan makar	273	9,2	263,8	96,6%
14	Tiongkok minta Kalimantan jaminan utang	396	6,7	389,3	98,3%
15	Ridwan Kamil jadi Kepala Otorita IKN	458	8,1	449,9	98,2%
16	RUU Perampasan Aset disahkan tertutup	324	7,3	316,7	97,7%
17	Gaji PNS/TNI/Polri dipotong 5% Tapera	387	9,6	377,4	97,5%
18	Prabowo tolak mobil dinas menteri	253	6,5	246,5	97,4%
19	KPU manipulasi Sirekap oleh Bjorka	418	8,4	409,6	98,0%
20	Subsidi Pertalite dihapus total	341	7,0	334,0	97,9%
	Rata-Rata	366,2	7,7	358,5	97,9%

Rumus perhitungan pada Percobaan

$$Efisiensi = \left(\frac{T_{Manual} - X_{Aplikasi}}{\bar{T}_{Manual}} \right) \times 100\%$$

Contoh perhitungan pada Percobaan 1:

$$Efisiensi = \left(\frac{287 - 7,2}{287} \right) \times 100\% = \left(\frac{279,8}{287} \right) \times 100\% = 97,5\%$$

d. Ringkasan Statistik

Statistik	Manual (detik)	Klarip (detik)
Rata-rata	366,2	7,7
Nilai Minimum	253	5,8
Nilai Maksimum	467	9,6
Penghematan Rata-rata	—	358,5 detik
Persentase Efisiensi	—	97,9%
Kecepatan Relatif	—	±47,6× lebih cepat

$$Efisiensi\ rata\ rata = \left(\frac{366,2 - 7,2}{366,2} \right) \times 100\% = \left(\frac{358,5}{366,2} \right) = 97,9\%$$

Berdasarkan 20 percobaan yang dilakukan pada kedua kondisi menggunakan klaim-klaim seputar isu politik dan pejabat publik Indonesia yang pernah beredar di media sosial, diperoleh hasil bahwa:

Pencarian manual membutuhkan rata-rata 366,2 detik (±6 menit 6 detik) untuk menyelesaikan satu proses verifikasi klaim politik, dengan rentang waktu antara 4 menit 13 detik hingga 7 menit 47 detik. Lamanya waktu disebabkan oleh perlunya menyaring berbagai artikel opini,

berita palsu, dan pernyataan resmi dari berbagai sumber yang tidak selalu mudah ditemukan.

Aplikasi Klarip hanya membutuhkan rata-rata 7,7 detik untuk menyelesaikan proses verifikasi yang sama, dengan rentang waktu antara 5,8 detik hingga 9,6 detik.

Aplikasi Klarip terbukti $\pm 47,6$ kali lebih cepat dibandingkan metode manual, dengan tingkat penghematan waktu rata-rata sebesar 97,9%.

Hasil ini mengonfirmasi bahwa otomatisasi sistem verifikasi berbasis Google Custom Search Engine dan Gemini AI yang diimplementasikan dalam aplikasi Klarip mampu memangkas durasi penelusuran klaim hoaks khususnya klaim-klaim politik yang memerlukan pengecekan lintas sumber secara sangat signifikan, sekaligus mengurangi beban kognitif dan risiko terpapar disinformasi yang dialami pengguna awam.

8. Hasil Pengujian Usabilitas (*System Usability Scale*)

a. Penjelasan Metode dan Skala Interpretasi

Pengujian usabilitas atau tingkat kebergunaan sistem dievaluasi secara kuantitatif menggunakan instrumen kuesioner *System Usability Scale* (SUS) yang dikembangkan oleh John Brooke. Metode ini telah digunakan secara luas dalam penelitian rekayasa perangkat

lunak untuk mengukur pengalaman pengguna (*User Experience/UX*). Instrumen SUS terdiri atas 10 butir pernyataan yang diukur menggunakan skala Likert 1 hingga 5 (Sangat Tidak Setuju hingga Sangat Setuju). Untuk mencegah bias respons, pernyataan disusun sedemikian rupa di mana pernyataan bernomor ganjil (1, 3, 5, 7, 9) memiliki sifat positif, sedangkan pernyataan bernomor genap (2, 4, 6, 8, 10) memiliki sifat negatif.

Tingkat penerimaan perangkat lunak diinterpretasikan berdasarkan skala penilaian standar SUS yang dijabarkan pada tabel berikut:

Skor SUS	Grade	Kategori
> 80,3	A	Excellent (Sangat Baik)
68 – 80,3	B	Good (Baik)
51 – 67,9	C	OK (Cukup)
25 – 50,9	D	Poor (Buruk)
< 25	F	Awful (Sangat Buruk)

b. Daftar Pernyataan Kuesioner SUS

No	Pernyataan	Sifat
P1	Saya merasa fitur pengecekan fakta berbasis AI di aplikasi Klarip ini mungkin sangat berguna untuk aktivitas saya sehari-hari.	Positif (+)
P2	Saya merasa alur penggunaan aplikasi ini terlalu rumit dan berbelit-belit untuk orang awam.	Negatif (-)
P3	Tampilan antarmuka aplikasi ini terlihat rapi, modern, dan membuat saya merasa nyaman saat menggunakannya.	Positif (+)
P4	Saya merasa butuh panduan khusus atau bantuan dari orang yang lebih paham teknologi untuk bisa menggunakan aplikasi ini.	Negatif (-)
P5	Fitur-fitur yang ada di dalam aplikasi ini (seperti pencarian berita dan riwayat analisis) berfungsi dengan sangat baik.	Positif (+)
P6	Tata letak tombol, teks, dan menu di dalam aplikasi ini terasa membingungkan serta tidak konsisten.	Negatif (-)
P7	Saya yakin sebagian besar masyarakat awam dapat memahami dan mempelajari cara menggunakan aplikasi ini dengan sangat cepat.	Positif (+)
P8	Saya merasa aplikasi ini sangat tidak praktis dan merepotkan saat digunakan untuk memverifikasi sebuah informasi.	Negatif (-)
P9	Saya merasa percaya diri dan tidak kebingungan saat pertama kali mencoba menganalisis klaim berita menggunakan aplikasi ini.	Positif (+)
P10	Saya merasa harus banyak belajar dan mencoba-coba terlebih dahulu sebelum bisa menggunakan aplikasi ini dengan lancar.	Negatif (-)

c. Profil dan Data Demografi Responden

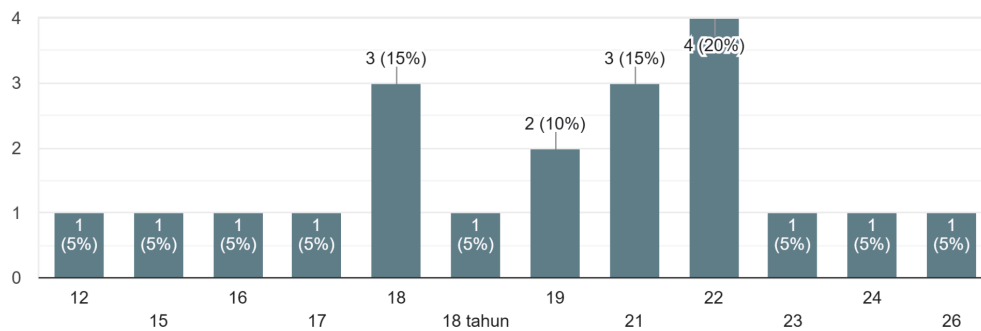
Pengambilan data pengujian usability melibatkan 20 responden yang termasuk dalam kelompok demografi pengguna media sosial aktif.

No	Nama/Inisial	Usia	Jenis Kelamin	Frekuensi Medsos	Pernah Kesulitan Cek Fakta
1	Nannindya Ainun Nisa	21	Perempuan	Sangat Sering (> 3 Jam)	Ya, sering
2	Sherly	22	Perempuan	Sangat Sering (> 3 Jam)	Ya, sering
3	Ridho Putra	19	Laki-laki	Sangat Sering (> 3 Jam)	Kadang-kadang
4	Alfarizi	21	Laki-laki	Cukup Sering (1–3 Jam)	Kadang-kadang
5	Peren Wijaya	18	Perempuan	Sangat Sering (> 3 Jam)	Kadang-kadang
6	Chysee	24	Laki-laki	Jarang (< 1 Jam)	Kadang-kadang
7	Kyoko	26	Perempuan	Sangat Sering (> 3 Jam)	Ya, sering
8	Erdi	22	Laki-laki	Sangat Sering (> 3 Jam)	Ya, sering
9	Vell	18	Perempuan	Cukup Sering (1–3 Jam)	Kadang-kadang
10	Kelly	15	Perempuan	Sangat Sering (> 3 Jam)	Kadang-kadang
11	Nathaniel Rainer Lavian	19	Laki-laki	Cukup Sering (1–3 Jam)	Ya, sering
12	Chase	22	Laki-laki	Sangat Sering (> 3 Jam)	Kadang-kadang
13	Ficishella Mewengkang	17	Perempuan	Sangat Sering (> 3 Jam)	Kadang-kadang

14	Afton	12	Laki-laki	Sangat Sering (> 3 Jam)	Kadang-kadang
15	Lumielle Viera	16	Perempuan	Sangat Sering (> 3 Jam)	Ya, sering
16	Anisa Sapitri	18	Perempuan	Sangat Sering (> 3 Jam)	Kadang-kadang
17	Dika	22	Laki-laki	Sangat Sering (> 3 Jam)	Ya, sering
18	Asell	23	Perempuan	Sangat Sering (> 3 Jam)	Ya, sering
19	Nguyen	21	Laki-laki	Sangat Sering (> 3 Jam)	Ya, sering
20	Claudya	18	Perempuan	Jarang (< 1 Jam)	Ya, sering

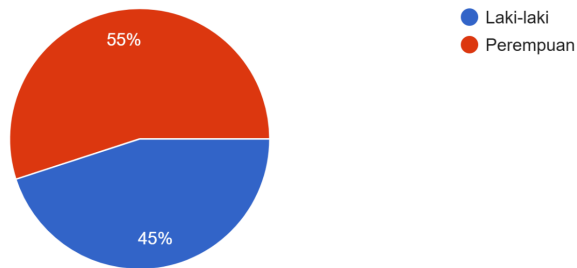
3. Usia:

20 jawaban



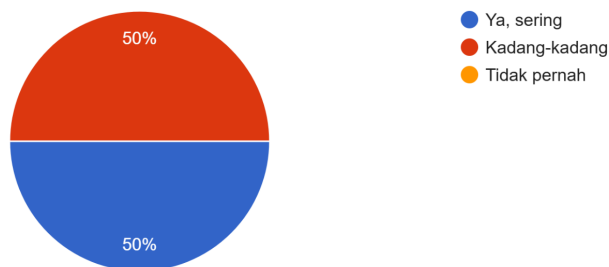
4. Jenis Kelamin:

20 jawaban



6. Pernahkah Anda merasa kesulitan memastikan kebenaran sebuah informasi/berita di media sosial?

20 jawaban



d. Hasil Kuesioner dan Perhitungan Skor SUS Per Responden

Perhitungan skor akhir SUS dilakukan melalui tahap konversi nilai dengan aturan spesifik: skor pada pernyataan positif (ganjil) dikurangi satu ($\text{Nilai} - 1$), sedangkan skor pada pernyataan negatif (genap) dikalkulasi dengan mengurangi nilai lima dengan skor jawaban ($5 - \text{Nilai}$). Seluruh skor konversi kemudian dijumlahkan dan dikalikan

dengan konstanta 2,5. Tabel berikut menyajikan hasil konversi dan kalkulasi akhir untuk seluruh responden:

No	Nama	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	Jumlah	Skor SUS
1	Nannindya	4	4	3	2	3	3	2	4	4	0	29	72,5
2	Sherly	4	3	4	3	4	4	2	4	3	1	32	80,0
3	Ridho Putra	4	3	4	0	3	0	4	4	0	0	22	55,0
4	Alfarizi	2	2	3	0	3	2	3	3	3	0	21	52,5
5	Peren Wijaya	2	4	3	3	3	4	3	3	4	3	32	80,0
6	Chysee	3	2	3	2	3	1	3	0	4	0	21	52,5
7	Kyoko	4	4	4	4	4	4	4	4	4	4	40	100,0
8	Erdi	4	4	4	3	4	2	4	0	1	0	26	65,0
9	Vell	4	3	3	4	4	3	3	2	4	2	32	80,0
10	Kelly	0	2	0	4	0	3	0	1	2	4	16	40,0
11	Nathaniel	4	3	4	3	4	3	4	3	4	3	35	87,5
12	Chase	3	3	3	4	3	3	3	1	4	3	30	75,0
13	Ficishella	4	4	4	3	4	4	4	4	4	4	39	97,5
14	Afton	2	4	4	4	4	2	4	4	2	2	32	80,0
15	Lumielle	4	4	4	4	4	4	4	4	4	4	40	100,0
16	Anisa Sapitri	2	3	2	1	3	3	1	3	3	2	23	57,5
17	Dika	4	4	4	4	4	4	4	4	4	4	40	100,0
18	Asell	4	4	4	4	4	4	4	4	4	4	40	100,0
19	Nguyen	4	4	4	2	4	3	4	2	4	2	33	82,5
20	Claudia	3	3	4	2	4	4	4	4	4	2	34	85,0

e. Rekapitulasi dan Statistik Skor SUS

No	Nama	Skor SUS	Grade	Kategori
1	Nannindya Ainun Nisa	72,5	B	Good
2	Sherly	80,0	B	Good
3	Ridho Putra	55,0	C	OK
4	Alfarizi	52,5	C	OK
5	Peren Wijaya	80,0	B	Good
6	Chysee	52,5	C	OK
7	Kyoko	100,0	A	Excellent
8	Erdi	65,0	C	OK
9	Vell	80,0	B	Good
10	Kelly	40,0	D	Poor
11	Nathaniel Rainer Lavian	87,5	A	Excellent
12	Chase	75,0	B	Good
13	Ficishella Mewengkang	97,5	A	Excellent
14	Afton	80,0	B	Good
15	Lumielle Viera	100,0	A	Excellent
16	Anisa Sapitri	57,5	C	OK
17	Dika	100,0	A	Excellent
18	Asell	100,0	A	Excellent
19	Nguyen	82,5	A	Excellent
20	Claudya	85,0	A	Excellent
	RATA-RATA	78,25	B	Good

f. Ringkasan Statistik SUS

Statistik	Nilai
Rata-rata Skor SUS	78,25
Skor Tertinggi	100,0 (Kyoko, Lumielle, Dika, Asell)
Skor Terendah	40,0 (Kelly)
Grade Keseluruhan	B (Good)

Berdasarkan hasil pengujian *System Usability Scale* (SUS) yang melibatkan 20 responden pengguna aktif media sosial, diperoleh rata-rata skor SUS sebesar **78,25**. Berdasarkan skala interpretasi SUS, skor tersebut berada pada rentang **68–80,3** yang secara spesifik dikategorikan sebagai **Grade B** atau masuk ke dalam kategori penerimaan **Good (Baik)**.

Tinjauan terhadap distribusi hasil evaluasi menunjukkan bahwa 8 responden (40%) memberikan penilaian pada kategori *Excellent*, 6 responden (30%) pada kategori *Good*, 5 responden (25%) pada kategori *OK*, dan 1 responden (5%) pada kategori *Poor*. Tidak ditemukan

satupun responden yang memberikan penilaian pada ambang batas terendah (*Awful*).

Hasil kuantitatif ini secara empiris mengindikasikan bahwa aplikasi Klarip telah memiliki tingkat usability yang baik dan dapat diterima secara positif oleh pengguna, terutama dalam aspek kemudahan interaksi antarmuka, navigasi aplikasi yang modern, dan utilitas fitur verifikasi klaim berbasis kecerdasan buatan. Pencapaian nilai skor yang mendekati ambang batas *Excellent* ($> 80,3$) merepresentasikan bahwa secara mayoritas, pengguna mampu mengoperasikan fungsionalitas inti perangkat lunak secara intuitif tanpa memerlukan modul panduan teknis secara khusus.

Meskipun perangkat lunak secara keseluruhan dinilai memiliki fungsionalitas yang mumpuni, masih terdapat sejumlah ruang untuk penyempurnaan, terutama guna mengakomodasi segmen pengguna yang belum terbiasa dengan karakteristik interaksi aplikasi berbasis penalaran linguistik (AI). Hal ini terefleksikan melalui kemunculan sejumlah kecil skor pada batas tengah dan bawah. Oleh karena itu, penyempurnaan pada siklus pengembangan di masa mendatang dapat direalisasikan

melalui integrasi fitur panduan interaktif layar (*onboarding tutorial*) pada fase inisialisasi aplikasi (*first-time use*) untuk mempercepat kurva adaptasi pengguna secara komprehensif.

BAB V

PENUTUP

A. Kesimpulan

Penelitian ini bertujuan untuk mengembangkan aplikasi berbasis *android* bernama **Klarip** yang memanfaatkan layanan *Google Custom Search Engine* (CSE) dan Gemini AI sebagai sarana cepat verifikasi klaim informasi yang beredar di media sosial. Pengembangan dilakukan menggunakan model desain instruksional (*Instructional Design*) SDLC (*Software Development Life Cycle*) dengan framework Flutter sebagai kerangka kerja utama. Berdasarkan hasil penelitian dan pengujian yang telah dilakukan, dapat ditarik kesimpulan sebagai berikut:

Aplikasi Klarip berhasil dikembangkan dengan mengimplementasikan empat modul fungsional utama, yaitu: (a) sistem autentikasi pengguna berbasis basis data SQLite lokal; (b) sistem verifikasi klaim menggunakan teknik *Retrieval-Augmented Generation* (RAG) yang mengintegrasikan Google CSE dan Gemini AI; (c) sistem manajemen riwayat analisis dengan operasi CRUD (*Create, Read, Update, Delete*) lengkap; serta (d) sistem pengaturan antarmuka dan kunci antarmuka pemrograman aplikasi (*API key*) yang dapat dikustomisasi oleh pengguna.

Hasil pengujian *Black Box Testing* yang dilakukan terhadap 50 skenario uji yang mencakup enam modul fungsional meliputi autentikasi,

verifikasi klaim, manajemen riwayat, profil pengguna, pengaturan API, dan navigasi antarmuka menunjukkan bahwa seluruh skenario (100%) berhasil berjalan sesuai dengan hasil yang diharapkan. Tidak ditemukan adanya kegagalan fungsi (*bug*) pada satupun skenario yang diujikan. Hal ini membuktikan bahwa aplikasi Klarip telah memenuhi seluruh kebutuhan fungsional yang telah dirancang.

Pengukuran efisiensi waktu pemrosesan (*Time on Task*) yang dilakukan melalui 20 percobaan dengan menggunakan klaim-klaim hoaks politik yang pernah beredar di media sosial Indonesia menunjukkan perbedaan yang sangat signifikan. Metode pencarian manual melalui peramban web membutuhkan waktu rata-rata 366,2 detik (± 6 menit 6 detik) per verifikasi, sementara aplikasi Klarip hanya membutuhkan waktu rata-rata 7,7 detik. Berdasarkan perhitungan matematis, aplikasi Klarip terbukti $\pm 47,6$ kali lebih cepat dengan tingkat penghematan waktu sebesar 97,9%. Hasil ini mengonfirmasi bahwa otomatisasi sistem verifikasi berbasis kecerdasan buatan mampu memangkas durasi penelusuran fakta secara esensial.

Hasil evaluasi *System Usability Scale* (SUS) yang melibatkan 20 responden dari kelompok pengguna aktif media sosial menghasilkan rata-rata skor SUS sebesar 78,25. Skor tersebut berada pada rentang 68 hingga 80,3 yang secara spesifik dikategorikan sebagai *Grade B (Good / Baik)* berdasarkan skala interpretasi standar SUS. Sebanyak 40%

responden memberikan penilaian pada kategori *Excellent* dan 30% pada kategori *Good*, yang mengindikasikan bahwa mayoritas pengguna (70%) menilai aplikasi ini sudah sangat mumpuni dalam aspek kemudahan penggunaan, kenyamanan desain antarmuka, dan kegunaan fitur utamanya.

B. Implikasi

Hasil penelitian ini memberikan kontribusi yang bermakna dan memunculkan beberapa implikasi penting, baik dari aspek akademis, teknologi, maupun sosial kemasyarakatan:

Penelitian ini membuktikan secara empiris bahwa teknik *Retrieval-Augmented Generation* (RAG) yang menggabungkan subsistem pengambilan informasi (*information retrieval*) dengan model bahasa besar (*large language model*) dapat diimplementasikan secara efektif dalam konteks pengecekan fakta berbahasa Indonesia. Pendekatan ini membuka peluang bagi penelitian lanjutan dalam bidang *Natural Language Processing* (NLP), jurnalisme komputasional (*computational journalism*), dan interaksi manusia-komputer (*Human-Computer Interaction/HCI*) di lingkungan akademik.

Dari perspektif rekayasa perangkat lunak, penelitian ini mendemonstrasikan bahwa integrasi layanan Google CSE dan Gemini AI dapat dieksekusi secara otonom dari sisi klien (*client-side*) tanpa memerlukan infrastruktur peladen (*server*) yang kompleks dan berbiaya tinggi. Pola arsitektur ini menawarkan alternatif yang ekonomis dan

berskala luas (*scalable*) untuk pengembangan aplikasi utilitas, khususnya bagi pengembang independen dengan sumber daya terbatas.

Kehadiran aplikasi Klarip memiliki potensi destruktif yang positif dalam meningkatkan literasi digital masyarakat, khususnya bagi generasi muda (*Generasi Z*) yang aktif mendistribusikan informasi di ekosistem digital. Pengurangan durasi verifikasi dari enam menit menjadi kurang dari delapan detik secara fundamental berhasil menurunkan hambatan psikologis (*cognitive friction*) bagi pengguna awam untuk melakukan pengecekan fakta sebelum membagikan ulang sebuah tautan. Implementasi perangkat lunak ini sejalan dengan upaya mitigasi penyebaran disinformasi yang tengah digalakkan oleh berbagai organisasi masyarakat sipil di Indonesia.

C. Saran

Berdasarkan temuan data, observasi, dan keterbatasan dalam penelitian ini, peneliti merumuskan sejumlah saran untuk perbaikan dan pengembangan di masa mendatang:

Dukungan Multibahasa: Pengembangan iterasi berikutnya diharapkan dapat mengakomodasi dukungan pemrosesan bahasa daerah (seperti bahasa Jawa atau Sunda) guna memperluas jangkauan deteksi misinformasi di tingkat akar rumput.

Pembuatan Versi iOS: Perangkat lunak saat ini baru dikompilasi untuk ekosistem Android. Mengingat *Flutter* adalah kerangka kerja lintas

platform, kompilasi aplikasi untuk sistem operasi iOS perlu diwujudkan guna menjangkau demografi pengguna Apple di Indonesia.

D. Penutup

Penelitian ini telah berhasil menyelesaikan proses perancangan, pengembangan, hingga pengujian perangkat lunak "Klarip" sebagai instrumen inovatif dan efisien dalam memfasilitasi masyarakat untuk memverifikasi kebenaran informasi di media sosial. Walaupun masih memiliki batasan dalam skalabilitas platform, temuan empiris pada penelitian ini membuktikan bahwa pemanfaatan kecerdasan buatan generatif—yang dikawinkan dengan arsitektur pencarian web dinamis—memiliki daya ungkit yang sangat besar dalam memerangi epidemi disinformasi dan hoaks di era pasca-kebenaran (*post-truth era*).

DAFTAR PUSTAKA

- Android Developers. (2025). *Mengenal Android Studio*. Diakses dari <https://developer.android.com/studio/intro>
- Borg, W. R., & Gall, M. D. (1983). *Educational Research: An Introduction* (4th ed.). Longman.
- Dzulfikar, M. A. A., & Asriningtias, Y. (2025). Aplikasi Inventory dengan Visualisasi Penjualan Berbasis Outflow Stok pada Toko Ritel. *TIN: Terapan Informatika Nusantara*, 6(7), 841–850.
- Effendi, E. (2023). User Behavior and Hoax Information on Social Media: Case of Indonesia. *Jurnal Studi Komunikasi*.
- Gemini Team. (2024). *Gemini: A Family of Highly Capable Multimodal Models*. Google DeepMind Technical Report. Publikasi resmi *update* Gemini dari DeepMind. Diakses dari <https://arxiv.org/abs/2312.11805>
- Google DeepMind. (2025). *Gemini 2.5: Pushing the Frontier with Advanced Reasoning and Agentic Capabilities*. Publikasi resmi *update* Gemini dari DeepMind. diakses dari <https://deepmind.google/discover/blog/gemini-2-5/>
- Google Developers. (2025). *Custom Search JSON API: Introduction*. Diakses dari

Guo, Z., Schlichtkrull, M., & Vlachos, A. (2022). A Survey on Automated Fact-Checking. *Transactions of the Association for Computational Linguistics*, 10, 178–206.

International Organization for Standardization. (2018). *Ergonomics of human-system interaction — Part 11: Usability: Definitions and concepts* (ISO 9241-11:2018).

Laugwitz, B., Held, T., & Schrepp, M. (2008). Construction and Evaluation of a User Experience Questionnaire. *HCI and Usability for Education and Work*, 63–76.

Li, X., Zhang, Y., & Malthouse, E. C. (2024). Large language model agentic approach to fact checking and fake news detection. *Proceedings of the 27th European Conference on Artificial Intelligence*.

Purnamasari, D., & Panjaitan, F. (2020). Pengembangan Aplikasi Mobile Berbasis Android Sebagai Media Informasi. *Jurnal Sistem Informasi*.

Putri, A. S., Eviyanti, A., & Hindarto, H. (2023). Rancang Bangun Aplikasi E-Commerce Berbasis Android Pada Toko Suryamart Menggunakan Framework Flutter. *Jurnal Teknologi Dan Sistem Informasi Bisnis*, 5(3), 257–265.

Lischka, J. A., & Garz, M. (2023). Clickbait news and algorithmic curation: A game theory framework of the relation between journalism, users, and platforms. *New Media & Society*, 25(8), 2073–2094.

- Setiawan, P. R., Ramadhan, R. A., & Labellapansa, A. (2022). Pelatihan Pemrograman Flutter. *Jurnal Pengabdian Masyarakat Dan Penerapan Ilmu Pengetahuan*, 3(1), 22–27.
- Sugiyono. (2019). *Metode Penelitian Kuantitatif, Kualitatif, dan R&D*. Bandung: Alfabeta.
- Supiana. (2022). *Mengenal Bahasa Pemrograman Dart untuk Pemula*. IT Press.
- Wijayanto, Berenschot, W., & Fahmi, I. (2021). *Cybertroops and public opinion manipulation through social media in Indonesia*. LP3ES & KITLV.
- Zelixify. (2025). *Apa Itu Flutter? Pengertian, Fungsi, dan Kelebihan Dibanding Framework Lain*.
<https://zelixify.web.id/blog/apa-itu-flutter-pengertian-fungsi-dan-kelebihan-dibanding-framework-lain>
- Thorne, J., Vlachos, A., Christodoulopoulos, C., & Mittal, A. (2018). FEVER: a Large-scale Dataset for Fact Extraction and VERification. *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*.
- Schlichtkrull, M., Guo, Z., & Vlachos, A. (2023). AVeriTeC: A Dataset for Real-world Claim Verification with Evidence from the Web. *Proceedings of the 37th Conference on Neural Information Processing Systems (NeurIPS 2023)*.

- Rahman, S. S., Islam, M. A., Alam, M. M., et al. (2025). Hallucination to truth: a review of fact-checking and factuality evaluation in large language models. *Artificial Intelligence Review*.
- Zeng, J., Abumansour, A. S., & Zubiaga, A. (2021). Automated fact-checking: A survey. *Language and Linguistics Compass*, 15(10), e12438.
- Risko, E. F., & Gilbert, S. J. (2016). Cognitive offloading. *Trends in Cognitive Sciences*, 20(9), 676-688.
- Pérez-Curiel, C., Velasco Molpeceres, A. M., & Rúas-Araújo, J. (2021). Journalistic Fact-Checking of Information in Pandemic: Stakeholders, Hoaxes, and Strategies to Fight Disinformation during the COVID-19 Crisis in Spain. *International Journal of Environmental Research and Public Health*, 18(4), 2009.
- Cavoukian, A. (2009). *Privacy by design: The 7 foundational principles*. Information and Privacy Commissioner of Ontario, Canada.
- Shi, W., Cao, J., Zhang, Q., Li, Y., & Xu, L. (2016). Edge computing: Vision and challenges. *IEEE Internet of Things Journal*, 3(5), 637-646.
- Fowler, M. (2003). *UML Distilled: A Brief Guide to the Standard Object Modeling Language* (3rd ed.). Addison-Wesley Professional.

Object Management Group (OMG). (2017). *OMG Unified Modeling Language (OMG UML) Version 2.5.1*.

<https://www.omg.org/spec/UML/2.5.1/PDF>

Sommerville, I. (2015). *Software Engineering* (10th ed.). Pearson.

Brooke, J. (1996). SUS: A "quick and dirty" usability scale. Dalam P. Jordan, B. Thomas, B. Weerdmeester, & I. L. McClelland (Editor), *Usability evaluation in industry* (hlm. 189–194). Taylor & Francis.

Lewis, J. R., & Sauro, J. (2009). The factor structure of the system usability scale. *International Conference on Human Centered Design*, 94–103.

https://doi.org/10.1007/978-3-642-02806-9_12

Nielsen, J. (1993). *Usability engineering*. Academic Press.

Nielsen, J., & Landauer, T. K. (1993). A mathematical model of the finding of usability problems. *Proceedings of the INTERACT'93 and CHI'93 Conference on Human Factors in Computing Systems*, 206–213.

<https://doi.org/10.1145/169059.169166>

Tullis, T. S., & Stetson, J. N. (2004). A comparison of questionnaires for assessing website usability. *Usability Professional Association Conference*, 1–12.

Bangor, A., Kortum, P. T., & Miller, J. T. (2008). An empirical evaluation of the System Usability Scale. *International Journal of Human-Computer Interaction*, 24(6), 574–594. <https://doi.org/10.1080/10447310802205776>

Sauro, J. (2011). *A practical guide to the System Usability Scale: Background, benchmarks & best practices*. Measuring Usability LLC.

Sauro, J., & Lewis, J. R. (2012). *Quantifying the user experience: Practical statistics for user research*.

Albert, W., & Tullis, T. (2013). *Measuring the user experience: Collecting, analyzing, and presenting usability metrics* (2nd ed.).

Karamoy, W. A. G., & Wibowo, R. A. (2025). Pengembangan aplikasi mobile untuk monitoring stok dan peringatan stok tipis bagi UMKM. *JUTEKOBID: Jurnal Teknologi dan Bisnis*, 153(44).

Kurnia, D. (2025). Pengembangan aplikasi mobile pengelola data sarana berbasis Android Studio dengan fitur CRUD menggunakan SQLite. *Jurnal GIT*, 3(2), 32-38.

Suhery, L., & Dilson, D. (2021). Model dan prototype aplikasi penerimaan mahasiswa baru (PMB) berbasis mobile menggunakan framework Flutter. *Jurnal SIMTIKA*, 4(2), 29-35.

Setiawan, R., & Lazuardi, M. (2023). Evaluasi usability aplikasi mobile menggunakan pendekatan task-based scenario dan System Usability Scale. *Jurnal Sistem Informasi dan Ilmu Komputer*, 7(2), 112-120.

Nurfaldini, S. I., Alfarizi, M. F. R., & Kuncoro, P. D. H. (2024). Analisis User Experience pada Aplikasi Mobile SIA Universitas Teknologi

Yogyakarta Dengan Metode User Experience Questionnaire. *Teknomatika: Jurnal Informatika dan Komputer*, 17(1), 48–55.

Rivansyah, M., Arifianto, D., & Suharso, W. (2023). Analisis user interface dan user experience pada SIA mobile UM Jember dengan metode User Experience Questionnaire. *JUSTINDO (Jurnal Sistem dan Teknologi Informasi Indonesia)*, 8(2), 108–115.

Saputri, S., Chairunisa, E. D., & Suryani, I. (2024). Pengembangan media pembelajaran digital nilai-nilai sejarah candi bumi ayu berbasis android. *FACTUM: Jurnal Sejarah dan Pendidikan Sejarah*, 13(1), 23-30.

Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.

Mufarroha, A. (2023). Usability Testing and User Interface Improvement of Mobile Banking Application. *Saintek: E-Journal UIN Malang*.

Schrepp, M. (2023). *User Experience Questionnaire Handbook*. Diterbitkan secara independen. (Dapat diakses di: www.ueq-online.org)

Nawansa, R. A., & Pratama, D. (2024). Evaluasi kualitas pengalaman pengguna menggunakan *User Experience Questionnaire*. *TEKNOFILE: Jurnal Sistem Informasi*, 2(10), 795–802.

Wijaya, K., Setiawan, A., & Hidayat, M. (2024). Analisis pengalaman pengguna pada aplikasi *mobile* menggunakan metode *User Experience Questionnaire*

(UEQ) dan *Usability*. *Jurnal Pengembangan Teknologi Informasi dan Ilmu Komputer (J-PTIIK)*, 8(2), 1120–1128.

Arifin, Z., & Susanto, A. (2023). Pengujian aplikasi mobile menggunakan metode *black box testing* checklist berdasarkan efektivitas sistem. *Jurnal Teknoinfo*, 17(1), 125–132.

Saputra, R. D., & Hidayat, T. (2024). Analisis efisiensi waktu pemrosesan algoritma sistem otomatisasi terintegrasi kinerja aplikasi. *Jurnal Media Informatika Budidarma*, 8(2), 540–547.

Pratama, A., & Nugraha, S. (2022). Analisis *usability* aplikasi menggunakan metode *System Usability Scale* (SUS). *Jurnal RESTI (Rekayasa Sistem dan Teknologi Informasi)*, 6(3), 415–422.

Pradnyana, I. G. A. A., & Rusjyanthi, N. K. D. (2023). Analisis Usability dan Pengujian Fungsionalitas Sistem Informasi Menggunakan Metode Usability Testing. *Jurnal Nasional Teknologi Informasi dan Aplikasinya (JNATIA)*, 1(3), 845-852.

LAMPIRAN
BIODATA MAHASISWA



Nama : Muhamad Alfath Arrahman

Tempat, Tanggal Lahir : Depok, 17 September 2003

Jenis Kelamin : Laki-laki

NIM : 2290343055

Program Studi : Teknologi Rekayasa Multimedia

No. HP : 089 6348 67973

Email : alfath.aracman@gmail.com

Alamat : Jl. Pramuka Raya, Kec. Pancoran
Mas, Kota Depok , RT.03,RW.11

Perguruan Tinggi : Politeknik Negeri Media Kreatif